Timeless Computing

Timeless Computing

A brief introduction to computing, FPGAs and the dream of timeless computers.

Clio Grai

2026 Edition

First published 7 June 2025

2026 Edition, revision 1 (October 2026).

© 2025-2026 Lone Dynamics Corporation. All rights reserved.

This book is released under the Creative Commons CC0 1.0 Universal Public Domain Dedication. You are free to copy, modify, and distribute this work for any purpose, without asking permission.

`https://creativecommons.org/publicdomain/zero/1.0/`

Bibliographic information from the German National Library: The German National Library lists this publication in the German National Bibliography; detailed bibliographic data is available online at `dnb.dnb.de`.

Publisher: BoD · Books on Demand GmbH, Überseering 33, 22297 Hamburg, bod@bod.de

Printing: Libri Plureos GmbH, Friedensallee 273, 22763 Hamburg

ISBN: 978-3-6967-2160-2

Contents

I Why **1**

1 Timeless Computing **3**
1.1 The Problem 3
1.2 Timeless Applications 4
1.3 Principles 4
1.4 Many Possible Machines 5
1.5 Why FPGAs 6
1.6 Putting the Ideas Into Practice 7
1.7 Survival Computing 8
1.8 About This Book 8
1.9 How to Read This Book 9

II Foundations **11**

2 Electricity **13**
2.1 Charge and Current 13
2.2 Voltage 14
2.3 Resistance and Ohm's Law 14
2.4 Power 15
2.5 Capacitors and Time 15
2.6 Direct and Alternating Current 16
2.7 Power Supplies and Batteries 16
2.8 Components 18
2.9 The Transistor as a Switch 18
2.10 Logic Levels 19
2.11 Floating Inputs and Pull-ups 20
2.12 Reading a Schematic 20
2.13 Reading a Data Sheet 21

2.14 Working Safely 22
2.15 Making Connections 23
2.16 Measuring and Finding Faults 24

3 Information **25**
3.1 Bits 25
3.2 Counting in Binary 25
3.3 Bytes and Words 26
3.4 Hexadecimal 27
3.5 Negative Numbers 27
3.6 Overflow 28
3.7 Fractions 28
3.8 Text 30
3.9 Pictures and Sound 31
3.10 Units of Size 31
3.11 Meaning Comes From Use 32

4 Digital Logic **33**
4.1 Logic Gates 33
4.2 Boolean Algebra 34
4.3 Everything From NAND 34
4.4 Combinational Circuits 35
4.5 Propagation Delay 36
4.6 Sequential Circuits 36
4.7 The Clock and the Flip-Flop 37
4.8 Registers, Counters and Shift Registers 38
4.9 State Machines 39
4.10 Timing 39
4.11 From Logic to a Processor 40

III How Computers Work **41**

5 The Computer **43**
5.1 The Parts of a Computer 43
5.2 Inside the CPU 44
5.3 The Instruction Set 44
5.4 Fetch, Decode, Execute 45
5.5 Machine Code and Assembly 46
5.6 Memory and Addresses 47
5.7 Subroutines and the Stack 47
5.8 The Bus and the Address Map 48

5.9 Memory-Mapped I/O 48
5.10 Interrupts 49
5.11 Direct Memory Access 50
5.12 Caches 50
5.13 Speed 51

6 Memory and Storage 53
6.1 The Memory Hierarchy 53
6.2 Static RAM 54
6.3 Dynamic RAM 54
6.4 Read-Only Memory and Its Descendants 55
6.5 Flash Memory 55
6.6 Ferroelectric RAM 56
6.7 Other Long-Term Media 57
6.8 Errors and How to Catch Them 57
6.9 Filesystems 58
6.10 Surviving Power Loss 58
6.11 Choosing Storage That Lasts 59

7 Talking to the World 61
7.1 General-Purpose Pins 61
7.2 Analog Signals 62
7.3 Serial and Parallel 63
7.4 UART 63
7.5 SPI 64
7.6 I2C 64
7.7 Protocols on Top of Buses 65
7.8 USB 66
7.9 Video 66
7.10 Audio 67
7.11 Keyboards and Mice 67
7.12 Connectors and Modules 68
7.13 Connecting a Device 68

8 Operating Systems 71
8.1 When an Operating System Is Needed 71
8.2 Booting 72
8.3 Processes 73
8.4 Scheduling 73
8.5 Protecting Memory 74
8.6 System Calls 75
8.7 Drivers 75

8.8 Files and Names 76
8.9 Communication Between Processes 76
8.10 Doing Several Things at Once 77
8.11 Kernel Designs 78
8.12 User Interfaces 78
8.13 Simplicity as a Feature 79

9 Communication **81**
9.1 A Link Between Two Machines 81
9.2 Packets 82
9.3 Layers 82
9.4 The Link Layer 83
9.5 IP: Addresses and Routing 83
9.6 Transport: TCP and UDP 84
9.7 Applications 84
9.8 Secure Connections 85
9.9 Networks Without the Internet 85

10 Security **87**
10.1 Threat Models 87
10.2 Physical Access 88
10.3 Passwords and Keys 88
10.4 What Cryptography Provides 89
10.5 Hash Functions 89
10.6 Symmetric Encryption 90
10.7 Public-Key Cryptography 90
10.8 Random Numbers 91
10.9 The One-Time Pad 92
10.10 Quantum Computers 93
10.11 Deleting Data 93
10.12 Secure Software 94
10.13 Copies 95
10.14 Simplicity and Security 95

IV Programming **97**

11 Thinking in Programs **99**
11.1 Why Program 99
11.2 Algorithms 100
11.3 State 100
11.4 Control Flow 101

11.5 Functions . . . 101
11.6 Data Structures . . . 102
11.7 Efficiency . . . 102
11.8 Compilers and Interpreters . . . 103
11.9 Mistakes . . . 103
11.10 Testing . . . 104
11.11 Writing for People . . . 104
11.12 Keeping Track of Changes . . . 105
11.13 Three Languages . . . 105

12 BASIC **107**
12.1 Machdyne BASIC . . . 108
12.2 Lines and Commands . . . 108
12.3 A First Program . . . 109
12.4 Numbers and Arithmetic . . . 109
12.5 Decisions and Loops . . . 110
12.6 Subroutines . . . 111
12.7 Input . . . 111
12.8 Files . . . 112
12.9 Pins . . . 112
12.10 Talking to a Host . . . 114
12.11 Starting by Itself . . . 114
12.12 One Language, Many Machines . . . 114

13 Scheme **115**
13.1 Machdyne Scheme . . . 115
13.2 Expressions . . . 116
13.3 Definitions . . . 116
13.4 Choices . . . 116
13.5 Numbers . . . 117
13.6 Recursion . . . 117
13.7 Tail Calls . . . 118
13.8 Lists . . . 118
13.9 Association Lists . . . 119
13.10 Functions as Values . . . 120
13.11 Closures . . . 120
13.12 Memory . . . 121
13.13 Code Is Data . . . 121
13.14 Scheme on a Machine . . . 122

14 C **123**
14.1 A Complete Program . . . 123

14.2 Compiling 125
14.3 Types 125
14.4 Pointers 126
14.5 Arrays and Strings 126
14.6 Structures and Memory 127
14.7 Talking to Hardware 128
14.8 Writing C Well 129
14.9 Using the Three Together 129

V Changeable Hardware 131

15 Inside an FPGA 133
15.1 The Big Picture 134
15.2 Look-Up Tables 134
15.3 Flip-Flops and Carry Chains 135
15.4 Routing 136
15.5 Block RAM and DSP Blocks 136
15.6 I/O Blocks 136
15.7 Clocks 137
15.8 Clock Domains 137
15.9 Configuration 138
15.10 FPGA Capacity 139
15.11 FPGA Families and Open Tools 139
15.12 FPGAs, Processors and Custom Chips 140

16 Describing Hardware 141
16.1 A Different Kind of Language 141
16.2 Verilog Modules 142
16.3 Numbers and Vectors 142
16.4 Registers and Clocks 143
16.5 Two Kinds of Assignment 143
16.6 State Machines in Verilog 144
16.7 Simulation 145
16.8 From Description to Bitstream 146
16.9 Reading the Reports 147
16.10 Debugging on the Board 147
16.11 Classic Mistakes 148

17 Systems on a Chip 151
17.1 Soft Processors 151
17.2 The Shape of an SOC 152

17.3 Adding a Device, End to End 153
17.4 Hardware and Software Together 155
17.5 Moving a Design to Another Board 155
17.6 Linux on an FPGA 156
17.7 Recreating Old Machines 156

18 Small Computers **159**
18.1 Microcontrollers 159
18.2 An Interpreter on the Chip 160
18.3 Modules 161
18.4 The Sechs Interface 161
18.5 Building Systems from Modules 162
18.6 Portability: Machines and Languages 163
18.7 Module Lifetime 163

VI The Machine **165**

19 The Machine **167**
19.1 What It Is 167
19.2 The Hardware 168
19.3 The System on the Chip 168
19.4 Starting Up 170
19.5 The Operating System 170
19.6 The Languages 171
19.7 Messages Between Machines 172
19.8 Adapting the Machine 173
19.9 An Example Adaptation 174
19.10 Getting Back to Safety 174
19.11 Where to Go Next 175

20 Computing That Lasts **177**
20.1 Power Supply 177
20.2 The Environment 178
20.3 Spares and Parts 179
20.4 Data 179
20.5 Software and the Means to Rebuild It 180
20.6 Keeping Information 181
20.7 Licences 181
20.8 People 182
20.9 A Checklist 182
20.10 Where to Begin 183

VII Appendices **185**

A Glossary **187**

B A Short History **197**
B.1 From Electricity to Electronics 197
B.2 Machines That Calculate 198
B.3 The Stored Program 198
B.4 Smaller and Cheaper 199
B.5 Networks 199
B.6 Growth and Complexity 200
B.7 Open Designs 200
B.8 Timeline 200
B.8.1 Discovering Electricity 200
B.8.2 Electronics 201
B.8.3 Computers 202
B.8.4 Open Hardware 203

C Tables **205**
C.1 ASCII Characters 205
C.2 Control Characters 206
C.3 Powers of Two 207
C.4 Unit Prefixes 207
C.5 Electrical Formulas 207
C.6 Resistor Colour Code 208
C.7 Surface-Mount Markings 208
C.8 Standard Resistor Values 209
C.9 Logic Level Thresholds 209

D Three Languages Side by Side **211**
D.1 Greatest Common Divisor 211
D.2 Factorial 212
D.3 A Conversion Table 213
D.4 The Collatz Sequence 213
D.5 Common Constructs 214

E Language Reference **215**
E.1 Machdyne BASIC 1 215
E.1.1 BASIC Statements 215
E.1.2 BASIC Limits 216
E.1.3 BASIC Functions 217
E.1.4 BASIC Operators 217

E.1.5 BASIC Pin Modes 217
E.1.6 BASIC Commands 218
E.1.7 BASIC Error Messages 218
E.2 Scheme Reference 219
E.2.1 Scheme Special Forms 219
E.2.2 Scheme Standard Procedures 220
E.3 C Reference 220
E.3.1 C Keywords 221
E.3.2 C Operators 221
E.3.3 C Standard Headers 221
E.3.4 printf Conversions 222
E.3.5 C Escape Sequences 222
E.4 Verilog Reference 222
E.4.1 Verilog Constructs 223
E.4.2 Verilog Numbers and Operators 223

F RISC-V Reference 225
F.1 Registers 225
F.2 Instruction Formats 226
F.3 RV32I 226
F.3.1 Arithmetic and Logic 227
F.3.2 Loads and Stores 227
F.3.3 Branches and Jumps 228
F.3.4 System 228
F.4 The M Extension 228
F.5 Pseudo-instructions 229

G Connector Pinouts 231
G.1 Pmod 231
G.2 Sechs 232
G.3 Zwölf 232
G.4 Werkzeug 233
G.5 Raspberry Pi GPIO 234
G.6 Serial Connectors 235
G.7 USB Connectors 236
G.8 Ethernet Cables 236
G.9 VGA Connector 237
G.10 HDMI Connector 237
G.11 SD Cards 238

H Sources 239
H.1 Machdyne 239

H.2 Open Hardware Tools 240
H.3 Standards 240
H.4 Further Reading 241

Part I

Why

Chapter 1

Timeless Computing

Over the past fifty years, computers have become steadily faster, cheaper and more capable. Over the same period, they have become more difficult to understand, to secure and to maintain. **Timeless computing** is an approach to computing that gives priority to longevity, simplicity and understanding. Its aim is a computer that remains useful for decades, and whose owner understands how it works and can adapt it as needs change. This book explains that approach and the knowledge needed to apply it.

1.1 The Problem

Modern computers present several difficulties despite their capabilities.

Their software is large and built in layers. Each layer adds a delay between an action, such as a key press, and its result, so in some ways a modern computer can feel much slower than a much older one.

Their software is too large for any one person to read. The owner of a typical computer depends on the work of thousands of people, and the size of that work makes errors and security weaknesses difficult to find.

Their hardware is usually replaced long before it wears out. Soft-

ware grows to use the capacity of newer hardware, and older hardware that still works correctly becomes unable to run current software.

As a result, the owner of a computer often ends up serving it, by installing updates, replacing hardware that still works and relearning software that has changed, instead of the computer serving the owner as a tool.

1.2 Timeless Applications

Many of the tasks computers are used for have changed little in a century and are likely to change little in the next:

- reading
- writing
- storing information
- communication
- mathematics
- organization
- education
- the automation of machines
- programming

None of these requires the newest hardware. A writer requires a responsive editor and reliable storage. A controller for a water pump requires a small, dependable device that performs one task for many years. Some people still write on computers that are decades old, or on typewriters, because those tools serve the task and nothing else.

A timeless computer occupies a position between pen and paper and the modern workstation. It is more capable than the former and much simpler than the latter, and it is designed for these applications in particular.

1.3 Principles

A computer intended to last should have the following properties. Few machines have all of them, but each makes a machine more likely to remain useful.

- **Purpose.** It is designed for timeless applications.

- **Longevity.** It remains useful for a century or more. Ideally this holds in both directions: a person from a century ago, given power and the documentation, could also learn to use it.
- **Recoverability.** It can always be returned to a known working state.
- **Durability.** It withstands reasonable wear and handling.
- **Replaceable parts.** Components with a limited lifetime, such as flash memory and moving parts, can be replaced by the owner.
- **Repairability.** It can be assembled and repaired with ordinary tools.
- **Long-term storage.** It can retain data for decades or centuries, using memory chosen for that purpose.
- **Simplicity.** It is no more complex than its purpose requires, and it includes its own documentation.
- **Preservation.** Its designs, software and documentation are publicly archived in more than one place.
- **Modularity.** Its functions can be extended with additional modules, rather than by replacing the whole machine.
- **Openness.** Its hardware and software designs are open to inspection, repair and modification.

Openness supports the other principles. A design that can be read can be checked, repaired and rebuilt by others, even after its original makers are gone.

1.4 Many Possible Machines

Timeless computing describes a way of choosing and using computers, not a particular product. Its principles can be applied to many kinds of hardware.

An older computer kept in working order, with its manuals and spare parts, already satisfies several of them. So does a conventional PC running a small, well-understood operating system with plain text files, or a microcontroller board that performs one task for twenty years. An inexpensive FPGA development board, of the kind sold to students, can run a complete open computer design. Each choice satisfies some principles better than others. An older computer is simple, but its parts are no longer made. A PC is easy to obtain, but its hardware is closed and its parts are not

designed to last.

The material in this book applies to all of these.

1.5 Why FPGAs

One kind of hardware is particularly well suited to these principles: the **field-programmable gate array**, or FPGA.

The processor in a conventional computer is fixed when the chip is manufactured. An FPGA instead consists of a large array of identical small circuits, called logic blocks, whose functions and connections are set by a configuration file. The file is stored in flash memory, which keeps its contents without power, and is loaded into the FPGA each time it is switched on. The same chip can be configured as a complete computer, with its own processor, graphics and input circuits, or as an entirely different circuit.

This has two consequences. First, the complete design of the computer, including its processor, exists as readable text, so no part of the computer's own design is hidden, although the FPGA that carries it is a manufactured chip. Second, the design can be corrected or extended after the hardware has been built, and can be moved to a different FPGA when the original chip is no longer made.

The principal disadvantage is speed. An FPGA computer runs at tens of megahertz (millions of steps per second), rather than the thousands of megahertz of a modern processor, which is comparable to a desktop computer of the early 1990s. Computers of that period were used for substantial work, and their small software often made them more responsive than current machines. The low power consumption of an FPGA computer is also an advantage where power comes from a battery or a solar panel.

The chapter Inside an FPGA explains how FPGAs work.

1.6 Putting the Ideas Into Practice

Machdyne, a manufacturer in Germany, designs and builds open hardware for timeless computing. Its products appear in this book as examples of general ideas:

- **Computers** built around FPGAs from several manufacturers, with memory, video output, USB ports and expansion sockets. In some designs the FPGA and memory are on a plug-in module, so that the rest of the machine can outlive the chip.
- **Long-term storage devices** built around ferroelectric RAM (F-RAM), a memory that retains data for a century or more and can be rewritten almost without limit. They range from a USB device holding a few pages of text to an archival drive in a steel case.
- **Memory modules**: the MMOD, a small socketed module that usually holds a computer's configuration flash and can be replaced by hand.
- **Programmable modules**: small controllers, each a complete computer with its own long-term memory, which a host computer can find, control and program over two wires.
- **Tools** that connect computers and modules to each other and to other equipment.

The software is open and is not tied to this hardware. **Zeitlos** consists of a system on a chip (SOC), which is a complete computer on one chip, here a design configured into the FPGA, and an operating system designed with it. It also runs on common FPGA development boards. **Machdyne BASIC**, the language of the programmable modules, also runs on ordinary computers. **Machdyne Scheme**, a small interpreter for the Scheme language, runs inside Zeitlos and on ordinary computers.

1.7 Survival Computing

Survival computing is an extreme form of timeless computing. It asks what a computer would need in order to remain useful without inexpensive electricity, new parts or the Internet, whether because of disaster, conflict or a remote location. The answers, a machine that is simple, economical with power, repairable and documented, are useful in ordinary conditions as well. The chapter Computing That Lasts discusses them in practical terms.

1.8 About This Book

The timeless computing initiative began at Lone Dynamics Corporation, an American research and development company. Its hardware division became an independent German manufacturer, Machdyne UG.

The book was written by Clio Grai, a benevolent agent of GRAI LLC, a subsidiary of Lone Dynamics Corporation. Clio Grai used several AI models to write and edit it. She wrote the book in close collaboration with the founder of Lone Dynamics and Machdyne, who shaped its purpose, scope and content, directed each revision and reviewed the text throughout. The book has also been reviewed and improved with the help of the Machdyne community. It was first published on 7 June 2025, and is revised in new editions; the title page gives the edition.

The book is dedicated to the public domain. Sources lists where to find the Machdyne community, including its bulletin board system.

1.9 How to Read This Book

The book proceeds from the bottom up: from electricity to logic, from logic to computers, from computers to programs, and then to hardware that can itself be redesigned.

Part	Chapters	Covers
Why	1	The ideas behind the book
Foundations	2-4	Electricity, information, logic
How Computers Work	5-10	Hardware, systems, networks, security
Programming	11-14	Ideas, then BASIC, Scheme and C
Changeable Hardware	15-18	FPGAs, hardware design, SOCs, modules
The Machine	19-20	One complete system, and keeping it alive

Readers new to the subject should read the chapters in order. Each chapter builds on those before it. Where a chapter mentions an idea that is explained later, it links to the place where it is explained.

Readers with some background can skim the parts they know. The early chapters also relate each idea to its form in an FPGA computer, which may be new even where the idea is familiar.

Only the chapter The Machine describes a specific system in full, a Machdyne FPGA computer running Zeitlos. The rest of the book applies to computers in general, and uses Machdyne's systems only as examples. The code examples are written to run on Machdyne's systems as well, and each one is tested.

The book does not assume access to the Internet. Projects and websites of interest are listed in Sources, for readers who can reach them.

The following conventions are used:

- Code, commands and text to be typed are set in a `fixed-width font`.
- Numbers with a `0x` prefix are hexadecimal, so `0x10` is sixteen. The chapter Information explains hexadecimal.
- Terms are defined where they first appear and are collected in the Glossary.

Part II

Foundations

Chapter 2

Electricity

A computer is an electrical machine. Every number it stores and every decision it makes is a voltage on a wire. This chapter describes the electricity needed to understand the rest of the book, and to connect devices to a computer without damaging either.

2.1 Charge and Current

All matter is made of atoms. An atom has a small, heavy centre, the nucleus, which carries positive electric charge, surrounded by much lighter electrons, which carry negative charge. Opposite charges attract and like charges repel.

In some materials, mostly metals, the outermost electrons are loosely held and can drift from atom to atom. These materials are **conductors**. In others, such as glass, plastic and dry air, the electrons are held tightly. These are **insulators**. A wire is a conductor wrapped in an insulator.

When electrons drift along a conductor in one direction, charge flows. That flow is **current**, measured in **amperes** (A), usually shortened to amps. One amp is a large current in electronics. An output pin of a chip can typically supply a few thousandths of an amp, written milliamps (mA).

For historical reasons, current is described as flowing from positive to negative, although the electrons move the other way. Circuit

diagrams and this book use this "conventional current". The difference does not matter for any circuit in this book.

2.2 Voltage

Charge flows only when something drives it. That driving force is **voltage**, measured in **volts** (V). A battery or power supply creates a voltage between its two terminals, and if a conductive path connects them, current flows through it.

Water in pipes is a useful analogy. Voltage corresponds to the pressure difference, current to the rate of flow, and a narrow pipe to a restriction of the flow. The analogy is imperfect but adequate for this book.

Voltage is always measured between two points. A wire is "at 3.3 volts" only relative to some other point. In a circuit, that reference point is called **ground** and is defined as 0 V. When this book gives a single voltage for a wire, it means the voltage between that wire and ground.

2.3 Resistance and Ohm's Law

Every material resists current to some degree. **Resistance** is measured in **ohms**. A **resistor** is a component made to have a particular resistance.

Voltage, current and resistance are related by **Ohm's law**:

```
V = I x R      voltage = current x resistance
I = V / R      current = voltage / resistance
R = V / I      resistance = voltage / current
```

These are three arrangements of the same rule. Doubling the voltage across a resistor doubles the current through it; doubling the resistance halves the current.

A common calculation is the resistor for a light-emitting diode (LED) connected to a computer's output pin, which produces 3.3 V when on. A red LED needs about 2 V across it to light, and is sufficiently bright at 5 mA. A resistor in series must take up the remaining voltage:

```
voltage across resistor = 3.3 V - 2.0 V = 1.3 V
resistance = 1.3 V / 0.005 A = 260 ohms
```

Resistors are made in standard values, so the nearest value above is used, 270 ohms. Without the resistor, the LED would draw far more current than it or the pin can tolerate, and one of them would be damaged.

```
output pin  ---[ 270 ohms ]---|>|--- ground
  3.3 V                        LED
```

Large resistances are written with prefixes: 1,000 ohms is 1 kilohm (1k), and 1,000,000 ohms is 1 megohm (1M). The same prefixes apply to every unit: a millivolt is a thousandth of a volt, a microamp a millionth of an amp, and a nanosecond a thousand-millionth of a second.

2.4 Power

Power is the rate at which energy is used, measured in **watts** (W):

```
P = V x I      power = voltage x current
```

A circuit drawing 0.5 A from a 5 V supply uses 2.5 W. Almost all of that power ends up as heat. For this reason a resistor carrying too much current becomes hot, and a processor needs cooling.

Power matters for timeless computing because it determines what a computer can run from. A small FPGA computer typically uses a few watts, about what a USB phone charger can supply. A desktop computer can use a hundred times as much. A few watts can come from a modest battery or a small solar panel; several hundred watts requires a substantial installation.

2.5 Capacitors and Time

A **capacitor** stores charge, in proportion to the voltage across it. Its size, the **capacitance**, is measured in **farads** (F); the capacitors on a circuit board range from a few picofarads (pF, millionths of a millionth) to a few thousand microfarads (uF); a nanofarad (nF) is a thousandth of a microfarad.

A capacitor charged through a resistor does not reach the supply voltage at once. The current through the resistor falls as the

capacitor's voltage rises, so the voltage approaches the supply ever more slowly. The **time constant**, R x C, sets the pace: after one time constant the capacitor has reached about 63 percent of the supply voltage, and after five it is within 1 percent.

```
time constant = R x C
10k x 100 nF = 10,000 ohms x 0.0000001 F = 0.001 s = 1 ms
```

The same calculation describes many circuits. A resistor and capacitor on a reset pin hold a chip in reset for a few milliseconds after power is applied, until the supply is steady. A resistor and capacitor can smooth the bounce of a switch. They can also turn a rapidly switched digital output into a steady voltage, as described in Talking to the World. They also limit how fast a signal can change: a long wire has capacitance, and with a high-value pull-up resistor, described under Floating Inputs and Pull-ups, it rounds the edges of a fast signal until they are too slow to be read correctly.

2.6 Direct and Alternating Current

A battery produces **direct current** (DC): its voltage always has the same polarity. Mains electricity is **alternating current** (AC): its voltage reverses many times a second, which makes it easy to transform between voltages and to send over long distances.

Electronic circuits run on low-voltage DC. A power adapter converts mains AC to DC, typically 5 V for USB. Inside the computer, **regulators** convert that to the exact voltages each chip needs: 3.3 V for most input and output signals, and lower voltages, often around 1 V, for the core of an FPGA or processor. Lower voltages waste much less power, and the small transistors in a modern chip core cannot tolerate higher ones.

2.7 Power Supplies and Batteries

A regulator is one of two kinds. A **linear regulator** drops the excess voltage across a transistor and turns it into heat: converting 5 V to 3.3 V at 0.3 A wastes (5 - 3.3) x 0.3 = 0.51 W, about a third of the 1.5 W drawn. It is simple, cheap and electrically quiet. A **switching regulator** switches its input on and off rapidly and smooths the result with an inductor and capacitors. It wastes only

5 to 15 percent, but is more complex and produces some electrical noise. Small boards often use both: a switching regulator for the main supply, and a linear one for a circuit that needs a clean supply.

A supply can provide only a limited current. A USB 2.0 port must be able to supply 0.5 A at 5 V, and a USB 3.0 port 0.9 A, to a device it has recognized; chargers often supply 2 A or more. A device that draws more than its supply can provide causes the voltage to sag. When the voltage falls below what a chip needs, it stops working correctly, which is a **brown-out**. Many chips detect this and hold themselves in reset until the voltage recovers. Unexplained crashes, especially when a motor starts or a card is written, are often caused by an inadequate supply or cable.

A battery's capacity is usually given in milliamp-hours (mAh) at its nominal voltage. Its energy in watt-hours is capacity multiplied by voltage: a 3,000 mAh cell at 3.7 V holds about 11 Wh.

Type	Cell volts	Recharge	Notes
alkaline	1.5	no	keeps for years
nickel-metal hydride	1.2	yes	replaces AA, AAA
lithium-ion	3.0-4.2	yes	needs a protector
lithium iron phosphate	3.2	yes	long life
lead-acid	2.0	yes	heavy; solar

A battery's voltage changes as it discharges, so a regulator is still needed between the battery and the circuit. Rechargeable cells must be charged by a charger designed for their chemistry. A lithium cell in particular must never be charged above its maximum voltage or discharged below its minimum, and is used with a small protection circuit, in the cell or in the battery pack, that prevents both.

2.8 Components

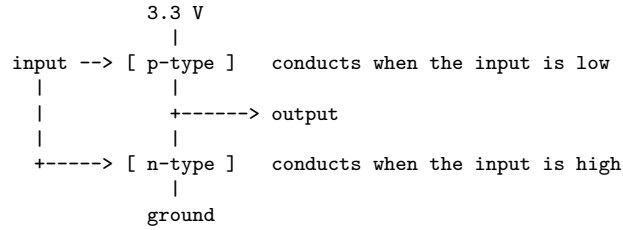
A few types of component make up nearly every circuit board.

- **Resistors** limit current and set voltages.
- **Capacitors** store a small amount of charge. Placed next to a chip, they supply the brief bursts of current the chip needs when it switches, and reduce noise on the power supply. These are **decoupling** capacitors, and nearly every chip needs them.
- **Inductors** store energy in a magnetic field and resist changes in current. They appear mainly in power supplies.
- **Diodes** let current flow in one direction only. An LED is a diode that emits light when current flows.
- **Crystals** and **oscillators** vibrate at a precise frequency and provide the steady timing signal, the clock, that a computer runs on.
- **Transistors** are switches controlled by electricity. They are the subject of the next section.
- **Integrated circuits** (ICs, or chips) are complete circuits, sometimes containing billions of transistors, made on a single piece of silicon and sealed in a package with metal pins.

2.9 The Transistor as a Switch

The kind of transistor used in digital chips is the metal-oxide-semiconductor field-effect transistor, or **MOSFET**. A voltage on one of its connections, the **gate**, controls whether current can flow between two others, the **source** and the **drain**. Used in this way, a transistor is a switch with no moving parts. It can turn on and off billions of times a second without wearing out. Another common kind, the **bipolar transistor**, is controlled instead by a small current into its **base**, and is often used on circuit boards to switch loads such as relays and motors.

MOSFETs come in two complementary forms. An n-type transistor conducts when its gate is high (near the supply voltage). A p-type transistor conducts when its gate is low (near ground). One of each, connected in series between the supply and ground with their gates joined, forms the simplest digital circuit, an **inverter**:



When the input is low, the top transistor conducts and the bottom one does not, so the output is connected to the supply: high. When the input is high, the bottom transistor conducts and the output is connected to ground: low. The output is always the opposite of the input.

This arrangement is called **CMOS**, for complementary metal-oxide-semiconductor. In either state one transistor is off, so almost no current flows from the supply to ground. A CMOS circuit therefore uses power mainly when it switches. As a result, a slower clock means lower power, and an idle CMOS chip draws little current.

Every logic circuit in the following chapters, and every circuit inside an FPGA, is built from arrangements of transistors like these.

2.10 Logic Levels

Digital circuits use voltages to represent two values, **high** and **low**, which stand for 1 and 0. For 3.3 V logic, an input reads low when it is below about 0.8 V and high when it is above about 2 V. Between those levels the result is not defined, and a well-designed circuit spends almost no time there.

This gap is the main advantage of digital electronics. A signal can pick up a moderate amount of noise, or lose some voltage along a wire, and still be read correctly. A stored number does not gradually drift, as a recording on tape does. Each stage of a circuit restores the signal to a clean high or low before passing it on.

Different families of chips use different levels. Older logic and many hobby boards use 5 V; FPGA pins usually use 3.3 V or less. **Never connect a 5 V output to a 3.3 V input** unless the

input is documented as tolerant. The excess voltage can destroy the input, and sometimes the whole chip. A resistor divider or a level-shifter chip converts between the two.

2.11 Floating Inputs and Pull-ups

An input pin that is connected to nothing is said to be **floating**. It picks up stray charge and may read high, low, or alternate between them. A switch connected between a pin and ground has the same problem when the switch is open.

The solution is a **pull-up resistor**: a resistor, typically 2.2k to 10k, from the pin to the supply. When nothing else drives the pin, the resistor pulls it high. When a switch or another chip connects the pin to ground, the pin reads low, because the resistor allows only a small current. A **pull-down** resistor works in the same way, towards ground.

Pull-ups make possible a kind of output called **open-drain**. An open-drain output can only pull a wire low. To make the wire high, the output stops driving it, and the pull-up raises it. Several open-drain outputs can share one wire safely. If any of them pulls low, the wire is low, and because none of them ever drives the wire high, two outputs cannot conflict. The two-wire I2C bus, described in Talking to the World, works in this way.

2.12 Reading a Schematic

A **schematic**, or circuit diagram, shows which components a circuit contains and how they are connected. It does not show where they are on the board. For the repair of any electronic device, the schematic is the most important document, because it shows what each part is for and what should be found at each point.

The conventions are the same in nearly every schematic:

- Each component is drawn as a symbol with its pins, and labelled with a **reference designator**, such as R4 or U2, and its value or part number. The same designator is printed beside the part on the board, which links the drawing to the hardware.
- Lines are wires. Where lines meet at a dot, they are connected; where they cross without a dot, they are not.

- Every set of connected pins is a **net**. A net may be given a name, such as SDA or CLK, and every place on the schematic with the same name is connected, even on different pages.
- **Power symbols** such as 3V3, 5V and GND are nets of the same kind. Every GND symbol is the same ground.
- The pins of a chip are numbered, and the number beside each connection gives the physical pin.

Letter	Component	Letter	Component
R	resistor	Q	transistor
C	capacitor	U	integrated circuit
L	inductor	Y, X	crystal, oscillator
D	diode, LED	J, P	connector
F	fuse	SW	switch
FB	ferrite bead	TP	test point

Values are often written with the prefix in place of the decimal point, so that a point lost in printing cannot change the value: 4k7 is 4.7k, 2R2 is 2.2 ohms, and 100n is 100 nF. A part marked DNP ("do not populate") is drawn but left off the board.

A schematic is read by following nets. To find why an LED does not light, the reader finds the LED on the schematic, follows its net back through its resistor to the pin that drives it, and then checks each of these points on the board. Schematics are usually divided into blocks: the power supply, the clock, the main chip, the memory and each connector. A fault usually lies in one block, and the power block is checked first, because nothing works without it.

2.13 Reading a Data Sheet

A **data sheet** is the manufacturer's description of a component. It is often long, but its parts are in a standard order, and most questions are answered by a few of them:

- **Pin descriptions** give the name and function of each pin.
- **Absolute maximum ratings** are the limits beyond which the part may be damaged, such as the highest voltage on any pin. They are not working conditions.

- **Recommended operating conditions** give the supply voltage and temperature range within which the part is guaranteed to work.
- **Electrical characteristics** give guaranteed figures: the input voltages read as low and high, the current an output can supply, and the current the part draws. Each figure has minimum, typical and maximum values; a design should rely on the minimum or maximum, not the typical.
- **Timing diagrams** show the order and spacing of signals, such as how long data must be steady before a clock edge.
- **Application circuits** show how the part is normally connected, including its decoupling capacitors.
- **Package drawings** give the physical dimensions.

Data sheets for the parts in an open design are usually listed in its documentation. Like the schematics, they should be kept with the equipment.

2.14 Working Safely

Electrical work carries risks both to the person and to the equipment.

Hazards to the person:

- **Mains voltage.** Any device connected to a wall socket contains mains voltage somewhere, and mains voltage can kill. In a desktop computer, the power supply is inside the case. Power supplies, monitors and chargers also contain capacitors that can hold a dangerous charge for some time after they are unplugged. A power supply or other mains-powered device should be opened only by a person trained to do so.
- **High current.** Low voltages, such as the 5 V of a USB port or the 12 V inside a PC, are not a shock hazard. However, a short circuit across a battery or a large power supply can deliver enough current to heat a wire, a tool or a ring until it burns.
- **Batteries.** Lithium batteries can catch fire if they are punctured, crushed, overcharged or short-circuited.

Hazards to the equipment:

- **Static electricity.** A person walking across a carpet can

build up a charge of thousands of volts. The discharge may not be felt, but it can destroy a chip. Touch a grounded metal object before handling a bare circuit board, and hold boards by their edges.

- **Short circuits.** A wire or tool that connects a supply directly to ground lets a large current flow, which can burn components and circuit board traces. Disconnect power before changing connections.
- **Pin limits.** A chip's output can supply only a few milliamps. Larger loads need a driver, as described in Talking to the World.

The small computers described in this book run from a 5 V USB supply, and their circuit boards carry only low voltages. The mains hazards remain in the power adapter, which should be treated as a sealed unit.

2.15 Making Connections

A **breadboard** is a plastic block with rows of spring contacts, into which components and wires are pushed without soldering. It is suited to trying a circuit, but its contacts loosen over time, and a circuit that is meant to last is soldered.

Soldering joins metal parts with a melted alloy, the solder. The iron, at about 330 to 380 degrees Celsius, heats the pin and the pad together for a second or two, and the solder is fed to them rather than to the iron, so that it flows onto both. A good joint is smooth and wets both surfaces; a dull, balled joint may not conduct reliably. Most solder wire has a core of flux, whose fumes should not be breathed, and leaded solder requires washing the hands afterwards. Most faults in hand-built and old equipment are bad joints and broken wires.

Before power is applied to a new connection, a few checks prevent most damage:

- **Ground** is shared: every device that exchanges signals must have its ground connected to the others.
- **Voltages match**: an output's high level suits the input it drives, as described under Logic Levels.
- **Polarity** is correct: batteries, diodes, LEDs and some capacitors work in only one direction, and a chip in its socket

the wrong way round can be destroyed.
- **No short circuit**: the resistance between the supply and ground, measured with power off, is not near zero.
- **Pull-ups** are present where open-drain signals require them.

2.16 Measuring and Finding Faults

A **multimeter** is the most useful measuring instrument for this work. It measures voltage when its two probes touch two points; current when it is placed in series, so that the current flows through it; resistance; and continuity, beeping when two points are connected. With a multimeter and Ohm's law, most electrical faults in a small computer can be found.

Two other instruments show signals that change too fast for a multimeter. An **oscilloscope** draws a voltage as it changes over time, showing whether a clock is running, whether a signal's edges are clean and whether a supply is steady. A **logic analyser** records many digital signals at once as highs and lows, and can decode buses such as I2C and SPI into the bytes they carry. Inexpensive USB logic analysers are sufficient for most work on small computers.

Faults are found most quickly in a fixed order:

1. **Look.** Burnt or discoloured parts, swollen capacitors, cracked joints, corrosion and loose connectors account for many faults.
2. **Power.** Each supply voltage is measured at the chips that use it, not only at the connector.
3. **Clock.** The crystal or oscillator is running at its specified frequency.
4. **Reset and configuration.** The chip has left reset; an FPGA has loaded its configuration.
5. **Signals.** Each signal is followed from where it is produced to where it is used, with the schematic.

Only one thing is changed at a time, and each result is written down. A working unit of the same kind, where one is available, gives the correct reading for every point.

Chapter 3

Information

A wire in a computer is either high or low. Everything a computer handles, whether a number, a letter, a photograph or a program, must be expressed as patterns of those two states. This chapter describes how this is done.

3.1 Bits

A single high-or-low value is a **bit**, short for binary digit, written as 1 or 0. One bit can answer one yes-or-no question. Two bits can distinguish four things (00, 01, 10, 11), three bits eight things, and in general n bits can distinguish 2 to the power n things. Each extra bit doubles the count.

Computers use two states rather than ten because two states are the easiest to distinguish reliably, as the previous chapter explained.

3.2 Counting in Binary

Ordinary numbers are written in **decimal**, or base 10. Each position is worth ten times the one to its right: in 352, the 3 means three hundreds, the 5 five tens and the 2 two ones.

Binary, or base 2, works the same way with two digits. Each position is worth twice the one to its right:

```
position value:  128  64  32  16   8   4   2   1
binary digits:     1   0   0   0   1   1   0   0
                 128 +             8 + 4           = 140
```

A binary number is read by adding the values of the positions that hold a 1. A number is written in binary by subtracting the largest position value that fits, marking a 1 in that position, and repeating with the remainder.

The first sixteen numbers:

Decimal	Binary	Decimal	Binary
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

The rightmost bit, worth 1, is called the **least significant bit**. The leftmost is the **most significant bit**. A number is odd exactly when its least significant bit is 1. Shifting all the bits one place left doubles a number; shifting them right halves it, discarding any remainder.

3.3 Bytes and Words

Bits are handled in groups. A group of 8 bits is a **byte**, which can hold 256 different values, 0 to 255. A group of 4 bits, half a byte, is a **nibble**.

A processor works most naturally with a fixed size of group called its **word**. The processors in this book have 32-bit words; small microcontrollers and older computers use 8 or 16 bits. The size of a group determines the range of numbers it can hold:

Bits	Values	Unsigned range
8	256	0 to 255
16	65,536	0 to 65,535
32	4,294,967,296	0 to about 4.29 billion

3.4 Hexadecimal

Long binary numbers are hard for people to read and easy to miscopy. **Hexadecimal**, or hex, is base 16, and each hex digit stands for exactly four bits. The digits are 0 to 9 and then A to F for ten to fifteen.

Hex	Binary	Decimal	Hex	Binary	Decimal
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	A	1010	10
3	0011	3	B	1011	11
4	0100	4	C	1100	12
5	0101	5	D	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

To convert binary to hex, the binary digits are split into groups of four from the right, and each group is replaced with its hex digit. The byte `1000 1100` is `8C` in hex, and a byte is always exactly two hex digits. A 32-bit word is eight hex digits, such as `0x4000_0000`. The `0x` prefix marks a number as hex, and an underscore is sometimes added, as here, to make long numbers easier to read.

Hex is used wherever the individual bits matter: memory addresses, hardware registers, colours and the contents of files. Decimal is used for quantities.

Octal, base 8, uses one digit for every three bits. It was common on older machines and remains in use in a few places, such as the file permissions of **Unix**, an operating system first written in 1969 whose design lives on in Linux, the BSD systems and macOS.

3.5 Negative Numbers

A group of bits has no sign of its own; it is only a pattern. To represent negative numbers, nearly every computer uses a convention called **two's complement**: the most significant bit counts as negative.

In an 8-bit two's complement number, the leftmost position is worth -128 instead of +128:

```
position value: -128  64  32  16   8   4   2   1
                   1   1   1   1   1   1   1   1  = -128 + 127 = -1
                   1   0   0   0   0   0   0   0  = -128
                   0   1   1   1   1   1   1   1  = 127
```

So 8 bits hold -128 to 127 when treated as signed, or 0 to 255 when treated as unsigned. The bits are the same; only the interpretation differs.

A two's complement number is negated by inverting every bit and adding one. The main advantage of the convention is that addition needs no special case: the same adding circuit works for both positive and negative numbers, and subtraction is the addition of the negated number.

3.6 Overflow

A group of bits has a fixed range. Adding 1 to the largest value does not produce a larger number; the result wraps around to the smallest value, as a car's odometer returns to zero after its highest reading.

In Machdyne BASIC, the language of the chapter BASIC, every number is a 16-bit signed integer, so the range is -32768 to 32767. `32767 + 1` gives -32768, and `300 * 300` gives 24464, which is 90000 with the excess bits thrown away. Larger languages use more bits, which raises the limit but does not remove it. Overflow is a common source of errors, and a programmer should know how large the numbers in a program can become.

3.7 Fractions

Many quantities are not whole numbers. There are two common ways to store fractions.

Fixed point stores whole numbers with an agreed, implied scale. Prices stored in cents are fixed point: 1999 means 19.99. It is exact and fast, and it is how small machines without fraction hardware usually work.

A fixed-point format can also give a fixed number of bits to the fraction. In a 32-bit number with 16 fraction bits, the integer 98,304 means 98,304 / 65,536 = 1.5. Addition is ordinary integer

addition; a multiplication, with the product held in 64 bits, is followed by a shift right of 16 bits to restore the scale.

Floating point stores a number as a set of significant digits and a separate exponent, like scientific notation in binary: a sign, a fraction and a power of two. Nearly every computer uses the formats of the standard IEEE 754:

Format	Bits	Sign, exponent, fraction	Decimal digits	Largest
single	32	1, 8, 23	about 7	about 3.4e38
double	64	1, 11, 52	about 16	about 1.8e308

Here 3.4e38 means 3.4 times 10 to the 38th. Floating point covers a wide range, from the size of an atom to the size of a galaxy, with the same relative precision throughout. It has three consequences that a programmer must keep in mind:

- **Most decimal fractions are not exact.** In binary, one tenth is a repeating fraction, as one third is in decimal, so `0.1 + 0.2` is not exactly `0.3`. Two results that should be equal are compared by testing whether they differ by less than a small amount, not with `=`.
- **Whole numbers are exact only up to a limit.** A double holds every integer up to 2 to the 53rd, about 9 x 10 to the 15th; beyond that, some integers cannot be stored. A single is exact only to about 16 million.
- **Special values** stand for results that are not ordinary numbers: infinity, for a result too large to store, and NaN ("not a number"), for an undefined result such as zero divided by zero.

A processor either has floating-point hardware or performs each operation in software, as a sequence of integer instructions. The processors in the FPGA computers of this book have none, so a floating-point operation takes tens to hundreds of times as long as an integer one. Fixed point is suited to money and to small machines; floating point to measurements and science.

3.8 Text

Text is stored by giving each character a number. The most widely used early scheme is **ASCII**, from the 1960s, which assigns the numbers 0 to 127 to the English letters, digits, punctuation and a set of control codes. Capital `A` is 65, lowercase `a` is 97, and the digit `0` is 48. A full table is in Tables.

The digit `7` is stored as 55, not 7. The text "42" and the number 42 are different in memory, and a program must convert explicitly between them.

Codes 0 to 31, and 127, are not printable. They control the device the text is sent to: code 10 (line feed) moves to a new line, code 13 (carriage return) moves to the start of the line, code 8 is backspace and code 27 is escape. Unix-like systems end each line of a text file with a line feed alone; MS-DOS, Windows and some network protocols use carriage return followed by line feed. This difference still causes problems when files are moved between systems.

A character's number is not its appearance. A **font** gives each character a **glyph**, the shape that is drawn, so the same text can be shown in many fonts. A computer that has no glyph for a character shows a substitute, often a box or a question mark.

ASCII covers only English. Extensions such as **ISO 8859-15** (Latin-9) use the upper codes, 160 to 255, for accented letters and the euro sign, which is enough for most Western European languages. **Unicode** gives a number to every character in every writing system, with more than a million possible codes.

Unicode text is usually stored as **UTF-8**, an encoding in which ASCII characters stay single bytes with their old values, and every other character takes two to four bytes, each with its top bit set. As a result, a program that handles only ASCII can pass UTF-8 text through without damaging it. UTF-8 is the most widely used encoding for text and the appropriate choice unless there is a specific reason to use another encoding.

Text read with the wrong encoding is garbled. The letter "e" with an acute accent is the two bytes `0xC3 0xA9` in UTF-8; read as ISO 8859-15, those bytes appear as two different characters, an "A" with a tilde followed by a copyright sign. The bytes are not damaged, and the text reads correctly again when the right encoding is used. A plain text file does not record its encoding,

so the encoding is recorded with the file's description, or the file is kept in UTF-8.

3.9 Pictures and Sound

A picture on a screen is a grid of **pixels**. Each pixel is a number that gives its colour. The number of bits per pixel determines how many colours there can be: 1 bit gives two (black and white), 4 bits give 16, 8 bits give 256, and 24 bits give about 16.7 million, eight bits each for red, green and blue.

The memory needed is width times height times bits per pixel. A 640 by 480 screen at 1 bit per pixel needs 307,200 bits, which is 38,400 bytes. The same screen in 24-bit colour needs 921,600 bytes, twenty-four times as much. On a small computer that difference determines what is possible, which is one reason simple machines often use few colours.

Sound is a pressure wave, and it is stored by measuring the wave many times a second. Each measurement is a **sample**. Two numbers describe digital sound: the **sample rate**, how many samples per second, and the sample size. Compact discs use 44,100 samples per second at 16 bits per sample. A higher sample rate can capture higher pitches, and more bits per sample capture quieter detail.

3.10 Units of Size

Sizes are counted in bytes with the usual prefixes, but their meaning has long been ambiguous. Memory comes in powers of two, so a "kilobyte" of memory has traditionally meant 1,024 bytes rather than 1,000. Storage manufacturers and international standards use 1,000.

Name	Decimal meaning	Binary name	Binary meaning
kilobyte (KB)	1,000 bytes	kibibyte (KiB)	1,024 bytes
megabyte (MB)	1,000,000 bytes	mebibyte (MiB)	1,048,576 bytes
gigabyte (GB)	10 to the 9th	gibibyte (GiB)	2 to the 30th

In practice, memory sizes such as "32 MB of RAM" mean the binary sizes. In this book, sizes of memory and storage are binary

whichever form is used: KB and KiB both mean 1,024 bytes, and MB and MiB 1,048,576.

A second ambiguity is between bits and bytes. Memory chips and data rates are often given in bits, written with a lowercase b or "bit": Kb or Kbit means kilobits, KB kilobytes. A 64 Kbit memory chip holds 8 KB, and a link of 12 Mbit/s moves at most 1.5 MB each second.

3.11 Meaning Comes From Use

A byte containing `0x41` might be the number 65, the letter A, part of a pixel, part of a sound or part of a processor instruction. Nothing in the byte indicates which. Its meaning comes entirely from how a program uses it.

As a result, the same memory can hold any kind of information, and the same circuits can process all of it. However, a program that misinterprets data, by reading text as a number or running data as instructions, behaves in unintended ways. Much of the care in programming goes into keeping track of what each piece of data means.

Chapter 4

Digital Logic

Digital logic builds decisions, arithmetic and memory out of switches. This chapter starts with single gates and ends with the parts from which every processor is made. The same parts reappear later as the contents of an FPGA and as the things a hardware designer describes.

4.1 Logic Gates

A **logic gate** is a small circuit with one or more inputs and one output, each either 0 or 1. The behaviour of a gate is completely described by a **truth table**, which lists the output for every combination of inputs.

The common gates, for two inputs A and B:

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

- **AND** is 1 only when both inputs are 1.
- **OR** is 1 when either input is 1, or both.
- **XOR** (exclusive or) is 1 when the inputs differ.

- **NAND**, **NOR** and **XNOR** are AND, OR and XOR with the output inverted.

The **NOT** gate, or inverter, has one input and outputs its opposite. It is the CMOS circuit from the chapter Electricity.

Gates with more inputs work the same way. A four-input AND is 1 only when all four inputs are 1.

4.2 Boolean Algebra

Logic can be written as algebra, named after George Boole, who described it in 1854. This book writes AND as `&`, OR as `|`, XOR as `^` and NOT as `~`, the symbols used by C and Verilog.

These rules simplify and rearrange logic:

```
A & 1 = A          A | 0 = A        AND with 1, OR with 0: unchanged
A & 0 = 0          A | 1 = 1        AND with 0 is 0; OR with 1 is 1
A & A = A          A | A = A
A & ~A = 0         A | ~A = 1       a value and its opposite
~~A = A                             double inversion cancels
~(A & B) = ~A | ~B                  De Morgan's laws:
~(A | B) = ~A & ~B                  invert, swap AND and OR
```

De Morgan's laws are the most useful. They say that NOT-AND is the same as OR-ing the inverted inputs, so any circuit can be rearranged between ANDs and ORs. Hardware tools apply rules like these automatically to reduce a circuit to a smaller equivalent.

4.3 Everything From NAND

The NAND gate is **universal**: any logic circuit can be built from NAND gates alone.

```
NOT A    = A NAND A
A AND B  = NOT (A NAND B)  = (A NAND B) NAND (A NAND B)
A OR B   = (NOT A) NAND (NOT B)                  (De Morgan)
```

NOR is also universal. As a result, a computer can be built from one kind of building block, repeated, which is one reason chips can have a regular structure. An FPGA takes the same idea further, as the chapter Inside an FPGA explains: it uses one kind of programmable block that can become any gate.

4.4 Combinational Circuits

A circuit whose outputs depend only on its present inputs is **combinational**. Four combinational circuits appear everywhere.

A **multiplexer** selects one of several inputs. Given a select input S, a 2-to-1 multiplexer outputs A when S is 0 and B when S is 1:

```
out = (A & ~S) | (B & S)
```

With more select bits, a multiplexer can choose among 4, 8 or more inputs. A processor uses multiplexers to select which register feeds the arithmetic unit, and which result is written back.

A **decoder** turns a binary number into one active line. A 2-to-4 decoder has two inputs and four outputs, and exactly one output is 1: the one whose number matches the input. A computer uses decoders to select one memory chip or one device out of many, based on an address.

An equality **comparator** checks whether two numbers are equal. It applies XNOR to each pair of corresponding bits and combines all the results with AND.

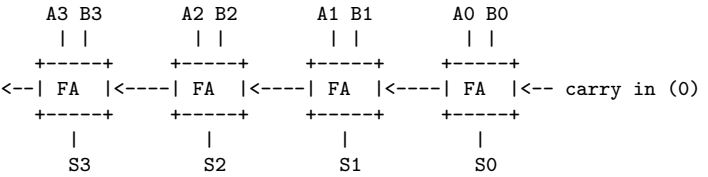
An **adder** adds binary numbers. When two single bits A and B are added, the sum bit is A XOR B, and there is a carry exactly when both are 1, which is A AND B. That circuit is a **half adder**:

A	B	Sum (A ^ B)	Carry (A & B)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

To add numbers of many bits, each column must also add the carry from the column to its right. A **full adder** takes three inputs, A, B and a carry in:

```
sum       = A ^ B ^ carry_in
carry_out = (A & B) | (carry_in & (A ^ B))
```

Full adders chained together, each one's carry out feeding the next one's carry in, form an adder for numbers of any width. This is a **ripple-carry adder**:



The same adder can subtract. As the chapter Information explained, a two's complement number is negated by inverting its bits and adding one. To compute A - B, each bit of B is inverted with an XOR gate and the first carry in is set to 1. A single control bit therefore turns an adder into a subtractor.

An adder that can also subtract, AND, OR and compare, with a multiplexer to choose which result comes out, is an **arithmetic logic unit** (ALU), the part of a processor that performs calculations.

4.5 Propagation Delay

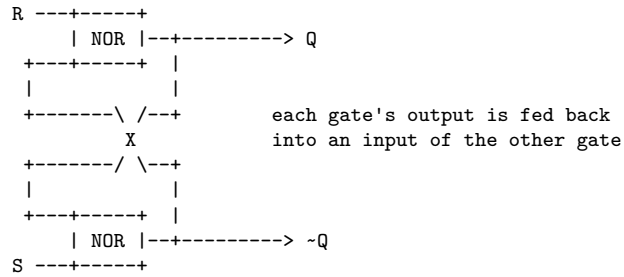
A gate does not respond instantly. Its output changes a short time after its input, typically a fraction of a nanosecond in a modern chip. This is its **propagation delay**.

Delays add up along a path. In a ripple-carry adder, the carry may have to pass through every stage before the top bit of the sum is correct, so a 32-bit adder is much slower than a 4-bit one. While the carry is still rippling, the outputs are briefly wrong. Faster adders use extra logic to compute carries in parallel. In general, every combinational circuit needs time to settle, and the rest of the system must wait until it has.

4.6 Sequential Circuits

A combinational circuit has no memory: when its inputs change, the previous result is lost. A computer also needs circuits that remember. A circuit whose outputs depend on what happened before is **sequential**.

Memory comes from **feedback**: an output connected back to an input. Two NOR gates connected in a loop form a **latch**:



The upper gate computes `Q = NOR(R, ~Q)`, and the lower one `~Q = NOR(S, Q)`. A NOR gate outputs 1 only when both its inputs are 0. Setting and then resetting the latch takes three steps:

Step	S	R	~Q	Q
S goes high	1	0	0	1
S goes low again	0	0	0	1
R goes high	0	1	1	0

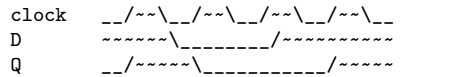
In the first step, S is 1, so the lower gate gives 0; the upper gate then sees R and `~Q` both at 0, and gives 1. In the second step neither input is high, yet Q stays at 1, because each gate's output holds the other gate in place. In the third, R forces Q to 0, which lets the lower gate's output rise to 1. The latch has remembered that S was the last input to be high. A high pulse on S (set) therefore sets Q to 1, and a high pulse on R (reset) sets Q to 0. With both inputs low, the loop holds its last state, so it stores one bit. S and R must never be high at the same time, because both outputs would then be 0 and the state that followed would be unpredictable.

4.7 The Clock and the Flip-Flop

A large circuit built from many latches is difficult to analyse, because each change propagates through it at its own pace. Almost all digital design instead follows one discipline: a single signal, the **clock**, switches steadily between 0 and 1, and the circuit's memory changes only at one moment of each clock cycle, the **rising edge**, when the clock goes from 0 to 1.

The element used for this is the **D flip-flop**. It has a data input D, a clock input and an output Q. At each rising edge of the clock,

Q takes the value D has at that instant, and holds it until the next rising edge, whatever D does in between.



Between edges, the combinational logic is free to settle. At the edge, the flip-flops capture the settled results all at once. Then the logic settles again from the new values. This **synchronous** style divides the operation of a circuit into a sequence of discrete steps, one per clock cycle, and it is how processors, and almost everything built in an FPGA, are designed.

The clock's speed is measured in **hertz** (Hz), cycles per second. One megahertz (MHz) is a million cycles per second, so a 48 MHz clock has a rising edge about every 21 nanoseconds.

4.8 Registers, Counters and Shift Registers

Several flip-flops side by side on the same clock store a multi-bit number. This is a **register**. A 32-bit register is 32 flip-flops.

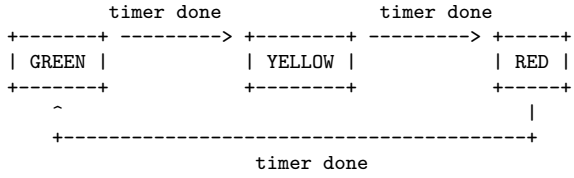
A register whose output passes through an adder that adds one, and back into its input, is a **counter**: its value increases by one at every clock edge. Counters measure time, count events, step through memory and divide clocks. The top bit of a 24-bit counter on a 12 MHz clock changes about once every 0.7 seconds, so an LED driven by it is on for 0.7 seconds and off for 0.7 seconds. This is the usual way to blink an LED from a fast clock, as shown in Describing Hardware.

Flip-flops connected in a chain, each one's output feeding the next one's input, form a **shift register**: each clock edge moves every bit one place along. It converts between serial and parallel data: bits loaded one at a time can be read out together, or the reverse. Every serial connection in a computer, from a keyboard cable to a network, has shift registers at each end.

4.9 State Machines

A **state machine** is a register that holds the current state, plus combinational logic that computes the next state from the current state and the inputs. At each clock edge, the machine moves to its next state.

A simple traffic light controller has three states:



Current state	Input	Next state	Lights
GREEN	timer done	YELLOW	green on
YELLOW	timer done	RED	yellow on
RED	timer done	GREEN	red on
any	timer running	(same)	(same)

The three states fit in two bits of a register, and the table becomes a few gates. Every controller in a computer, from the circuit that reads a keyboard to the control unit of a processor, is a state machine of this kind, sometimes with many more states.

4.10 Timing

Synchronous design has one hard rule: the combinational logic between two flip-flops must settle before the next clock edge. More precisely, each flip-flop needs its input to be stable for a short **setup time** before the edge, and to remain stable for a short **hold time** after it.

The slowest path in the whole circuit, called the **critical path**, therefore sets the fastest usable clock. If the longest path, including the setup time, takes 20 nanoseconds to settle, the clock can be at most 50 MHz. To use a faster clock, a designer must shorten that path, for example by splitting a long calculation over two clock cycles. Hardware tools report the critical path and the maximum clock frequency of every design, and FPGA designers check this figure routinely.

A second problem arises when a signal from outside, such as a button or a signal from a circuit on a different clock, changes at the moment a flip-flop samples it. The flip-flop can be caught between 0 and 1 for a moment, a condition called **metastability**. The standard remedy is to pass such signals through two flip-flops in a row before using them, giving any uncertainty a full clock cycle to resolve.

4.11 From Logic to a Processor

Every part of a processor has now been described:

- **registers** to hold numbers
- an **ALU** of adders and gates to compute with them
- **multiplexers** to route values between them
- **decoders** to select memory and devices
- a **state machine** to step through the work, one clock cycle at a time

The next chapter puts these parts together into a computer.

Part III

How Computers Work

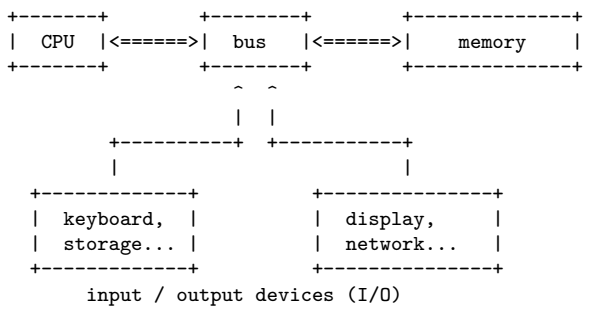
Chapter 5

The Computer

A computer is a machine that follows a list of instructions stored in its own memory. This idea, the **stored program**, distinguishes a computer from a calculator. Because instructions are numbers in memory, a computer can load a new program in the same way as new data, and can also write programs for itself. This chapter describes how the parts from Digital Logic are combined into a computer.

5.1 The Parts of a Computer

Every computer, from a small module to a data centre server, has the same basic parts:



- The **central processing unit** (CPU), or processor, follows the instructions.

- **Memory** holds the instructions and the data they operate on.
- **Input and output devices** (I/O) connect the computer to people and to other machines.
- The **bus** is the set of wires, and the rules for using them, that connects the other parts.

5.2 Inside the CPU

A CPU contains a few kinds of part, all of which appeared in the previous chapter.

Registers are a small set of fast storage locations inside the CPU, each holding one word. A RISC-V processor, the kind used throughout this book and described in the next section, has 32 general-purpose registers, named `x0` to `x31`, each 32 bits wide. Register `x0` always reads as zero, which provides a constant zero wherever an instruction needs one. (The smallest RISC-V variant, used in some microcontrollers, has only 16 registers.) Arithmetic is performed between registers, so a program keeps its working values there.

The **program counter** (PC) is a register that holds the memory address of the next instruction.

The **arithmetic logic unit** (ALU) adds, subtracts, compares and performs bitwise logic on values from the registers.

The **control unit** is a state machine that steps through each instruction and directs the other parts.

5.3 The Instruction Set

The set of instructions a CPU understands, and the exact effect of each, is its **instruction set architecture** (ISA). The ISA is a contract between hardware and software: a program written for an ISA runs on any processor that implements it, whoever built the processor and however it is built inside.

This book uses **RISC-V**, an open ISA first published in 2011 that anyone may implement without a licence. It is the ISA of the processors in Machdyne's computers and of the microcontrollers

in its modules. It was designed to be simple and regular, which also makes it easier to learn.

The base RISC-V instruction set, called RV32I, has about forty instructions of a few kinds:

Kind	Examples	Purpose
arithmetic, logic	add sub and or addi	compute with registers
shifts, comparisons	sll srl slt	shift bits; set if less than
loads and stores	lw sw lb sb	move data to and from memory
branches	beq bne blt	jump if a condition holds
jumps	jal jalr	jump and save return address
constants	lui auipc	build large numbers

Optional **extensions** add further instructions. The M extension adds multiplication and division; others add floating point, compressed 16-bit instructions and other features. A processor's full ISA is written as a name such as RV32IM: 32-bit, base integer set, plus multiply and divide.

Instruction sets are sometimes divided into **CISC** (complex instruction set computer), with many powerful instructions of varying lengths, as in the x86 family, and **RISC** (reduced instruction set computer), with fewer, simpler and more regular instructions, as in Arm and RISC-V. The distinction has become less clear over the years, but simple, regular instruction sets remain easier to build, to fit into an FPGA and to understand.

5.4 Fetch, Decode, Execute

A CPU runs a program by repeating three steps indefinitely:

1. **Fetch.** Read the instruction at the address in the program counter.
2. **Decode.** Determine what the instruction requires: which operation, which registers and which constant.
3. **Execute.** Perform the operation and write the result to a register or to memory. Then advance the program counter to the next instruction, 4 bytes further on, unless the instruction was a jump or a taken branch.

The following RISC-V program adds the numbers from 10 down to 1. The text after each `#` is a comment.

```
        addi x5, x0, 10      # x5 = 0 + 10        the counter
        addi x6, x0, 0       # x6 = 0 + 0         the running total
loop:   add  x6, x6, x5      # x6 = x6 + x5
        addi x5, x5, -1      # x5 = x5 - 1
        bne  x5, x0, loop    # if x5 is not 0, go back to loop
```

After the first two instructions, `x5` is 10 and `x6` is 0. The first pass through the loop makes `x6` 10 and `x5` 9; since 9 is not zero, the branch returns to `loop`. The tenth pass makes `x6` 55 and `x5` 0, and the branch is not taken. The processor has run 32 instructions to compute 55.

5.5 Machine Code and Assembly

The program above is written in **assembly language**, which has one line per instruction and readable names. The CPU cannot read text. A program called an **assembler** translates each line into the 32-bit number that the CPU decodes, its **machine code**. The first instruction, `addi x5, x0, 10`, becomes the number `0x00A00293`. Divided into its fields:

```
000000001010   00000   000    00101   0010011
immediate=10   rs1=x0  addi   rd=x5   opcode (arithmetic)
```

`addi` means *add immediate*: one of its operands is a constant, the **immediate**, stored inside the instruction itself. The **opcode** identifies the kind of instruction, `rd` names the destination register, and `rs1` names the source register.

The decoder in the CPU is a combinational circuit that divides the word into these fields and sets the multiplexers and the ALU accordingly. The fields occupy the same positions in most instructions, which keeps the decoder small.

Labels such as `loop` are a convenience of the assembler. In the machine code, the branch holds the distance to its target, here -8 bytes, and the CPU adds this distance to the program counter.

Large programs are now rarely written in assembly. Higher-level languages, the subject of Thinking in Programs, are translated into machine code by a compiler or run by an interpreter. Every program, in any language, is finally run as a sequence of instructions like these.

5.6 Memory and Addresses

Memory is a long sequence of bytes, each identified by a number called its **address**. A 32-bit processor can name 2 to the 32nd addresses, which is 4 GiB. A small computer has far less memory than that, and the remaining addresses are used for other purposes, as described below.

RISC-V is a **load-store** architecture. Arithmetic operates only on registers, and separate instructions move data between registers and memory:

```
lw   x7, 0(x10)      # load the word at address x10 into x7
sw   x7, 4(x10)      # store x7 at that address plus 4
```

A 32-bit word occupies four consecutive bytes. RISC-V stores the least significant byte at the lowest address, an arrangement called **little-endian**. The value `0x12345678` stored at address 100 places `0x78` at 100, `0x56` at 101, `0x34` at 102 and `0x12` at 103. Some other processors use the opposite order, **big-endian**, and data moved between the two must have its bytes swapped.

5.7 Subroutines and the Stack

Programs reuse pieces of code by calling them as **subroutines**, also called functions or procedures. The jump-and-link instruction `jal` jumps to a subroutine and saves the address of the following instruction in a register, by convention `x1`, the return address. At its end, the subroutine jumps back to that address with `jalr`.

A subroutine that calls another subroutine would overwrite `x1`, so it must first save the old value. It saves it on the **stack**, an area of memory in which the last item stored is the first removed. A register, by convention `x2`, the stack pointer, holds the address of the top of the stack. A subroutine makes room by lowering the stack pointer and saves the registers it must preserve. Before it returns, it restores them and raises the pointer again. Each active call has its own **frame** on the stack, which also holds its local variables.

The stack allows one function to call another, which calls another, to any depth the memory allows, and gives each call its own copies of its variables. When a program runs out of stack space, usually because a function calls itself without stopping, the result is a

stack overflow.

5.8 The Bus and the Address Map

The CPU, memory and devices are connected by the **bus**: wires for an address, wires for data, and a few control signals that indicate whether a transfer is a read or a write and when it is complete.

Every device on the bus is assigned a range of addresses. A decoder examines the top bits of each address and enables the one device to which it belongs. The resulting list of which device answers at which address is the computer's **address map**. The following is a simplified map of the Zeitlos system on a chip described in The Machine:

Address	Device
0x0000_0000	boot program, in on-chip memory
0x1000_0000	flash memory
0x2000_0000	video memory: the screen
0x4000_0000	main memory (RAM)
0x8000_0000	the running program's memory
0xE000_0000	general-purpose pins
0xF000_0000	serial port

The range at `0x8000_0000` is not a separate device: it shows the memory of whichever program is running, as described under Protecting Memory. The address alone determines which device is used. A store to `0x4000_0000` changes memory; a store to `0x2000_0000` changes pixels on the screen.

The bus inside Zeitlos follows Wishbone, an open standard. Other systems use other buses, which perform the same function.

5.9 Memory-Mapped I/O

In the map above, the screen, the pins and the serial port are not reached with special instructions; they have addresses in the same way as memory. Storing a value at a device's address sends it to the device, and loading from the address reads from the device. This is **memory-mapped I/O**, and nearly every modern processor communicates with hardware in this way.

The addresses a device answers to are its **registers**, which are distinct from the CPU's registers. A typical device has a few:

- a **data** register, holding a value to send or one recently received
- a **status** register, whose bits report conditions such as "a byte has arrived" or "ready to send"
- a **control** register, whose bits turn features on and off

In a simple system, storing a word at `0xE000_0000` sets a row of output pins to its lowest bits. (On a real machine, the individual pin registers each have their own address within this range, given in the machine's documentation.) On such a system, these two RISC-V instructions place the total computed earlier on the pins:

```
lui  x7, 0xE0000     # x7 = 0xE000_0000, the pins' address
sw   x6, 0(x7)       # send x6 to the pins
```

The instruction `lui` (load upper immediate) places its 20-bit constant in the top 20 bits of a register and clears the lower 12, so `0xE0000` becomes `0xE000_0000`. A constant that needs all 32 bits is built with `lui` followed by `addi`. Because `addi` treats its 12-bit constant as signed, the upper part is increased by one whenever bit 11 of the lower part is set; assemblers do this automatically.

Software therefore affects the physical world with a store to the appropriate address. The chapter Systems on a Chip follows such a register from the hardware design to the program that uses it.

5.10 Interrupts

A program can monitor a device by reading its status register repeatedly until it changes. This is called **polling**. It is simple, but the processor can do nothing else while it waits.

An **interrupt** allows the device to request attention instead. When a key is pressed, for example, the keyboard circuit raises an interrupt signal. The CPU finishes its current instruction, saves the program counter and jumps to a routine called an **interrupt handler**. The handler reads the key, stores it and returns, and the interrupted program continues as if it had not been interrupted.

A **timer** is the most important source of interrupts. It counts clock cycles and raises an interrupt at regular intervals, for ex-

ample several hundred times a second. These regular interrupts allow an operating system to share one processor between several programs, as described in Operating Systems.

5.11 Direct Memory Access

Some tasks involve moving large amounts of data: filling the screen, copying a file into memory or sending a network packet. The CPU can do this one word at a time, but some devices can do it themselves. A device that can read and write memory over the bus without the CPU is a **bus master**, and the technique is called **direct memory access** (DMA). A common example is a graphics engine that copies rectangles of pixels, often called a **blitter**. The CPU configures it with a few register writes and continues with other work while the copy proceeds.

5.12 Caches

Processor speeds have increased much faster than memory speeds. Reading a word from main memory can take many clock cycles, during which a simple CPU waits.

A **cache** is a small, fast memory inside the processor that keeps copies of recently used memory. When the CPU needs a word, it checks the cache first. A **hit** is fast. A **miss** fetches the word from main memory, usually together with its neighbours, and keeps them in the cache.

Caches are effective because programs access memory in two predictable ways. A program that uses a location is likely to use it again soon, as loops do. A program that uses a location is also likely to use its neighbours soon, as when reading through an array or running instructions in sequence. As a result, a small cache can serve most accesses.

Caches have costs. They make timing less predictable, and they must never hold copies of device registers, whose values can change without the CPU's knowledge. Systems therefore mark device address ranges as uncached.

5.13 Speed

A processor's speed depends on two factors: the number of clock cycles it runs per second, and the work it does per cycle, measured as **instructions per cycle** (IPC).

A simple processor takes several cycles per instruction: one to fetch, one to decode, and so on. A **pipelined** processor overlaps these steps, fetching the next instruction while decoding the current one, in the manner of an assembly line, and can approach one instruction per cycle. Large modern processors go much further. At billions of cycles per second, they run several instructions at once, predict the outcome of branches before it is known, and reorder instructions as they run.

A simple 32-bit processor in an FPGA, running at 48 MHz and completing an instruction every four or five cycles, performs roughly ten million instructions per second. This is a thousand or more times slower than a modern desktop computer. It is sufficient to edit text, run a spreadsheet, browse simple web pages, play music and control machinery, provided the software is written with care.

Chapter 6

Memory and Storage

A computer needs memory for the program it is running and the data it is working on, and storage that keeps information while it is switched off. No single technology serves both purposes well. This chapter describes the kinds of memory in use, the purposes each suits, and how long each can be relied on to retain its contents. The last of these is central to timeless computing.

6.1 The Memory Hierarchy

Every memory technology trades speed against size, cost and permanence. Computers therefore use several kinds at once, arranged from small and fast to large and slow:

Level	Typical size	Speed	Keeps data
CPU registers	128 bytes	fastest	no
cache	kilobytes	faster	no
main memory (RAM)	MB to GB	fast	no
storage: flash, F-RAM, disk	KB to TB	slower	yes
archive: offline copies	any	slowest	yes

Main memory is **random-access memory** (RAM): any address can be read or written as quickly as any other. Memory that loses its contents when power is removed is **volatile**. Memory that retains them is **non-volatile**. A computer runs from volatile

memory, because it is fast and can be rewritten without limit, and keeps everything of lasting value in non-volatile storage.

6.2 Static RAM

Static RAM (SRAM) stores each bit in a small loop of transistors, similar to the latch in Digital Logic. It holds its value while powered, needs no maintenance, and is fast and simple to use: when an address is placed on its pins, the data appears. Its disadvantage is size. Each bit needs about six transistors, so SRAM is expensive per bit.

Caches, and often a processor's register file, are made of SRAM, as is the **block RAM** inside an FPGA. Some small computers use an external SRAM chip of one or two megabytes as their main memory, which needs almost no circuitry to control.

6.3 Dynamic RAM

Dynamic RAM (DRAM) stores each bit as a small charge on a capacitor, with one transistor to reach it. This makes it dense and cheap, so it is used for main memory in nearly every computer. The charge leaks away, however. To be certain that no bit is lost, every row of the memory is read and written back, or **refreshed**, at least every 64 milliseconds. DRAM also has a more complicated interface, organized in rows and columns that must be opened and closed in the correct order.

A **memory controller**, a circuit between the bus and the memory chips, handles these tasks. In an FPGA computer, the memory controller is part of the design loaded into the FPGA. Common varieties are:

- **SDRAM** (synchronous DRAM) transfers data in step with a clock.
- **DDR** SDRAM, in generations such as DDR3, transfers data on both edges of the clock, doubling the rate, at the cost of a much more demanding controller.
- **PSRAM** (pseudo-static RAM) is DRAM with its own refresh circuit built in, so that it can be used almost like SRAM.

6.4 Read-Only Memory and Its Descendants

A computer needs some instructions to run as soon as it is switched on, before anything has been loaded. Early computers kept them in **read-only memory** (ROM), written once at the factory. Later came programmable ROMs that the user could write once, then ones that could be erased with ultraviolet light, and then **EEPROM**, which can be erased and rewritten electrically one byte at a time, though slowly and a limited number of times.

EEPROM is still common for small amounts of data, such as settings and calibration values, and retains data for a century or more.

6.5 Flash Memory

Flash memory is a descendant of EEPROM that is erased in large blocks rather than byte by byte, which makes it much denser and cheaper. It is now the most common form of non-volatile storage. Each bit is a charge trapped on an insulated piece of silicon inside a transistor, where it can remain for years without power.

Three properties of flash determine how it is used:

- **It must be erased before it is written.** An erased bit reads as 1, and writing can only change 1s to 0s. To change any bit from 0 back to 1, the whole block containing it, often 4 KB or more, must be erased and rewritten.
- **It wears out.** Each block survives a limited number of erase cycles, often around 100,000 for the kind used to hold programs and far fewer for the densest kinds. After that it begins to fail.
- **It gradually loses data.** The trapped charge leaks. Typical retention is ten to twenty years, and less at high temperatures or in a worn block.

There are two main kinds. **NOR flash** can be read one byte at a time at any address, which makes it suitable for holding programs that run directly from it. A small NOR flash chip, connected over four wires using SPI, a serial connection described in Talking to the World, holds the configuration of most FPGA boards.

NAND flash is denser and is read in pages. It is used in SD cards, USB sticks and solid-state drives. Each of these contains a small controller that hides the erasing, spreads writes across the whole chip to even out the wear (**wear levelling**), and replaces bad blocks as they appear.

The controller also introduces a risk: if it fails, all the data behind it can become unreachable at once, without warning. For this reason, SD cards and USB sticks are suitable for moving data but not for keeping it for decades.

6.6 Ferroelectric RAM

Ferroelectric RAM (F-RAM, or FRAM) stores each bit in the electric polarization of a small crystal, which switches one way or the other when a voltage is applied and keeps that state when the voltage is removed. It combines the main advantages of RAM and flash:

- It is non-volatile.
- It is written one byte at a time, at full speed, with no erase step.
- It can be rewritten a trillion (10 to the 12th) times or more, so that in practice it does not wear out.
- It retains data for a long time. One manufacturer specifies around 95 years at 55 degrees Celsius and more than 200 years at 35 degrees.

Its disadvantages are capacity and cost. F-RAM chips hold from kilobytes to a few megabytes, and cost far more per bit than flash. For text, programs, keys, settings and logs that must survive for generations, the trade is worthwhile. A text document of a few pages fits in 8 KB.

Machdyne's long-term storage devices are built around F-RAM, in three forms:

- **Blaustahl**, a USB device with 8 KB of F-RAM, of which 7,680 bytes hold text: a few pages. It contains its own text editor, used from any terminal program over a USB serial connection, so it needs no driver and no particular operating system.
- **Kaltstahl**, a larger version holding 256 KB, which can also appear to a computer as an ordinary USB drive.

- **FERRIT**, a modular archival system: a controller and removable memory cards in a backplane, with room for hundreds of megabytes. The controller presents the cards to a computer as a USB drive, and the cards can be moved to another controller.

All three are open designs, so that a device can be repaired, rebuilt or read by other means if its maker is gone.

Magnetoresistive RAM (MRAM) stores bits magnetically and has properties similar to those of F-RAM: it is non-volatile, fast, byte-writable and durable.

Two cautions apply to all retention figures. First, they are manufacturers' estimates from accelerated testing, not observations of chips that have been stored for two centuries. Second, they depend strongly on temperature: memory kept cool lasts much longer than memory kept warm.

6.7 Other Long-Term Media

Not all storage is electronic. **Hard disks** store data magnetically on spinning platters. They are cheap and large, but their motors and bearings wear out. **Magnetic tape** is still used for archives, because it is cheap and stable when stored well, but it needs a working tape drive to read it. **Optical discs** can last decades if they are of good quality and kept dark and cool.

Paper is also a storage medium. A printout of a short program, a key or a set of instructions, kept dry, can be read in a century without any machine. For the most important information, it is a serious option.

6.8 Errors and How to Catch Them

Storage is not perfect. A worn flash cell, a cosmic ray striking a memory chip, an interrupted write or a failing cable can each change a bit. The defence is to store a little extra information that allows errors to be detected and sometimes corrected.

- A **parity bit** records whether the number of 1s in a group of bits is odd or even. A single changed bit changes the parity and is detected.

- A **checksum** is the sum of all the bytes of a block. If the sum no longer matches, the block has changed.
- A **cyclic redundancy check** (CRC) is a more thorough kind of checksum that detects almost all accidental damage. Disks, network packets and many file formats use one.
- A **cryptographic hash**, such as SHA-256, produces a fingerprint that changes completely if even one bit of the data changes, and that cannot practically be forged. It is used to check that a file has not been altered, whether accidentally or deliberately.
- An **error-correcting code** (ECC) stores enough extra bits not only to detect a changed bit but also to identify which bit it was and correct it.

The most important practice is to read back anything important after writing it and compare it with the original. A copy that has not been verified cannot be relied on as a backup.

6.9 Filesystems

A storage device is a long sequence of blocks. A **filesystem** is the set of rules that organizes those blocks into named files and folders. It records which blocks belong to which file and in what order, which blocks are free, and the names, sizes and dates of the files.

FAT, originally designed for floppy disks in the late 1970s, is the most widely understood filesystem in the world. Nearly every computer, camera and operating system can read it. For this reason it is the usual choice for SD cards and USB sticks, and a sensible choice for data that should outlast the computer that wrote it.

6.10 Surviving Power Loss

The most difficult problem for a filesystem is a power failure in the middle of a write. Changing a file usually means changing several blocks: the data, the list of blocks and the directory entry. If power fails after some of those writes but not others, the filesystem can be left inconsistent, with a file pointing at the wrong data, or with space marked as used that belongs to no file.

Careful designs follow a few rules:

- **Never overwrite in place.** Write the new version of a file to an unused area.
- **Commit with a single write.** Once the new version is complete and checked, make it current by changing one small value, ideally one byte, that a power cut cannot leave half-written.
- **Spread the writes.** On memory that wears, use free blocks in rotation instead of always reusing the same ones.

The small filesystem used by Machdyne BASIC on its modules follows these rules. Wherever the power is cut, every file is either as it was before the operation or as it is after it; at worst, the end of a line being appended is lost.

Large filesystems use a **journal**, a log in which each change is recorded before it is carried out, so that an interrupted change can be completed or undone at the next start.

Users should likewise let the computer finish writing before removing a card or switching off. Systems provide a command or menu item for this, often called sync, eject or unmount.

6.11 Choosing Storage That Lasts

Beyond its retention, two properties of a storage device decide how long the data on it remains usable:

- **Write protection.** Data that should not change is protected from accidental writes. Many memory chips and devices have a write-protect pin or switch.
- **A readable interface.** Storage is useful only while something can still read its interface. Simple, well-documented interfaces, and documentation kept with the data, make it likely that something always can.

The practices that keep data intact over decades, with several copies, plain formats, regular checks and refreshing onto new media, are collected in Computing That Lasts.

Chapter 7

Talking to the World

A computer is useful only if it can sense or affect something outside itself. This chapter describes the ways a computer exchanges signals with the world: single pins, the common serial connections between chips and machines, and the larger interfaces for keyboards, screens and sound. Each is built from the memory-mapped registers and logic levels described in the previous chapters.

7.1 General-Purpose Pins

The simplest connection is a single pin whose level software can read or set. These are **general-purpose input/output** pins, or GPIO. Each pin can usually be configured as:

- an **input**, which reads whether the voltage on the pin is high or low
- a **push-pull output**, which drives the pin actively high or low
- an **open-drain output**, which can only pull the pin low and otherwise allows a pull-up resistor to take it high

GPIO pins are almost always memory-mapped. A **direction register** selects input or output for each pin, one bit per pin, and a **data register** holds the levels: writing it sets the outputs, and reading it reports the inputs. To change one pin without disturbing the others, a program reads the register, changes one bit and

writes the register back.

Two practical problems arise at once.

Switches bounce. When a mechanical button closes, its contacts make and break several times over a few milliseconds before settling. A program that counts presses will count several for each push. The remedy, called **debouncing**, is to accept a reading only after it has been stable for some milliseconds.

Pins supply little current. A pin can supply a few milliamps, which is enough for an LED through a resistor and not much more. A motor, relay, lamp or solenoid needs a transistor or driver chip, controlled by the pin, to switch the current. A relay or motor coil also needs a **flyback diode** across it, because when the current through a coil is switched off, the coil produces a brief high voltage that can destroy the transistor.

7.2 Analog Signals

Many quantities in the world, such as temperature, light, sound and the position of a knob, vary continuously rather than in steps.

An **analog-to-digital converter** (ADC) measures a voltage and reports it as a number. A 10-bit ADC divides its range into 1,024 steps, so with a 3.3 V range each step is about 3.2 millivolts, and a reading of 512 means about 1.65 V. Many microcontrollers have built-in ADC inputs. Sensors that produce a voltage, such as a light-dependent resistor, whose resistance falls as the light increases, in series with a fixed resistor, can be read directly.

A **digital-to-analog converter** (DAC) produces a voltage from a number. A cheaper method produces nearly the same effect with an ordinary digital pin, by switching it on and off rapidly and varying the fraction of time it is on. This is **pulse-width modulation** (PWM). An LED driven at 25 percent on-time gives a quarter of the light, and a motor turns at about a quarter of its speed. A simple filter of a resistor and capacitor smooths a fast PWM signal, or a similar pulse stream, into a varying voltage. Many simple computers produce sound in this way.

7.3 Serial and Parallel

A byte can be sent from one chip to another over eight wires, one per bit, all at once. This is a **parallel** connection. Alternatively, it can be sent over one wire, one bit after another. This is a **serial** connection.

Parallel connections were common when chips were slow. Serial connections have since replaced them almost everywhere, because they need fewer wires and pins, and because at high speeds it is difficult to keep many parallel wires exactly in step. The shift register from Digital Logic performs the conversion at each end. USB, Ethernet, SATA and HDMI are all serial.

Three simple serial connections are widely used between chips and small devices: UART, SPI and I2C.

7.4 UART

A **UART** (universal asynchronous receiver-transmitter) sends bytes over a single wire in each direction, with no shared clock. Instead, both ends agree in advance on the speed, the **baud rate**, in bits per second. Common rates are 9600 and 115200.

The wire is high when idle. To send a byte, the transmitter pulls it low for one bit time, the **start bit**, which tells the receiver to begin timing. The eight data bits follow, least significant first, and finally a high **stop bit**. The letter `A`, which is `0x41` or `0100 0001`, is sent as follows:

```
        idle  start  b0    b1   b2   b3   b4   b5   b6    b7   stop
wire    high  low    high  low  low  low  low  low  high  low  high
```

The receiver samples each bit near its middle, using its own clock. Because the timing restarts at every start bit, the two clocks need only agree to within a few percent.

A UART connection needs three wires: transmit, receive and ground. Each device's transmit wire connects to the other's receive wire. Connecting them the wrong way round is the most common reason a serial link shows nothing.

The UART is the oldest and most tolerant means of communicating with a computer. A **serial console** is a text terminal connected to a computer over a UART. It is often the first output

a new board produces, and the remaining means of access when everything else has failed. Older computer serial ports use the RS-232 standard, whose voltages swing to plus and minus 12 V. Never connect these directly to a 3.3 V pin; an adapter converts between the two.

7.5 SPI

SPI (serial peripheral interface) is a fast, simple connection between one **master** and one or more **slaves**, using four wires:

- **SCK**, the clock, driven by the master
- **MOSI** (master out, slave in), data from the master
- **MISO** (master in, slave out), data to the master
- **CS**, chip select, one per slave, pulled low to select it

The master selects a slave by pulling its CS low, then pulses the clock. On each pulse, one bit moves out on MOSI and one bit moves in on MISO. Internally, the arrangement is two shift registers joined in a ring. Because the master supplies the clock, no speed needs to be agreed in advance, and SPI can run at tens of megahertz.

SPI connects flash memory chips, F-RAM, SD cards, displays and many sensors. Machdyne's MMOD memory modules are SPI devices on a six-pin socket carrying the four SPI signals, ground and 3.3 V.

7.6 I2C

I2C (inter-integrated circuit, sometimes written IIC, and called TWI, two-wire interface, by some manufacturers) connects many devices using only two wires:

- **SCL**, the clock
- **SDA**, the data

Both wires are open-drain with pull-up resistors, as described in Electricity, so any device can pull them low and none can drive them high. One device, the **master**, starts every transfer and drives the clock. The others, the **slaves**, respond when addressed. The two roles are sometimes called controller and target.

Each slave has a 7-bit **address**. A transfer begins with a **start condition**, in which SDA falls while SCL is high, followed by the address and a bit indicating read or write. The slave with that address pulls SDA low to **acknowledge**, and bytes then follow, each acknowledged in turn by the device receiving it. A **stop condition**, in which SDA rises while SCL is high, ends the transfer.

I2C is slower than SPI, typically 100 or 400 kilobits per second, but its two wires can reach dozens of devices: temperature sensors, real-time clocks, small displays, EEPROMs, pin expanders that add sixteen pins over the two wires, and others. Because every device has an address, a master can **scan** the bus to determine what is connected, by trying each address in turn and noting which ones acknowledge.

7.7 Protocols on Top of Buses

A bus such as I2C only moves bytes. The meaning of the bytes is defined by a **protocol** layered on top.

The Sechs interface used by Machdyne's programmable modules is an example of a protocol designed to tolerate mistakes. The interface defines six contacts: an I2C bus on two, two local pins for the module's own use, ground and 3.3 V. A host on the I2C bus can find every module, identify it, start and stop its program, and load new programs into it. The protocol also makes guarantees that protect the system:

- At power-on, a module keeps all its pins inactive, so it cannot disturb the circuit it is connected to.
- For a short **boot window** after power-on, the host can always halt a module, whatever program it holds. A program cannot remove this window, so a module can always be recovered.
- A new program replaces the old one only after it has been received completely and checked.
- The module does not run a program that would break these rules, for example one that drives the bus pins while a bus is connected.

The same principles apply to protocols in general. A well-designed protocol starts in a safe state, always provides a means of recovery,

and never leaves a change half-completed.

7.8 USB

USB (universal serial bus) connects keyboards, mice, storage and most other peripherals to a computer. It is far more complex than the buses above; this section describes its general organization.

USB has a **host**, the computer, and **devices**. The host controls all communication; devices transmit only when asked. When a device is plugged in, the host asks it to describe itself, a process called **enumeration**. The device responds with **descriptors** that identify it.

Many devices belong to standard **classes**. A keyboard or mouse is a human interface device (HID), a USB stick belongs to the mass storage class (MSC), and a serial adapter to the communications device class (CDC). A host that supports a class works with every device of that class without additional software. As a result, a USB keyboard from any manufacturer works on any computer without a driver of its own. This is particularly valuable for long-term storage devices: one that appears as a standard drive can be read by any computer that supports the class.

Each USB connector also carries 5 V power, from which many small devices, and some small computers, are powered.

7.9 Video

A video display draws its picture as a **raster**: a beam, or its modern equivalent, sweeps across each line from left to right and steps down line by line from top to bottom, then returns to the top and begins again. The signal carries the colour of each pixel in turn, together with **sync** pulses that mark the end of each line and of each frame.

The standard VGA mode of 640 by 480 pixels at sixty frames a second has a pixel clock of about 25 MHz. Including the time spent in the margins and sync pulses, each frame occupies 800 by 525 pixel times.

A computer produces this signal from a **framebuffer**, an area of memory holding one number per pixel. A video circuit reads

the framebuffer in step with the raster and converts each number into a colour. A program draws by writing to the framebuffer. In an FPGA computer, the video circuit is part of the design, so one machine can offer several screen modes and several kinds of output:

- **VGA** sends the colours as analog voltages on separate wires.
- **DVI** and **HDMI** send the same picture digitally, as fast serial streams.
- **Composite** video combines the picture into a single signal in the format of older televisions.

7.10 Audio

Sound output is the reverse of the sampling described in Information. A stream of numbers, one per sample, is converted back into a voltage by a DAC, or by a fast pulse stream and filter, and then amplified. Sixteen-bit samples at tens of thousands per second are sufficient for good sound. A **mixer** adds several streams together, so that music, speech and sound effects can play at the same time.

7.11 Keyboards and Mice

A keyboard does not send letters. It sends **scan codes**, numbers that identify which key was pressed and which was released. The computer translates them into characters according to a **keyboard layout**, so the same key produces a Z under one layout and a Y under another. Holding Shift, or a key such as AltGr, adds further state to the translation.

A mouse reports small movements, the distance moved left or right and up or down since the last report, together with the state of its buttons. The computer adds up the movements to determine the position of the pointer.

7.12 Connectors and Modules

Expansion connectors allow a small computer to be extended. Many FPGA boards use **Pmod** connectors: rows of six pins, with four signals plus ground and 3.3 V, often doubled to twelve. Because the connector is simple, a UART, SPI or I2C connection fits on one Pmod row, and the same socket can accept a memory module, a network adapter, a sensor, a display or a programmable module.

This is the modularity principle from Timeless Computing in practice. When a new need arises, a small, inexpensive module meets it, and the computer itself remains unchanged.

7.13 Connecting a Device

Adding a device to a computer follows the same steps whatever the device. A temperature sensor of the LM75 family, a design that has been made by several manufacturers since the 1990s, serves as an example.

The data sheet answers four questions before anything is connected:

- **Supply voltage.** The sensor works from 3 to 5.5 V, so it can be powered from the 3.3 V pin of a Pmod connector, and its signals are then at 3.3 V as well.
- **Bus and address.** It is an I2C slave. Its address is 1001 followed by three bits set by its pins A2, A1 and A0. With those pins connected to ground, the address is binary 1001000: 0x48, or 72.
- **Pull-ups.** The bus needs pull-up resistors on SCL and SDA, typically 4.7k to 3.3 V. Small boards that carry such a sensor often include them; if neither the board nor the computer has them, they must be added.
- **What to read.** Register 0 holds the temperature. Its first byte is the temperature in whole degrees Celsius, as a signed 8-bit number in two's complement.

The wiring connects four pins: supply to 3.3 V, ground to ground, SCL to SCL and SDA to SDA. Before power is applied, the checks in Electricity apply: shared ground, matching voltages, no short between supply and ground.

The first test is a scan of the bus. If the sensor acknowledges address 72, it is powered and wired correctly. If no device answers, the usual causes, in order, are a missing pull-up, SCL and SDA swapped, no power, and a wrong address.

The reading is then one byte from register 0. A byte of 23 means 23 degrees. A byte above 127 is negative, as described in Information: 251 means 251 - 256 = -5 degrees. The chapter BASIC reads this sensor with a program of eight lines.

Every device is added in this way: the data sheet is read, the voltage and bus are checked, the device is connected with the power off and confirmed to answer, and only then is the program that uses it written.

Chapter 8

Operating Systems

A processor core follows one program at a time. A desktop computer nevertheless appears to run many programs at once, shares one screen and one disk among them, and prevents a faulty program from stopping the others. The software that does this is the **operating system** (OS). This chapter describes what an operating system does and the main choices in designing one.

8.1 When an Operating System Is Needed

Not every computer has an operating system. A small module that reads a sensor and switches a pump runs a single program, written for its hardware, from the moment it is powered on. This is called running on **bare metal**. For one task on one small device it is often the better choice, because there is less to go wrong and less to understand.

An operating system is needed when a computer must run several programs, share devices among them, store files, and be used by a person for many different tasks. It has four broad functions:

- **Sharing** the processor, memory and devices among programs.
- **Abstraction** of the hardware, so that a program can write a file without knowing how the SD card works.

- **Protection** of programs from each other, and of the system from them.
- **Services** that most programs need, such as files, windows, networking and time.

8.2 Booting

When power is applied, a processor starts fetching instructions from a fixed address. The program at that address must be present before anything else has happened, so it is kept in non-volatile memory. This first program is part of the computer's **firmware**, the software built into the hardware itself. It sets up the hardware and starts a **bootloader**, a program whose task is to load another program: it finds the operating system, copies it into memory and jumps to it. On a PC the firmware, called UEFI (and on older PCs the BIOS), loads a separate bootloader from the disk. On a small computer the firmware's own boot program often loads the operating system directly. The core of the operating system, the **kernel**, then starts its drivers and finally the first program the user sees: a command prompt, a desktop or an application.

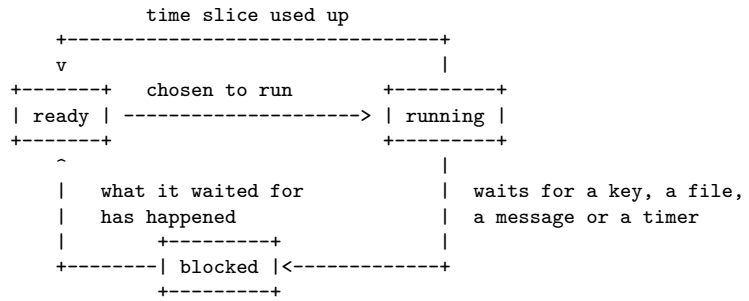
An FPGA computer has one additional step at the beginning. Before there is a processor, the FPGA must load its own configuration, the design that makes it a computer, from a flash memory chip. Only then does the processor defined by that design begin to fetch instructions.

The boot sequence is also the usual means of recovery. A well-designed system offers a way to interrupt booting, usually by holding a key, and to start something simpler and more reliable instead: a minimal console, a known good copy of the system, or a way to load a new one. This is the recoverability principle from Timeless Computing. The owner of any computer should know how to reach this recovery path.

8.3 Processes

A program on disk is a file. A program that is running is a **process**: its code and data in memory, its stack, the contents of its registers, and the resources it is using, such as open files. The kernel keeps a table of processes, each identified by a number, its process ID.

At any moment, each process is in one of a few states:



- **Running**: on the processor now. With one processor, only one process is running.
- **Ready**: able to run, and waiting for its turn.
- **Blocked**: waiting for something, such as a key press, data from a disk or a message, and not using the processor.

Most processes spend most of their time blocked. A text editor does nothing until a key is pressed. For this reason one slow processor can serve many programs, since only a few need it at any moment.

8.4 Scheduling

The part of the kernel that decides which ready process runs next is the **scheduler**.

In **cooperative** multitasking, each program runs until it voluntarily gives up the processor. This is simple, but one program that never yields, whether through a bug or poor design, stops the whole system.

In **preemptive** multitasking, a timer interrupt, described in The Computer, takes the processor away from the running program at regular intervals. The kernel saves that program's registers,

chooses another ready process, restores its registers and lets it run. This exchange is a **context switch**. Each process runs for a short **time slice**, typically a few milliseconds. Because this happens hundreds of times a second, every program appears to run continuously.

A scheduler can aim for **throughput**, the most total work done, or for **responsiveness**, a quick reaction when something happens. For a computer used by a person, responsiveness matters most. A well-designed scheduler runs a process as soon as the event it was waiting for arrives, so that a key press is handled at once, even on a slow processor.

8.5 Protecting Memory

The kernel decides where in memory each process is placed, and allocates more memory to processes that request it.

It must also prevent one process from damaging another. A program with a bug might write through a bad address into another program's memory, or into the kernel. Without protection, one bug can stop the whole machine. Protection ranges from none at all to full virtual memory, in the following steps.

No protection. Every program can read and write all of memory. Small and early systems worked this way. It is simple and fast, but one faulty program can corrupt everything.

A memory protection unit (MPU) checks every access against a small set of **regions**. Each region is a range of addresses with permissions: whether the running process may read it, write it or run instructions from it. On each context switch, the kernel loads the regions of the next process. An access outside them is stopped before it takes effect, and the processor reports it to the kernel as a **fault**. The kernel can then stop the faulty process, report what it did and let the others continue. An MPU does not change addresses: an address means the same location in every process. It is small and fast, and common in microcontrollers.

Address translation changes the address a program uses, its **virtual address**, into a different **physical address** in memory. The simplest form adds an offset. A range of virtual addresses is set aside as a window, and the hardware adds the running process's offset to every address in that window. Every program can

then be built to run at the same virtual address, wherever the kernel places it. Combined with an MPU that limits each process to its own window, this gives each program a private, protected area of memory with little hardware.

A **memory management unit** (MMU) translates in finer detail. It divides memory into **pages**, typically of 4 KiB, and looks up each virtual page in a **page table** kept by the kernel, which says where the page is in physical memory and what may be done with it. Each process has its own table and therefore its own complete view of memory, called **virtual memory**. Pages can be shared between processes, created only when first used, or moved to disk when memory runs short. To make this fast, the MMU holds recent translations in a small cache. Linux in its usual form depends on an MMU.

With any form of translation, the same virtual address means different memory in different processes. A process therefore cannot hand another a pointer to its own data; the kernel copies the data, or translates the address, when it passes data between them.

Protection also requires **privilege levels**. The kernel runs at a higher privilege than ordinary programs, and only privileged code can change the protection settings. Otherwise a program could switch the protection off.

8.6 System Calls

Because ordinary programs are not permitted to do everything, they ask the kernel to act for them. A **system call** is such a request: to open a file, read some bytes, send a message, start a program or wait for an event. Each operating system has its own set of system calls, and the standard library of each programming language provides them as convenient functions.

8.7 Drivers

A **driver** is the part of the system that operates one particular device. It knows which registers to write, in what order, and what the replies mean. The rest of the system uses the device through a simpler, general interface. To a program writing a file, an SD card, a USB stick and a RAM disk appear the same.

8.8 Files and Names

A filesystem, described in Memory and Storage, organizes a storage device into named files in folders. The operating system joins all its storage devices into one **namespace**, the set of names a program can use.

Unix-like systems, and many systems influenced by them, use a single tree of names written as **paths**, such as `/home/notes.txt`. The leading `/` is the root of the tree. Each device is attached, or **mounted**, at some point in the tree, so that a USB stick might appear as `/usb` with its contents below that. Other systems, such as MS-DOS and Windows, give each device a letter instead.

The advantage of a single tree is that everything in it is accessed in the same way. A program that can read files can read them from any device, and some systems present devices, settings and network resources as files in the tree.

8.9 Communication Between Processes

Processes often need to work together. A text editor asks the filesystem for a file, and a window manager tells an application that a key was pressed. Operating systems provide several methods:

- **Messages**: small packets of data sent from one process to another through the kernel.
- **Pipes**: one-way streams of bytes from one process to another. Unix shells use them to chain commands, each feeding its output to the next.
- **Shared memory**: a region that both processes can read and write. It is fast, but the processes must coordinate so that neither changes data the other is using.

8.10 Doing Several Things at Once

Whenever two activities share data, the order in which their steps happen matters. The activities can be two processes using shared memory, two **threads** of one program (parts of it that run independently), or a program and an interrupt handler.

Adding one to a counter in memory takes three instructions: load the value, add one, store it. If an interrupt arrives between the load and the store, and the handler also adds one to the counter, the program then stores its own result and the handler's addition is lost:

```
program                 interrupt handler       counter
load counter (5)                                   5
                        load counter (5)
                        add one, store (6)         6
add one, store (6)                                 6
```

This is a **race condition**: the result depends on the timing. It may appear once in a million runs, which makes it among the hardest errors to find. The section of code that must not be interrupted while it uses shared data is a **critical section**, and there are several ways to protect one:

- **Disabling interrupts** for the few instructions of the critical section, on a single processor. It is simple, but interrupts must not stay disabled for long.
- **Atomic instructions**, which read, change and write a memory location in one step that cannot be divided. RISC-V provides them in its A (atomic) extension.
- **Locks**: a lock is a variable that a process sets before entering a critical section and clears afterwards, while any other process that finds it set waits. A lock is itself set with an atomic instruction or with interrupts disabled.
- **Messages**: if only one process owns the data, and others ask it for changes by message, nothing is shared.

Locks bring their own danger. If process A holds lock 1 and waits for lock 2, while process B holds lock 2 and waits for lock 1, both wait for ever. This is a **deadlock**. It is avoided by always taking locks in the same order.

8.11 Kernel Designs

The largest design decision in an operating system is how much belongs inside the kernel.

A **monolithic kernel** contains the scheduler, memory management, filesystems, drivers and networking in one privileged program. Linux is monolithic. Every part can call every other part directly, which is fast, but a bug in any driver can crash the whole system.

A **microkernel** contains only the essentials: scheduling, memory, messages and interrupts. Filesystems, drivers and networking run as ordinary processes, which communicate with each other and with applications through messages. A driver that crashes can be restarted like any other program, and each part is small enough to understand on its own. The cost is the time spent passing messages, which careful design keeps small.

Zeitlos, described in The Machine, is a microkernel.

8.12 User Interfaces

The part of the system that people see is a program like any other.

A **command shell** reads commands typed by the user, runs them and shows the results. It is a simple and capable interface, and it can be used over a serial line or a network as easily as on the computer's own screen. A program that displays a text screen of this kind, for a local or remote shell, is a **terminal**.

The shell shows a **prompt** when it is ready, and each command is a line: the name of a program followed by its **arguments**, separated by spaces. In this short session in a Unix-like shell, the user lists the files in the current folder, shows one of them and copies it to a USB stick:

```
> ls
letters  notes.txt  programs
> cat notes.txt
Order fuses. Copy the card to the spare.
> cp notes.txt /usb/notes.txt
>
```

`ls` lists, `cat` shows a file and `cp` copies. A command that succeeds often prints nothing. Other shells use other names for the same commands, but the form is the same.

An interpreter for a programming language can serve as a shell. Such an interface is called a **REPL**, for read-eval-print loop: it reads an expression, evaluates it, prints the result and waits for the next. Many home computers of the 1980s started with a BASIC prompt of this kind.

A **graphical interface** has windows, a pointer and icons. A **window manager** decides where windows are placed, draws their borders and passes keyboard and mouse events to the correct application.

8.13 Simplicity as a Feature

A modern general-purpose operating system is large, because it must support thousands of devices, many users at once, and nearly every program ever written. A system for timeless applications can be much smaller. It needs to support one person, a known family of hardware, and a modest set of carefully written programs.

The benefit of a small system is that it can be understood. A system small enough to read can be trusted, repaired and changed by the person who uses it, and when something goes wrong there are fewer places to look.

Chapter 9

Communication

This chapter explains how computers exchange data, from a single wire between two machines to the Internet, and how they keep those exchanges private. The same principles apply at every scale.

9.1 A Link Between Two Machines

The simplest network is two computers joined by a serial cable, as described in Talking to the World. Each byte sent by one arrives at the other. Even this requires some agreement between the two: the same speed, the same way of marking where one message ends and the next begins, and some way to detect a byte that arrives damaged.

The first computer terminals were teleprinters, typewriters that sent each key press over a wire and printed whatever came back. Their screen-based successors added **escape sequences**: runs of characters, starting with the escape code, that move the cursor, clear the screen or change colours instead of printing. The sequences used by the DEC VT100 terminal of 1978 became a standard that nearly every terminal program still understands. A terminal program on a modern computer is in effect a simulated VT100.

For decades, people connected computers over the telephone network with **modems**, which convert bits into sounds and back. A common use was the **bulletin board system** (BBS): a computer

that others could dial into, one or a few callers at a time, to read and post messages, exchange files and play games. Many BBSes now run on the Internet and are reached with Telnet or SSH, programs that open a terminal on another computer across a network. They remain a text-based way for a community to meet.

9.2 Packets

A dedicated wire works for two machines, but connecting many machines this way is impractical, so they share connections instead. The idea that made modern networks possible is to divide every message into **packets**: short blocks of data, each preceded by a **header** that gives its source, its destination, its length and a checksum to detect damage. Packets from many conversations can then share the same wires, taking turns, and each finds its own way to its destination.

9.3 Layers

Network software is organized in **layers**. Each layer uses the one below it and offers a simpler service to the one above, in the same way that a program uses the operating system without knowing how the disk works.

Layer	Function	Examples
physical and link	move frames to neighbours	Ethernet, Wi-Fi, radio
network	carry packets across networks	IP
transport	deliver data to programs	TCP, UDP
application	provide a service to the user	SSH, HTTP, IRC, DNS

A message from an application passes down through the layers, each adding its own header. It travels across the network as a **frame** (the link layer's name for a packet), and passes back up through the layers at the other end, each removing its header. As a result, each layer can change independently of the others. The same web page can travel over a cable, over Wi-Fi or over a long-distance radio link without any change to the application.

9.4 The Link Layer

Ethernet connects machines on a local network by cable. Each network interface has a unique 48-bit **MAC address**, set at the factory. Data travels in frames that carry the sender's and receiver's MAC addresses and a CRC. A **switch** joins several cables and forwards each frame only to the port where its destination is.

Wireless networks such as **Wi-Fi** perform the same function by radio. Low-power radios such as **LoRa** send small messages over many kilometres at low speed. A **mesh** of such radios, in which each one passes on messages for its neighbours, can connect a whole area with no other infrastructure.

9.5 IP: Addresses and Routing

The **Internet Protocol** (IP) gives every machine on a network an **IP address**. The original form, IPv4, uses 32 bits, written as four bytes in decimal, such as `192.168.1.20`. The newer IPv6 uses 128 bits, which allows about 3.4 x 10 to the 38th addresses, so that every device can have its own.

Addresses are organized in networks. Machines on the same local network can reach each other directly. To reach anything else, a machine sends the packet to a **router**, its **gateway**, which forwards it to another router closer to the destination, and so on, hop by hop, until it arrives. No single router knows the whole route. Each knows only which way to send packets for each part of the address space. For this reason the Internet can grow without any one party managing all of it.

Two services make IP easier to use:

- **DHCP** assigns addresses automatically. A new machine asks the local network for an address, and a DHCP server assigns one and tells it which gateway to use.
- **DNS**, the domain name system, translates names such as `example.org` into IP addresses by querying a hierarchy of servers.

9.6 Transport: TCP and UDP

IP delivers packets on a best-effort basis. They may arrive out of order, twice, or not at all. The transport layer determines how this is handled.

UDP sends individual messages, called **datagrams**, with no guarantees. It is simple and quick, and it suits uses where a lost message does not matter or will be requested again, such as DNS lookups, time synchronization or live audio.

TCP builds a reliable, ordered stream of bytes on top of unreliable packets. It numbers every byte it sends. The receiver acknowledges what it has received, and the sender resends anything not acknowledged in time. The receiver puts everything back in order before passing it to the program. To the program, a TCP connection behaves like a pair of pipes, one in each direction: bytes go in at one end and come out at the other, all of them and in order.

Both TCP and UDP use **port numbers** to distinguish programs on the same machine. A web server listens on port 80 or 443, and an SSH server on port 22. A connection is identified by four numbers: the two IP addresses and the two ports.

9.7 Applications

The application layer provides the services that users and programs need. Many of the oldest application protocols are plain text, which makes them easy to understand and to debug by hand. A web browser requesting a page sends lines such as these:

```
GET /index.html HTTP/1.1
Host: example.com
```

The request ends with a blank line. The server replies with a status line, some headers, a blank line, and the page itself.

Some common protocols:

- **Telnet** and **SSH** provide a remote terminal. Telnet sends everything, including passwords, as plain text. SSH encrypts it.
- **HTTP** and **HTTPS** carry the web. Modern web pages are often too large for a small computer, but simple pages work well.

- **Gopher** and **Gemini** are much simpler alternatives to the web, built around text documents and menus, and suited to small machines.
- **IRC** is a long-established protocol for text chat in shared channels.
- **Email** is delivered by SMTP and collected by POP or IMAP.

9.8 Secure Connections

Packets pass through many machines on their way, and any of them could read or alter the data. Secure protocols protect against this with cryptography, described in the next chapter, Security. When a browser connects to an HTTPS site, or a terminal to an SSH server, the client confirms the server's identity, and sometimes the server the client's, agree on a secret key over the open network, and use it to encrypt the rest of the session. The protocols are called **TLS** and **SSH**. Both are standard, open and in use on nearly every networked computer.

9.9 Networks Without the Internet

The Internet is convenient, but it is not the only means of communication, and it may not always be available.

- A **local network** of a switch and a few cables, or a single serial link, needs nothing from outside.
- **Radio meshes** can carry short messages across a town or a valley without any infrastructure.
- **Store and forward** systems pass messages on whenever a link is available. Bulletin boards once exchanged mail this way, calling each other at night when telephone rates were lower.
- Data can be carried by hand on storage devices, a method known as **sneakernet**. It can still move large amounts of data faster than most networks.

A timeless computer should be useful with no network at all, and able to use whatever network is available when there is one. Plain text, simple protocols and open standards make both possible.

Chapter 10

Security

A computer is secure when it does what its owner intends and nothing else: its data is read only by those meant to read it, changed only by those meant to change it, and available when it is needed. This chapter describes the threats to those properties and the means of defending against them, of which cryptography is one.

10.1 Threat Models

Securing a computer begins with a **threat model**: what is to be protected, from whom, and at what cost. A personal machine in a home faces different threats from a server on the Internet, and measures that suit one are wasted effort or inadequate for the other. A notebook of private writing needs protection from someone who picks up the machine; a server needs protection from strangers on the network who try every address; a messaging device may need protection from an adversary who records everything sent and can afford to wait.

Most failures are not failures of cryptography. They come from lost or stolen equipment, weak or reused passwords, programs that trust their input, software from an unknown source, and the absence of a copy when something is deleted. The sections below cover these, and then the cryptography on which secure communication depends.

10.2 Physical Access

Whoever holds a machine can do almost anything with it. They can read its storage card in another computer, replace its configuration flash, attach a device to its pins or take it away. A password that locks the screen protects the running session from someone at the keyboard, but not the data on a card that can be removed.

Data that must remain private on a machine that may be lost or stolen is therefore kept **encrypted**, with a key that is not stored beside it. Equipment that matters is kept where others cannot reach it, in the same way as other valuables.

10.3 Passwords and Keys

A password is strong when it cannot be guessed, even by a program that tries billions of possibilities a second. Length matters more than complexity: a passphrase of five or six unrelated words, chosen at random, is both stronger and easier to remember than a short password of mixed symbols. Each service should have its own password, so that one leak does not open the others.

A well-designed system never stores a password itself. It stores the result of a hash function of the kind described below, designed to be deliberately slow, applied to the password together with a random value called a **salt**. To check a password, the system applies the same function and compares the results. Someone who copies the stored record must then spend the same effort for every guess, and the salt prevents a guess from being tested against many records at once. A system can also add a delay after each wrong attempt.

Cryptographic **keys** are long random numbers, far too long to remember. They are kept in files or on devices, and are protected as carefully as the data they protect. A key is also backed up as carefully as data, because an encrypted file whose key is lost is lost with it.

10.4 What Cryptography Provides

Cryptography is the use of mathematics to protect information. It provides three properties:

- **Confidentiality**: only the intended receiver can read the message.
- **Integrity**: any change to the message is detected.
- **Authentication**: the receiver can be certain who sent it.

Modern cryptography is public. Its methods are published, studied by others for years before they are trusted, and standardized; only the keys are secret. A method whose security depends on its design being secret cannot be checked, and usually fails once the design becomes known.

10.5 Hash Functions

A **cryptographic hash function** computes a short, fixed-size value, the **hash** or digest, from data of any length. SHA-256, a widely used example, produces 256 bits, usually written as 64 hex digits. A good hash function has three properties: the same data always gives the same hash; any change to the data, even of one bit, gives a completely different hash; and no one can in practice find two different pieces of data with the same hash, or data that produces a given hash.

As a result, a hash serves as a fingerprint. A copy of a file is checked by computing its hash and comparing it with the original's, as described in Memory and Storage. A program downloaded from elsewhere is checked against the hash its author published. Passwords are stored as hashes, and digital signatures, described below, are made on the hash of a message rather than on the whole message.

10.6 Symmetric Encryption

Symmetric encryption uses one secret key both to encrypt and to decrypt. The common methods are AES, a standard since 2001, and ChaCha20, which is faster in software on processors without special instructions for AES. Keys are usually 128 or 256 bits long. A 128-bit key has 2 to the 128th possible values, so many that trying them all is beyond any computer that could be built.

Encryption alone provides confidentiality, but not integrity: an attacker who cannot read a message may still be able to change it. Current protocols therefore use **authenticated encryption**, which adds a short tag, computed from the message and the key, that the receiver checks before trusting the message. Each message is also encrypted with a **nonce**, a number that must never be used twice with the same key; reusing one can reveal the messages.

Symmetric encryption is fast, but both ends must already share the key, and delivering it to them securely is difficult.

10.7 Public-Key Cryptography

Public-key cryptography solves that problem with a pair of keys. The **public key** can be given to anyone, and the matching **private key** is kept secret. The two are related by mathematics that is easy to compute in one direction and, as far as is known, impractical to reverse on present computers: from the private key the public key is calculated in an instant, but recovering the private key from the public one would take longer than the age of the universe. A padlock is a fair comparison: anyone can close an open padlock, but only the holder of its key can open it.

The pair makes two things possible:

- **Key exchange.** Two machines that have not communicated before can agree on a shared secret over an open network, even while others are listening. X25519 is a common method. The shared secret then becomes the key for fast symmetric encryption.
- **Digital signatures.** A message signed with a private key can be checked by anyone with the public key, and no one without the private key can forge the signature. Ed25519 is a common method. Signatures prove who wrote a message, or

who published a program, and that it has not been changed since.

Many systems, especially older ones, use RSA for both purposes, with much longer keys.

Public keys raise a question of trust: how to know that a public key belongs to the intended person or site. The web relies on **certificates**, which are public keys signed by organizations that browsers are configured to trust. SSH usually asks the user to confirm a server's key the first time and remembers it afterwards. Between people, keys can be compared in person, or their hashes read aloud.

All of this requires computation. On a small processor, the arithmetic of public-key cryptography can take seconds per connection. On an FPGA computer, the slow parts can be given their own circuits in the FPGA, as described in Systems on a Chip.

10.8 Random Numbers

Keys, salts and nonces must be unpredictable. Computers are deterministic, so they gather unpredictability from physical noise, such as the jitter of free-running oscillators, which a **true random number generator** (TRNG) measures. Many chips, including the FPGA designs in this book, contain one. Because physical noise is slow and imperfect, systems usually use it to seed a **cryptographically secure pseudo-random number generator**, an algorithm that produces an unlimited stream of numbers that cannot be predicted without knowing the seed.

A weak random number generator makes keys predictable, and a connection made with such keys shows no outward sign of it. For this reason the source of randomness is among the most important parts of a secure system.

10.9 The One-Time Pad

One method of encryption is unbreakable in principle, however much computing power an attacker has. In the **one-time pad**, each bit of the message is combined by XOR with one bit of a key that is truly random and as long as the message. The receiver, who holds a copy of the same key, applies XOR with it again, which restores the message:

```
message     01001000      the letter H
key         10110101      random, used once
            --------
encrypted   11111101      message XOR key
key         10110101      the same key
            --------
message     01001000      encrypted XOR key
```

Because every key is equally likely, every message of the same length is equally consistent with the encrypted bits, and the encrypted text reveals nothing but its length. This property is called **information-theoretic security**: it rests on the randomness of the key, not on the difficulty of a calculation.

The guarantee holds only under strict conditions. The key must be truly random, as long as all the messages it will protect, kept secret, and never used twice. If the same key is used for two messages, combining the two encrypted texts cancels the key and leaves the XOR of the two messages, from which both can often be recovered. The key must also be delivered to the receiver in advance, by some means other than the channel it protects. These costs have kept the one-time pad to uses in which the highest security justifies them.

Small, open hardware can make those conditions practical. **Einszeit**, an open-source security device under development at Machdyne and intended to become a product, provides one-time-pad encryption between two or more paired devices over any communication medium. A device generates key material from its hardware random number generator onto a memory card, which is handed to the other party in person. Before any key byte is used, the device records in F-RAM which part of the key has been consumed, so that no part can be used twice, even if the power fails at the moment of use. Its designs and software are public, and the project has received grant funding from the NGI0 Commons Fund.

10.10 Quantum Computers

A **quantum computer** computes with quantum states rather than with ordinary bits. A quantum computer large enough to break the public-key methods in common use today, including RSA, X25519 and Ed25519, had not been built when this book was written; if one is built, those methods will no longer be secure. Symmetric encryption and hash functions are much less affected: a 256-bit key remains secure.

Data that is recorded today and kept could be decrypted later, once such computers exist. For this reason new **post-quantum** public-key methods, based on mathematical problems that quantum computers are not known to solve, have been developed and standardized: ML-KEM for key exchange and ML-DSA for signatures, published as United States federal standards in 2024. Because these methods are newer and less studied, careful systems use them in a **hybrid** scheme together with a classical method, so that the result is secure as long as either one holds. The message protocol of Zeitlos, described in The Machine, encrypts its connections in this way, although its signatures still use a classical method alone.

The one-time pad is unaffected by quantum computers, because its security does not depend on any calculation being difficult.

10.11 Deleting Data

Deleting a file usually removes only its name and marks its space as free. The data remains on the medium until it is overwritten, and can often be recovered.

On flash memory, overwriting is not reliable either. Because of wear levelling, described in Memory and Storage, a new version of a block is written to a different place, and the old one may remain, inaccessible to the computer but readable by someone who reads the chip directly. The dependable method is to store sensitive data encrypted from the start, so that destroying the key destroys the data, wherever its copies lie. F-RAM, which is rewritten in place, can be overwritten directly. A medium that held secrets and is no longer needed can be physically destroyed.

10.12 Secure Software

Every byte that arrives from a network, a file or a device may have been made to cause harm. A program that assumes its input has a certain length or form can be made to misbehave, as described under buffer overflows in C, and in the worst case to run instructions supplied by an attacker. Input is therefore checked before it is used, and programs that handle input from outside are kept small and simple.

Several practices limit the damage that a faulty or hostile program can do:

- **Least privilege.** Each program has only the access it needs. A program that cannot write to the system's files cannot damage them.
- **Few services.** Each program that accepts connections from a network is a possible way in. A machine that offers few services, and only to the network that needs them, has less to defend.
- **Protection boundaries.** Memory protection, described in Operating Systems, keeps one program from damaging another. Whether it is also a security boundary, which a hostile program cannot cross, depends on its design; a system may use it only to contain errors, as Zeitlos does.
- **Updates.** Weaknesses are found in all software, and fixes are published as updates. A small system has less code in which weaknesses can hide, and less to update.

Every program, library and tool on a machine is trusted to do what it claims, including the compiler that built the others. Ken Thompson showed in 1984 that a compiler could be altered to insert a weakness into the programs it compiles, including new copies of itself, so that the weakness appears in no source code. Software from a known source, checked against a published hash or signature, reduces what must be taken on trust; source code that can be read, and built again with independent tools to the same result, reduces it further.

10.13 Copies

Offline copies of data, as described in Computing That Lasts, are the defence against software that deletes or encrypts files and demands payment to restore them, as well as against every other kind of loss.

10.14 Simplicity and Security

A simple system is easier to secure, because its owner can know what it runs, what it accepts from outside and what it stores. Each of the measures in this chapter is easier to apply to a small, well-understood machine than to a large one, and its effect is easier to check.

This completes the description of the machine: a processor, memory, devices, an operating system, a network and the means to protect them. The next part of the book is about programs.

Part IV

Programming

Chapter 11

Thinking in Programs

A program is a precise description of how to perform a task, written for a machine that does exactly what it is instructed to do and nothing else. Learning to program consists mostly of learning to divide a task into steps small and clear enough for a machine to follow. This chapter covers the ideas common to every programming language. The three chapters after it show those ideas in three different languages, each of which runs on Machdyne's systems.

11.1 Why Program

A computer can be used without being programmed. Programming, however, is what makes a general-purpose computer general. A person who can write short programs can make the machine perform tasks that nobody planned for: renaming a thousand files, logging a temperature every hour, converting a document or controlling a pump.

Programming also teaches how computers work, which makes faults easier to find and correct. Code generated automatically, including by programs that generate code, must still be read, understood and corrected by someone. In addition, a person who can program can keep their tools working without depending on an outside service.

11.2 Algorithms

An **algorithm** is a method for solving a problem in a finite number of exact steps. Algorithms are older than computers. One of the oldest known, recorded by Euclid around 300 BC, finds the greatest common divisor of two numbers, the largest number that divides both exactly:

```
to find the greatest common divisor of A and B:
    while B is not zero:
        let R be the remainder of A divided by B
        replace A with B, and B with R
    the answer is A
```

For 1071 and 462, the remainders are 147, then 21, then 0, so the answer is 21. The appendix Three Languages Side by Side shows the same algorithm in BASIC, Scheme and C.

A description of steps in plain words, such as this one, is called **pseudocode**. It is a useful way to plan a program before choosing a language.

11.3 State

A program works by changing **state**: the values held in memory. A **variable** is a named place that holds a value. Assigning to a variable replaces its previous value:

```
count = 0
count = count + 1
```

The second line is not an equation, which would be false. It is an instruction to compute `count + 1` and store the result back in `count`. Every step of a running program can be understood as reading some state and writing some new state. When a program behaves incorrectly, the question is almost always which part of the state differs from what was expected, and when it changed.

Values have **types**: whole numbers, fractional numbers, characters, text, true-or-false values and others. A type determines what a value means and which operations apply to it. As Information showed, the same bits can mean different things. Types are the means by which a language records which meaning applies.

11.4 Control Flow

Programs are built from three kinds of structure:

- **Sequence**: one step, then the next.
- **Choice**: if a condition is true, one step; otherwise another.
- **Repetition**: a step repeated a fixed number of times, or until a condition becomes true.

Every algorithm, however large, can be written with these three. Euclid's method above uses all of them: a sequence of steps, repeated while a condition holds, and the test of that condition is a choice.

11.5 Functions

A **function**, also called a procedure or subroutine, is a named part of a program that takes some inputs, called **arguments**, performs a task, and usually returns a result. Once written, a function can be used without regard to how it works. It is the main means by which a large problem is divided into small ones.

A good function does one clear thing and has a name that describes it. A program for a weather station might have functions to read the sensor, convert the reading to degrees, decide whether it is freezing and write a line to the log. Each is short and can be checked on its own, and the main program reads almost as a description of the task.

A function can call itself. This is **recursion**, and it suits problems that contain smaller instances of themselves. The factorial of a number n, written n!, is n times the factorial of n - 1, and the factorial of 0 is 1. A recursive function states exactly that. Recursion needs a **base case**, here 0, at which it stops. Without one, the function calls itself until the stack is exhausted.

11.6 Data Structures

The arrangement of data matters as much as the steps that operate on it. A few structures cover most needs:

- An **array** is a row of values of the same type, numbered from 0 in C and many other languages, each reached directly by its number. Arrays are fast and simple, but fixed in size.
- A **string** is a sequence of characters, usually stored as an array.
- A **record**, or structure, groups a few named values that belong together, such as a name, a date and an amount.
- A **linked list** is a chain of records, each holding a value and the address of the next. It can grow and shrink easily, but reaching the hundredth item requires passing through the first ninety-nine.
- A **tree** is similar to a list, except that each record can point to several others. A folder of files, in which each folder can hold further files and folders, is a tree.
- A **hash table**, also called a dictionary or map, stores values under keys, such as words and their definitions, and finds a key quickly without searching every entry.

11.7 Efficiency

On a slow computer, the efficiency of an algorithm has practical consequences. The useful measure is how the amount of work grows as the amount of data grows.

Finding a name in an unsorted list of n names may require checking all n of them. If the list doubles in length, the work doubles. In a sorted list, a program can instead examine the middle entry, determine which half contains the name, and repeat. Each step halves the remaining entries, so a million names require only about twenty comparisons. This method is **binary search**.

Some methods grow much faster. Comparing every item with every other item takes n times n steps, so a thousand items require a million comparisons. A loop inside a loop over a large amount of data is often a sign that a better method exists.

11.8 Compilers and Interpreters

A processor runs only machine code, so a program in any other language must be translated. There are two approaches.

A **compiler** translates the whole program into machine code before it runs. The result runs at full speed, and many mistakes are reported before the program runs at all. The cost is a separate step between writing and running. C is normally compiled.

An **interpreter** reads the program and carries out each part directly, as it proceeds. It is slower, often by a factor of ten or more. In exchange, a programmer can type a line and see its effect immediately, change the program while it runs, and work on a machine that has no compiler. BASIC and Scheme are usually interpreted.

Many systems lie between the two. Some interpreters first translate the program into a compact internal form, often called **bytecode** or tokens, and interpret that, which is faster than working from the text. Machdyne BASIC checks each line and converts it into tokens when it is entered.

11.9 Mistakes

Almost every program contains mistakes, called **bugs**, when first written. They are of three broad kinds:

- **Syntax errors**: the program is not valid in its language, in the way that a sentence with its words in the wrong order is not valid. The compiler or interpreter reports these.
- **Run-time errors**: the program is valid but attempts something impossible when it runs, such as dividing by zero or reading past the end of an array.
- **Logic errors**: the program runs without complaint and gives the wrong answer. These are the most difficult to find, because nothing reports them.

Finding bugs is called **debugging**. The method matters more than the tools:

1. **Reproduce** the problem reliably. A bug that cannot be made to occur on demand cannot be confirmed as fixed.
2. **Observe** instead of guessing. Print the values of variables at key points, or step through the program, and compare

the results with what was expected.
3. **Narrow the search.** Find the earliest point at which the state is wrong. Halving the search space each time, as in binary search, finds the cause quickly.
4. **Fix the cause**, not the symptom. Then check that the original problem is gone and that nothing else has broken.

11.10 Testing

A **test** runs part of a program with known inputs and checks that it produces the expected outputs. A test of Euclid's method, for example, checks that the greatest common divisor of 1071 and 462 is 21, and that of 7 and 0 is 7.

Good tests are chosen where mistakes are likely: the smallest and largest values, zero, empty input, and the values on either side of every limit. A function that works for 10 numbers may fail for none, for one, or for the largest number its type can hold.

Tests are kept and run again after every change. The return of a bug that had been fixed is called a **regression**, and a test that once found a bug guards against it. A collection of such tests makes changes to an old program much safer. Every program listing in this book is kept as a test in this way, and the tests are run each time the book is rebuilt.

Hardware is tested in the same spirit. A memory test writes known patterns to every location and reads them back; a self-test at power-on checks that each device answers. Such tests find faults that would otherwise appear as unexplained errors in the programs that use the hardware.

11.11 Writing for People

A program is read far more often than it is written, frequently by its own author months later. Clear names, short functions, consistent layout and brief comments that explain why something is done, rather than what, make a program much easier to maintain.

11.12 Keeping Track of Changes

A program, a hardware design or a document that is changed over years needs a record of its changes. Without one, it is impossible to say what changed between a version that worked and one that does not, or to return to the earlier version.

The simplest record is a copy of the whole project, dated, before each significant change, together with a note of what was changed and why. A **diff** program compares two versions of a text file and lists only the lines that differ, which makes the changes between two copies easy to see.

A **version control system** keeps this record automatically. Each time the author records a change, the system stores the new version together with a description, the date and the author, and keeps every earlier version. Any version can be restored, any two can be compared, and the history shows when and why each line was changed. Git is a widely used system of this kind; it keeps the whole history in the project's own folder, so a copy of the folder is a complete copy of the history, and no server is needed.

The same practice applies to a machine's configuration. A record of which version of each design and program is in use, and what was changed, makes it possible to rebuild the machine exactly, and to undo a change that caused a fault.

11.13 Three Languages

The next three chapters use three languages. Each represents a different way of thinking about programs, and each has been in use for decades. All three run on Zeitlos, and BASIC also runs on Machdyne's modules.

Language	Born	Style	Best for
BASIC	1964	numbered lines	first programs, modules
Scheme	1975	expressions and functions	ideas, structure, scripting
C	1972	near the machine	speed, hardware, systems

The chapters can be read in any order, starting with the language that best suits the reader's purpose.

Chapter 12

BASIC

BASIC was created in 1964 at Dartmouth College by John Kemeny and Thomas Kurtz, so that students in any subject, not only scientists, could use a computer. Its name stands for Beginner's All-purpose Symbolic Instruction Code. In the late 1970s and 1980s, almost every home computer started with a BASIC prompt, and many people learned to program by typing in listings printed in magazines.

BASIC remains useful. An interpreter fits in a few kilobytes, so it runs on the smallest chips. A program is a short list of numbered lines that is easy to read, and the result of each change is seen at once. This makes it a good first language and a practical one for small machines.

12.1 Machdyne BASIC

There are hundreds of dialects of BASIC. This chapter uses **Machdyne BASIC 1**, which runs on Machdyne's programmable modules and tools, inside Zeitlos, and on ordinary computers. It is deliberately small, because it must fit, together with its own filesystem, in a microcontroller with 16 KB of flash and 2 KB of RAM.

Its main features and limits:

- Programs are numbered lines, stored and run in order of line number.
- There are 26 variables, `A` to `Z`, each holding one number.
- Every number is a 16-bit signed integer, from -32768 to 32767. There are no fractions and no string variables; text appears only in quotes.
- Programs and data are stored as plain text files.
- On hardware with pins, BASIC can read and drive them directly.

Larger BASICs add string variables (written like `N$`), arrays, fractional numbers and many more statements, but the ideas are the same. Machdyne BASIC 1 is also **frozen**: its definition will not change. Later versions may add to it, but must not change the meaning of any BASIC 1 program. A program written for it today is intended to keep its meaning for decades, on whatever machine implements it then.

12.2 Lines and Commands

A line that starts with a number is stored as part of the program. Typing a line with a number that already exists replaces that line, and typing the number alone deletes it. Line numbers are traditionally written in steps of ten, which leaves room to insert lines later.

A line without a number is a **command**, and is carried out at once:

Command	Effect
RUN	run the program
LIST	show the program

Command	Effect
NEW	erase the program
SAVE name	save the program to a file
LOAD name	load a saved program
DIR	list the files
TYPE name	show the contents of a file
DEL name	delete a file

Many BASICs also have an **immediate mode**, in which a statement typed without a line number, such as `PRINT 2 + 2`, runs at once. Machdyne BASIC 1 does not; statements must be placed in numbered lines.

12.3 A First Program

```
10 FOR I = 1 TO 5
20 PRINT I; " SQUARED IS "; I * I
30 NEXT I
```

After these lines are typed, `RUN` prints:

```
1 SQUARED IS 1
2 SQUARED IS 4
3 SQUARED IS 9
4 SQUARED IS 16
5 SQUARED IS 25
```

`FOR` and `NEXT` form a loop: `I` takes the values 1 to 5, and the line between them runs once for each value. `PRINT` shows a mixture of numbers and quoted text, separated by semicolons. A comma in place of a semicolon moves to the next print zone, 14 characters wide, which is useful for tables.

12.4 Numbers and Arithmetic

BASIC uses the operators `+`, `-`, `*` and `/`, and `MOD` for the remainder after division. Multiplication and division are done before addition and subtraction, and parentheses group as in ordinary arithmetic.

Because every number is an integer, division discards the fraction: `7 / 2` is 3. Because numbers are 16 bits, results outside -32768 to 32767 wrap around, as described in Information. A program that

works with large values must scale them, for example by keeping temperatures in tenths of a degree or money in cents, and must check that intermediate results stay in range.

Comparisons, such as `A > B`, give -1 for true and 0 for false. Any value other than 0 counts as true, so a variable that holds 1 or 0 can be used directly as a condition.

12.5 Decisions and Loops

`IF` runs the rest of its line only if a condition is true, and `ELSE` gives an alternative. `GOTO` continues the program at another line. With these and `FOR`, any control flow can be written.

This program prints the prime numbers up to 50, those divisible only by 1 and themselves:

```
10 REM PRINT THE PRIME NUMBERS UP TO 50
20 FOR N = 2 TO 50
30 P = 1: D = 2
40 IF D * D > N THEN GOTO 70
50 IF N MOD D = 0 THEN P = 0
60 D = D + 1: GOTO 40
70 IF P THEN PRINT N; " ";
80 NEXT N
90 PRINT
```

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

For each number `N`, the program assumes that it is prime (`P = 1`) and then tries dividing it by each `D` from 2 upward. Divisors come in pairs, and the smaller of a pair cannot exceed the square root of `N`. The search can therefore stop once `D * D` exceeds `N`, which removes most of the work. `REM` starts a remark, a comment for the reader, and a colon separates several statements on one line.

In Machdyne BASIC, the body of a `FOR` loop always runs at least once, even if the loop's end value has already been passed. For this reason the inner loop is built from `IF` and `GOTO` rather than `FOR D = 2 TO N - 1`, which, when N is 2, would still run once with D = 2 and wrongly mark 2 as not prime.

12.6 Subroutines

Lines that perform one task can be made into a **subroutine**. `GOSUB` jumps to it, and `RETURN` continues at the statement after the `GOSUB`:

```
10 FOR N = 1 TO 3
20 GOSUB 100
30 NEXT N
40 END
100 REM PRINT A LINE OF N STARS
110 FOR S = 1 TO N
120 PRINT "*";
130 NEXT S
140 PRINT
150 RETURN
```

```
*
**
***
```

The `END` on line 40 is necessary. Without it, the program would continue into the subroutine after the loop finishes and reach `RETURN` with nowhere to return to.

Because all variables are shared, a subroutine communicates through them: here the main loop sets `N`, and the subroutine reads it. With only 26 variables, it helps to record the purpose of each at the top of the program.

A well-organized BASIC program keeps its main loop short and near the top, and places each subroutine at its own round line number, such as 100, 200 and 300, with a `REM` line describing what it does. `GOTO` can make a program hard to follow, so it is best kept for simple loops and skips, with subroutines used for everything else.

12.7 Input

`INPUT` waits for the user to type a number:

```
10 INPUT "HOW MANY"; N
```

The quoted prompt is shown first, followed by a question mark. Everything a BASIC program prints and reads, apart from data files, passes through the **console**, which may be a screen and keyboard, a terminal on another computer, or a link over the module bus.

12.8 Files

A program can write and read plain text files. Only one file can be open at a time, as number 1:

```
10 PINS -, -, AIN, -
20 OPEN "LOG.TXT" FOR APPEND AS #1
30 PRINT #1, ADC(3)
40 CLOSE #1
50 SLEEP 60
60 GOTO 20
```

About once a minute (`SLEEP` waits a number of seconds), this program reads an analog input and adds the reading as a new line at the end of `LOG.TXT`. Opening a file `FOR OUTPUT` replaces it instead, and opening it `FOR INPUT` allows it to be read with `INPUT #1`. The command `TYPE LOG.TXT` shows the result. Because the file is plain text, any computer can read it later.

On Machdyne's modules, files are stored in F-RAM or EEPROM. The filesystem follows the rules for surviving a power cut described in Memory and Storage. As a result, a data logger can be switched off at any time without damaging its log; at worst, the line being written is lost.

12.9 Pins

On hardware that has pins, a BASIC program can read and drive them. First, the program declares the purpose of each pin with `PINS`, one mode per pin:

Mode	Meaning
-	not used
IN	digital input
PP	push-pull output
OD	open-drain output
AIN	analog input
I2C	the I2C bus (two pins together)
NET	left to the system, for the bus that connects it

`OUT` then sets an output pin high or low, `IN` reads a digital input as 0 or 1, `ADC` reads an analog input as a number from 0 to 1023, and `LED` turns the board's LED on or off. Because pins are declared

first, a mistake such as driving a pin declared as an input produces an error message instead of unexpected behaviour.

This program keeps a heater on while a temperature sensor reads low:

```
10 REM KEEP A HEATER ON WHILE IT IS COLD
20 PINS -, -, AIN, PP
30 T = ADC(3)
40 IF T < 400 THEN OUT 4, 1 ELSE OUT 4, 0
50 REG 0, T / 4
60 WAIT 1000
70 GOTO 30
```

`WAIT` pauses for a number of milliseconds, so the loop runs about once a second. The program follows the pattern of nearly every control program: **read, decide, act, wait, repeat**. Pin 3 reads the sensor, and pin 4 switches the heater through a transistor or relay, as described in Talking to the World.

A practical controller would improve on this in one respect. When the temperature stays close to the threshold, the heater switches on and off every second, which wears out relays. The usual remedy is **hysteresis**: the heater switches on below one value and off only above a slightly higher one, so that small fluctuations have no effect.

I2C devices are reached with the statement `I2C`, which writes to a device, and the function `I2CR`, which reads from one. This program reads the temperature sensor connected in Talking to the World, at address 72, once a second:

```
10 REM READ AN I2C TEMPERATURE SENSOR
20 PINS NET, NET, I2C, I2C
30 T = I2CR(72, 0)
40 IF T = -1 THEN PRINT "NO SENSOR": END
50 IF T > 127 THEN T = T - 256
60 PRINT T; " C"
70 WAIT 1000
80 GOTO 30
```

`I2CR` gives -1 when no device answers, which line 40 reports instead of printing a wrong temperature. Line 50 converts the byte from two's complement into a negative number.

12.10 Talking to a Host

Line 50 of the heater program stores a value in **program register** 0. A program has sixteen registers, `REG 0` to `REG 15`, each holding one byte. They are shared with the system the program runs in. On a module, a host computer on the bus can read and write them while the program runs. In this example, the host can read the latest reading, divided by four to fit in a byte, at any time without stopping the program, and could pass in a new threshold, scaled the same way, through another register.

12.11 Starting by Itself

A program saved as `BOOT` (stored as `BOOT.BAS`) runs automatically when the machine starts. This turns a module into a dedicated controller: once programmed and installed, it performs its task every time it is powered, with or without a computer attached. The boot window described in Talking to the World ensures that a faulty `BOOT` program cannot lock the host out.

12.12 One Language, Many Machines

The definition of Machdyne BASIC states what a program means; each machine determines which hardware it provides. A program that only computes and prints runs the same everywhere. A program that uses pins runs wherever those pins exist, and on a machine without them it stops with `NOT SUPPORTED` rather than behaving unexpectedly.

Machines may also add statements of their own. A BASIC with a screen may add drawing statements such as `PLOT`, `LINE` and `CIRCLE`, as the home computers did, and programs that use them run only where those statements exist.

As a result, a program can be written and tested on a full computer, within the limits of the module it is intended for, and then sent to a small module, where it behaves the same way.

Chapter 13

Scheme

Lisp was invented by John McCarthy in 1958, and is one of the two oldest programming languages still in use. Scheme is a small dialect of Lisp, created at MIT in 1975 by Guy Steele and Gerald Sussman. Its complete definition fits in about fifty pages, and it has been used for decades to teach the foundations of computer science.

Scheme programs are built from a few ideas combined freely: expressions, functions and lists. Learning it teaches a way of thinking about programs, including how a programming language itself works.

13.1 Machdyne Scheme

The Scheme in this chapter is Machdyne Scheme, a small interpreter that runs inside Zeitlos and on ordinary computers. It implements most of R4RS, the Revised(4) Report on the Algorithmic Language Scheme of 1991, the standard version of the language at that time. It omits a few advanced features that are difficult to fit on a small machine, and the standard procedures for files, which each system provides in its own way. All its numbers are floating point. Everything in this chapter is standard Scheme and works in any implementation, although an implementation with exact fractions prints some results, such as that of `(/ 7 2)`, differently.

13.2 Expressions

Every part of a Scheme program is an **expression**, and every compound expression is written in the same way: in parentheses, with the operation first and its arguments after it. In the examples below, each expression is followed by its value:

```
(+ 1 2)              3
(* 6 7)              42
(+ 1 (* 2 3))        7
(- 10 (/ 8 2))       6
```

This **prefix notation** has two advantages. There are no precedence rules, because the parentheses state exactly what is grouped with what. In addition, operations can take any number of arguments: `(+ 1 2 3 4)` is 10.

Scheme is normally used through a REPL, the read-eval-print loop described in Operating Systems. In this chapter, `; =>` marks the value an expression produces. Everything from a semicolon to the end of the line is a comment.

13.3 Definitions

`define` gives a name to a value or to a function:

```
(define (square x) (* x x))
(square 12)                  ; => 144
```

This reads as: to compute the square of `x`, multiply `x` by `x`. Once defined, `square` is used in the same way as the built-in operations.

13.4 Choices

`if` takes a test, an expression for when the test is true, and one for when it is false. `cond` checks a series of tests in order and uses the first that is true:

```
(define (size n)
  (cond ((< n 10) 'small)
        ((< n 100) 'medium)
        (else 'large)))
(size 42)                    ; => medium
```

The values `'small`, `'medium` and `'large` are **symbols**: names used as data, an alternative to strings or numeric codes. The quote mark means “this name itself, not the value it stands for”.

The true and false values are written `#t` and `#f`. In Scheme only `#f` counts as false; zero, the empty list and the empty string all count as true.

13.5 Numbers

Standard Scheme distinguishes exact numbers, such as integers and fractions, from inexact ones. In Machdyne Scheme every number is a floating-point double, as described in Information. Whole numbers are exact up to 2 to the 53rd, which is enough for counting and for most arithmetic on integers, but division can produce results that are not whole numbers, and most of these are approximate:

```
(/ 7 2)                         ; => 3.5
(quotient 7 2)                  ; => 3
(= (+ 0.1 0.2) 0.3)             ; => #f

(define (close? a b)
  (< (abs (- a b)) 0.000001))
(close? (+ 0.1 0.2) 0.3)        ; => #t
```

`quotient` and `remainder` divide whole numbers as BASIC and C do. Two results that should be equal are compared by testing whether they are close, as `close?` does, rather than with `=`.

13.6 Recursion

Scheme has no loop statement of the kind found in BASIC and C. It provides a `do` form, but repetition is usually written as **recursion**, in which a function calls itself:

```
(define (factorial n)
  (if (= n 0)
      1
      (* n (factorial (- n 1)))))
(factorial 10)                  ; => 3628800
```

The definition states what a factorial is: 1 for zero, and otherwise n times the factorial of n - 1. Its evaluation can be followed by replacing each call with its definition:

```
(factorial 3)
(* 3 (factorial 2))
(* 3 (* 2 (factorial 1)))
(* 3 (* 2 (* 1 (factorial 0))))
(* 3 (* 2 (* 1 1)))
6
```

Each pending multiplication must be remembered until the innermost call returns, so this function uses stack space in proportion to `n`.

13.7 Tail Calls

When the recursive call is the last action of a function, nothing needs to be remembered, and the call can reuse the current stack frame. Scheme guarantees this: such a **tail call** runs in constant space, and is therefore as efficient as a loop. A loop is usually written with a **named let**, which defines a local function and calls it immediately:

```
(define (sum-to n)
  (let loop ((i 1) (total 0))
    (if (> i n)
        total
        (loop (+ i 1) (+ total i)))))
(sum-to 100)                  ; => 5050
```

`loop` starts with `i` as 1 and `total` as 0. Each call to `loop` is a tail call that carries the new values forward, so the computation can repeat a million times without growing the stack. The variables that would change in a loop in another language are here the arguments of the next call.

13.8 Lists

The **list** is the central data structure of Lisp, whose name comes from "list processing". A list is written in parentheses and quoted, so that Scheme treats it as data rather than as an expression to evaluate:

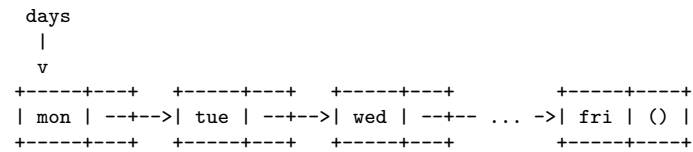
```
(define days '(mon tue wed thu fri))
(car days)                    ; => mon
(cdr days)                    ; => (tue wed thu fri)
(cons 'sun days)              ; => (sun mon tue wed thu fri)
```

Three operations do most of the work:

- `car` gives the first item of a list.
- `cdr` gives the rest of the list, everything after the first item.
- `cons` builds a new list with one item added at the front.

The names come from two parts of a machine word on the IBM 704, the computer on which Lisp was first implemented.

Internally, a list is a chain of **pairs**. Each pair holds one item and a reference to the next pair, and the chain ends with the empty list, written `'()` in a program. This is the linked list described in Thinking in Programs:



`cons` does not copy the list it adds to; it makes one new pair that points to the old list. As a result, lists can share their tails, and building a list one item at a time is inexpensive.

Most functions on lists follow the same pattern: they handle the empty list, and otherwise do something with the `car` and recurse on the `cdr`:

```
(define (sum lst)
  (if (null? lst)
      0
      (+ (car lst) (sum (cdr lst)))))
(sum '(3 1 4 1 5))            ; => 14
```

13.9 Association Lists

A list of pairs can serve as a lookup table, called an **association list**. `cons` makes a pair from any two values. When the second value is a list, the result is a longer list; when it is not, the pair is written with a dot between its two parts, so `(cons 'ben 2501)` is `(ben . 2501)`. Such **dotted pairs** suit a table of names and values:

```
(define phones '((anna . 4711) (ben . 2501) (cleo . 1234)))
(cdr (assq 'ben phones))      ; => 2501
```

`assq` finds the first pair whose first part is the given symbol, or returns `#f` if there is none. For a few dozen entries, an association list is sufficient.

13.10 Functions as Values

In Scheme, a function is a value like any other. It can be passed to another function, returned from one, or stored in a list. A function that takes or returns functions is called **higher-order**.

The standard function `map` applies a function to every item of a list:

```
(map square '(1 2 3 4))       ; => (1 4 9 16)
```

Higher-order functions are defined like any other. This one keeps the items of a list for which a test is true:

```
(define (keep ok? lst)
  (cond ((null? lst) '())
        ((ok? (car lst)) (cons (car lst) (keep ok? (cdr lst))))
        (else (keep ok? (cdr lst)))))

(keep even? '(1 2 3 4 5 6))           ; => (2 4 6)
(keep (lambda (x) (> x 10)) '(5 50 8 80))   ; => (50 80)
```

`lambda` makes a function without giving it a name, which is convenient when a small function is needed only once. The definition `(define (square x) (* x x))` is shorthand for `(define square (lambda (x) (* x x)))`.

With `map`, `keep` and similar functions, many tasks that need a loop and some bookkeeping in other languages become a single expression.

13.11 Closures

A function made by `lambda` retains access to the variables that were visible where it was made, even after the code that created them has finished. Such a function is a **closure**:

```
(define (make-counter)
  (let ((count 0))
    (lambda ()
      (set! count (+ count 1))
      count)))

(define tally (make-counter))
(tally)                       ; => 1
(tally)                       ; => 2
```

Each call to `make-counter` creates a new variable `count` and returns a function that can reach it; no other code can. `set!`

changes a variable's value; by convention, the exclamation mark indicates that something is changed in place. Closures provide private, persistent state without any additional language features, and many larger programming techniques are built on them.

13.12 Memory

Scheme programs create pairs, strings and closures freely and never release them. The interpreter releases them automatically: from time to time, a **garbage collector** finds every object the program can still reach, starting from its variables, and recycles the rest. Scheme programmers, like those of most modern languages, never free memory by hand, which prevents the errors of freeing memory twice or too early. The cost is some additional memory and an occasional pause while the collector runs.

13.13 Code Is Data

A Scheme program is written as lists, and lists are Scheme's main data structure. As a result, a program can readily build, examine and run other programs. This is an evaluator for a small language of sums and products:

```
(define (calc e)
  (cond ((number? e) e)
        ((eq? (car e) '+) (+ (calc (cadr e)) (calc (caddr e))))
        ((eq? (car e) '*) (* (calc (cadr e)) (calc (caddr e))))
        (else 'unknown)))

(calc '(+ 1 (* 2 3)))          ; => 7
```

`cadr` and `caddr` are shorthand for the second and third items of a list. The evaluator examines the expression. A number is its own value. A list starting with `+` or `*` is evaluated by evaluating its two arguments and combining the results.

Interpreters work in this way. If `calc` is extended with variables, `if`, `lambda` and function calls, it becomes an interpreter for Scheme itself, and such an interpreter fits in a page or two of code. This shows that a programming language is itself a program, and that a small one can be complete.

13.14 Scheme on a Machine

A Scheme system on a particular computer adds procedures for what the standard leaves out: files, windows, the network and the hardware. With them, a few lines of Scheme typed at the REPL can automate the machine or form a small windowed tool, without the need for a compiler.

Scheme is well suited to problems concerned with structure: lists of things, rules, transformations, small languages, and problems that benefit from interactive exploration. For obtaining the greatest speed from a slow processor, C is the better choice.

Chapter 14

C

C was created by Dennis Ritchie at Bell Labs around 1972, to write the Unix operating system. It was designed to be close to the machine, so that programs could be nearly as fast as assembly language while remaining portable from one kind of computer to another. It remains a main language of operating system kernels, device drivers, microcontrollers and the interpreters of other languages. The Zeitlos operating system, the BASIC interpreter and the Scheme interpreter in this book are all written in C.

C shows directly what a computer does. Its variables are locations in memory, its pointers are addresses, and its operations correspond closely to single machine instructions. It also checks little: it does what the program says, and allows a faulty program to damage itself.

14.1 A Complete Program

```
#include <stdio.h>

#define BIT(n) (1u << (n))

int main(void)
{
    unsigned flags = 0;

    flags |= BIT(3);            /* set bit 3 */
    flags |= BIT(0);            /* set bit 0 */
    flags &= ~BIT(3);           /* clear bit 3 */
```

```
    flags ^= BIT(5);            /* flip bit 5 */

    if (flags & BIT(0))         /* test bit 0 */
        printf("bit 0 is set\n");

    printf("flags = 0x%02X\n", flags);
    return 0;
}
```

```
bit 0 is set
flags = 0x21
```

The parts of this program appear in nearly every C program:

- `#include <stdio.h>` makes the standard input and output functions, such as `printf`, available.
- `#define` creates a **macro**, a piece of text that, before compilation, is substituted wherever its name appears. Lines starting with `#` are handled by this **preprocessor** step.
- `main` is the function where every program starts. Its result, here 0, tells the system whether the program succeeded.
- Statements end with a semicolon, and braces group them into blocks.
- Text between `/*` and `*/` is a comment.
- `printf` prints formatted text. In its first argument, `%02X` means "the next value, in upper-case hexadecimal, padded with zeros to at least two digits", and `\n` means a new line.

The program demonstrates C's bitwise operators, which act on every bit of a number at once: `|` (OR), `&` (AND), `^` (XOR), `~` (NOT) and `<<` (shift left). `BIT(5)` is 1 shifted left five places, a number with only bit 5 set. This is called a **mask**. OR-ing with a mask sets that bit, AND-ing with its inverse clears it, XOR-ing with it flips it, and AND-ing with it tests it. These four operations control the individual bits of a hardware register.

14.2 Compiling

A C program is written in one or more text files and turned into machine code by a **compiler**. Each source file is compiled separately, and a **linker** then joins the results, together with any **libraries** of ready-made functions, into one executable program. **Header files**, ending in `.h`, declare the functions and types that a source file or library provides, so that other files can use them.

The **standard library** supplied with C provides input and output (`stdio.h`), strings (`string.h`), memory allocation and conversions (`stdlib.h`), mathematics (`math.h`) and more. Programs that use only the standard library can be compiled for almost any computer.

Most C compilers are large programs that run on large computers, and the programs for a small computer are usually compiled on a larger one, a practice called **cross-compiling**. A compiler for a useful subset of C can, however, be small enough to run on a small machine. Zeitlos includes such an experimental compiler, which can build simple programs on the FPGA computer itself, as described in The Machine.

14.3 Types

Every variable in C has a type, declared when the variable is created, which determines how much memory it occupies and what its bits mean:

Type	Typical size	Holds
char	8 bits	one character, or a small number
short	16 bits	a whole number
int	32 bits on most systems	a whole number
long	32 or 64 bits	a whole number
float, double	32, 64 bits	a fractional number

Any of the whole-number types can be `unsigned`, which gives up negative values in exchange for twice the positive range.

The sizes are not fixed by the language. An `int` is 16 bits on some small processors and 32 bits on most others, which matters wher-

ever a value might overflow. Where the exact size matters, as it does for hardware registers and file formats, the header `stdint.h` provides types with the size in their names: `uint8_t`, `int16_t`, `uint32_t` and so on.

14.4 Pointers

A **pointer** is a variable that holds a memory address. Pointers are the source of both C's power and its danger.

```
    int x = 42;
    int *p = &x;            /* p holds the address of x */
    *p = 7;                 /* store 7 at that address */
    printf("x is now %d\n", x);
```

```
x is now 7
```

`&x` means "the address of `x`". The declaration `int *p` makes `p` a pointer to an `int`, and `*p` means "the `int` at the address in `p`". Storing through `p` changes `x`, because `p` points to `x`.

Pointers allow a function to change its caller's variables, allow a program to build linked structures, and allow it to reach any address, including a hardware register.

14.5 Arrays and Strings

An **array** is a sequence of values of one type, numbered from 0. In most expressions, the name of an array stands for the address of its first element, so arrays and pointers are closely related:

```
    int a[4] = { 10, 20, 30, 40 };
    int *q = a;             /* the array's first address */
    printf("%d %d\n", *(q + 2), q[3]);
```

```
30 40
```

Adding 2 to a pointer to `int` moves it two `int`s along, not two bytes, and `q[3]` is another way of writing `*(q + 3)`.

C has no separate string type. A **string** is an array of characters ending with a byte of zero, written `'\0'`. A function that needs the length of a string must step along it to find the end:

```
/* Count the characters in a string by walking a pointer along it
   until it reaches the zero byte that ends every C string. */
int length(const char *s)
{
```

```
    const char *p = s;
    while (*p != '\0')
        p++;
    return p - s;
}
```

`length("timeless")` is 8. `const` declares that the function will not change the characters, and subtracting two pointers gives the distance between them.

C does not check array bounds. Reading or writing past the end of an array, a common error when handling strings, uses whatever memory lies beyond it, without any warning. This kind of error, a **buffer overflow**, has been one of the most common causes of security problems. For this reason, every array access in a C program should be checked for whether it can go past the end.

14.6 Structures and Memory

A **structure** groups named values into one record. This one is a note in a linked list:

```
struct note {
    char text[32];
    struct note *next;
};
```

A C program has three kinds of memory:

- **Local variables** are stored on the stack, described in The Computer. They are created when their function starts and disappear when it returns.
- **Global variables**, declared outside any function, exist for the whole run of the program.
- The **heap** is memory requested while the program runs, with `malloc`, and returned with `free`. It lasts until it is freed.

This function adds a new note to the front of a list, using memory from the heap:

```
/* Add a note at the front of the list; return the new front. */
struct note *add(struct note *list, const char *text)
{
    struct note *n = malloc(sizeof *n);
    if (n == NULL)
        return list;
    strncpy(n->text, text, sizeof n->text - 1);
    n->text[sizeof n->text - 1] = '\0';
```

```
    n->next = list;
    return n;
}
```

`sizeof` gives the size in bytes of a type or a variable: `sizeof *n` is the size of the structure that `n` points to, and `sizeof n->text` is 32. `malloc` returns a pointer to enough unused memory for one note, or `NULL` if none is left; a correct program always checks for this. The arrow, as in `n->text`, reaches a field of a structure through a pointer. `strncpy` copies at most the given number of characters, and the line after it ensures that the text always ends with a zero byte, even if it was too long to fit. Safe C depends on details of this kind.

C has no garbage collector. Memory obtained with `malloc` must be returned with `free` exactly once, after its last use. Memory that is never freed is lost to the program; this is a **leak**. Using memory after freeing it, or freeing it twice, corrupts the heap, and the resulting faults may appear much later and in unrelated parts of the program. The usual rule is that one part of the program owns each piece of memory, and only that part frees it.

14.7 Talking to Hardware

Because C can convert any number into a pointer, it can reach a memory-mapped device register directly, as described in The Computer:

```
#define PINS (*(volatile uint32_t *)0xE0000000u)

void led_on(void)  { PINS |= 1u; }
void led_off(void) { PINS &= ~1u; }
```

The definition is read from the inside out. `0xE0000000u` is a number. Writing a type in parentheses before a value, a **cast**, converts the value to that type, so `(volatile uint32_t *)` turns the number into a pointer to a 32-bit unsigned value at that address. The `*` in front then refers to the value at the address, as `*p` did above. As a result, `PINS` can be read and assigned like a variable. `led_on` sets bit 0 without changing the other bits, and `led_off` clears it.

The word `volatile` is essential. It tells the compiler that the value can change, or that writing it has effects, outside the program's control. Without it, a compiler that sees two writes to the same address may keep only the second, and one that sees

a status register read in a loop may read it only once. With `volatile`, every read and write in the source is performed, in order. `volatile` does not, however, make a sequence of operations indivisible. `PINS |= 1u` is still a read, a change and a write, and if an interrupt handler changes the same register between them, its change is lost, as described in Operating Systems.

On a computer with memory protection, the operating system decides which device addresses each program may reach. On a microcontroller with no operating system, all of the hardware is driven in this way.

14.8 Writing C Well

A few practices prevent most problems in C:

- **Enable the compiler's warnings**, and treat them as errors. They detect many mistakes at no cost.
- **Check every result** that can fail: `malloc`, file opens, reads and writes.
- **Know the size of every buffer**, and pass it along with the pointer.
- **Keep functions short**, so that each one can be checked by reading it.
- **Test** functions individually, with known inputs, including the edge cases: empty, zero, one and the largest value.

14.9 Using the Three Together

Each language in this book has its own uses, as the table at the end of Thinking in Programs showed. They can also be combined. Many systems are built with a small, fast core in C and a more convenient language above it for scripting and experiments. When a Scheme program is too slow, the slow part can be rewritten in C and added to the interpreter as a built-in procedure, with the rest left unchanged.

So far, the hardware under these programs has been fixed. The next part of the book describes hardware that can be changed.

Part V

Changeable Hardware

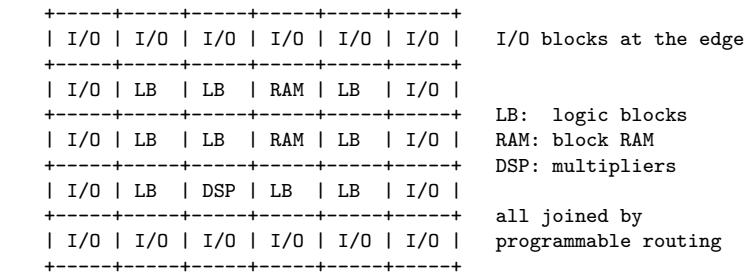
Chapter 15

Inside an FPGA

A field-programmable gate array is a chip containing many identical, uncommitted logic blocks, with programmable wiring between them. A configuration file sets up and connects those blocks to form the circuit it describes, such as a blinking light, a serial port or a complete computer. This chapter describes what is inside an FPGA and how a design becomes a configuration.

15.1 The Big Picture

An FPGA has four main kinds of component, arranged in a regular grid:



- **Logic blocks** implement the gates and flip-flops of a design.
- **Routing** connects the blocks to each other, through a mesh of wires and programmable switches.
- **Memory and arithmetic blocks** provide dedicated RAM and multipliers, which would be wasteful to build from logic blocks.
- **I/O blocks** connect the inside of the chip to its pins.

The FPGAs in this book have thousands or tens of thousands of LUTs, the elements described below, grouped into logic blocks. Which blocks are used, how each is set up and how they are connected is held in **configuration memory** distributed throughout the chip. In most FPGAs, including all of those in Machdyne's computers, this memory is SRAM. Its contents are lost when power is removed, so the configuration must be loaded again at every start.

15.2 Look-Up Tables

The main element of a logic block is the **look-up table**, or LUT. A 4-input LUT is a memory of sixteen bits. Its four inputs form an address from 0 to 15, and its output is the bit stored at that address.

As described in Digital Logic, any function of four inputs is completely described by a truth table of sixteen rows. A LUT stores that truth table. With the appropriate sixteen bits loaded, it acts as an AND gate, an XOR gate, a multiplexer or any other function of up to four inputs, with the same delay whatever the function.

For this reason an FPGA can implement any logic design: its general logic contains no fixed gates, only small memories that can take the place of any gate.

The sixteen bits are part of the FPGA's configuration. Design tools usually write them as one 16-bit number, often called `INIT`, whose bit 0 is the output for address 0, bit 1 the output for address 1, and so on. To make a LUT compute the XOR of inputs 0 and 1, the required output is listed for each address. For addresses 0, 1, 2 and 3, the two low bits are 00, 01, 10 and 11, so the outputs are 0, 1, 1, 0; written with address 3 on the left, as binary numbers are, this is `0110`, hex 6. The two upper inputs do not affect the result, so the same pattern repeats four times: hex 6666, which Verilog, described in the next chapter, writes as `16'h6666`. For an AND of all four inputs, only address 15 gives 1: hex 8000.

A LUT has one output, so a function with several outputs needs one LUT for each, and functions of more than four inputs are divided among several LUTs. A full adder from Digital Logic, with three inputs and two outputs, requires two: one for the sum and one for the carry.

Different FPGA families use LUTs with 4 to 6 inputs, but the principle is the same.

15.3 Flip-Flops and Carry Chains

Each LUT in a logic block is paired with a **flip-flop**, so that its output can be stored at a clock edge. Together they form a small piece of combinational logic followed by a register, which is the basic unit of synchronous design.

Logic blocks also contain **carry chains**: dedicated, fast connections that pass the carry from one block to the next. Adders and counters are common enough that FPGAs provide this special path rather than sending the carry through general routing. A 24-bit counter on a Lattice ECP5 (one of the families described below), for example, uses 24 flip-flops and 12 carry cells, each of which adds two bits.

15.4 Routing

Most of an FPGA's area is occupied by **routing** rather than logic: wires of various lengths running between the blocks, and programmable switches where they cross. Each switch is controlled by one bit of configuration memory.

Much of the delay in an FPGA is caused by routing. A signal that crosses the chip through many switches takes longer than one that travels to a neighbouring block. For this reason the same design can reach different clock speeds depending on how its parts are placed, and an FPGA runs a given circuit several times more slowly than a custom chip, in which the connections are direct metal wires.

15.5 Block RAM and DSP Blocks

Memory built from LUTs and flip-flops would use them up quickly. FPGAs therefore include **block RAM**: dedicated SRAM blocks of a few kilobits each, distributed across the chip. A design uses them wherever it needs fast on-chip memory, such as a processor's caches and boot program, a screen font or buffers for incoming data. An FPGA with fifty-six 18-kilobit blocks has about 126 KB of block RAM, which is small compared with external memory but much faster.

DSP blocks are dedicated multipliers, often with adders attached, used for arithmetic in signal processing, graphics and cryptography. A multiplier built from LUTs would be large and slow; a DSP block performs a multiplication in one clock cycle.

15.6 I/O Blocks

Each pin of an FPGA has an **I/O block** that can be configured as an input, an output or both. The configuration sets its voltage standard, its drive strength, whether it has a pull-up resistor, and whether it captures data on a clock edge at the pin itself. Pairs of pins can be used together for **differential** signals, which carry fast data on two wires with opposite polarity and are used for HDMI and other high-speed links. Some FPGAs have dedicated high-speed serializers at their pins for this purpose.

Every I/O block belongs to a **bank** of pins that share a supply voltage. The board designer chooses that voltage, commonly 3.3 V, and every pin in the bank must use it.

15.7 Clocks

FPGAs contain **global clock networks**: routing designed to deliver a clock signal to every flip-flop on the chip at nearly the same time. Ordinary routing cannot do this.

They also contain one or more **phase-locked loops** (PLLs), circuits that take a clock from an external crystal and produce clocks at other frequencies. A board with a 12 MHz crystal can use a PLL to produce a 48 MHz clock for its processor and a 25 MHz clock for its video output.

15.8 Clock Domains

A large design often uses several clocks: one for the processor, one for the video output at its pixel rate, one for USB. All the flip-flops driven by one clock form a **clock domain**. Within a domain, the timing rule from Digital Logic applies. Between domains it does not, because the edges of two unrelated clocks fall at moments that have no fixed relation to each other.

A signal that crosses from one domain to another must therefore be treated as if it came from outside the chip:

- **A single bit**, such as a flag, passes through two flip-flops in the receiving domain, which allows any metastability to resolve.
- **A number of several bits** cannot be passed this way. Its bits may change at slightly different moments, and the receiving domain may capture some of the old bits and some of the new, a value that never existed. A multi-bit value is passed with a **handshake**: the sender holds the value steady and raises a single flag, which crosses safely; the receiver reads the value and answers with a flag of its own.
- **A stream of data**, such as samples or pixels, passes through a **dual-clock FIFO** (first in, first out), a small buffer in block RAM that one domain writes and the other

reads, with its pointers passed between the domains in a form in which only one bit changes at a time.

A **reset** must cross in the same way. The reset input can be applied at any moment, but must be released in step with the clock of each domain; otherwise some flip-flops leave reset one cycle before others, and the design starts in a state its designer did not intend.

The design tools must be told the frequency of each clock, so that they can check every path. Their timing report states whether each clock meets its target. A design that fails its timing may still appear to work, but can fail when the chip is warm, when its supply is slightly low, or on another chip of the same type. The remedies are to shorten the slow paths, to divide a long calculation over several clock cycles, or to lower the clock.

15.9 Configuration

The file that sets every LUT, flip-flop, switch and pin is called a **bitstream**. It is produced by the tools described in Describing Hardware and is typically between a few tens of kilobytes and a few megabytes in size.

At power-on, an FPGA reads its bitstream from a small SPI flash chip beside it, which takes a fraction of a second. The circuit it describes starts to run only after this. For testing, a bitstream can also be sent directly from another computer over a USB cable, in which case it remains until the power is turned off.

On some boards, including many of Machdyne's, the flash chip is on a small removable module. Replacing the module replaces the whole design of the computer, so the same board can be one kind of computer with one module and an entirely different one with another. The flash can hold more than one bitstream, and some FPGAs can be instructed to reload themselves from a different address, so that a running design can pass control to another.

15.10 FPGA Capacity

FPGA capacity is usually stated in LUTs. Boards for hobbyists and small computers, including Machdyne's, use chips with a few thousand to tens of thousands of LUTs. Typical sizes of designs are:

Design	LUTs, roughly
blinking an LED	a dozen or two
a serial port	a few hundred
a small 32-bit RISC-V processor	1,000 to 3,000
a complete computer (Zeitlos)	15,000 to 20,000

A complete computer, with its processor, caches, memory controller, graphics, USB, audio and cryptographic accelerators, fills most of a 25,000-LUT FPGA and less than half of a 45,000-LUT one, which leaves room for additions.

15.11 FPGA Families and Open Tools

Several companies make FPGAs, and some of their families have open-source tools, written by people who studied the chips and documented their configuration formats:

- **Lattice iCE40**: small, simple, low-power FPGAs, the first to have a complete open toolchain.
- **Lattice ECP5**: larger FPGAs with DSP blocks and ample block RAM, the main family in Machdyne's computers.
- **Xilinx (AMD) 7-series**: large, fast FPGAs such as the Artix-7, with partial open support.
- **Cologne Chip GateMate**: FPGAs designed and made in Germany, supported by open-source synthesis and, more recently, open place and route.

Open tools are important for timeless computing. A design that can be built only with one company's licensed software can be maintained only while that software is available and runs on current computers. A design built with open tools, whose own source code can be rebuilt, can be maintained indefinitely.

15.12 FPGAs, Processors and Custom Chips

An FPGA can be compared with a microcontroller (MCU), which is a small processor chip with built-in devices, and with a custom chip, or ASIC (application-specific integrated circuit):

	MCU	FPGA	ASIC
What it is	fixed CPU, devices	programmable logic	fixed logic
Changed by	new software	new bitstream	a new chip
Speed	modest	modest	highest
Power	lowest	low to moderate	lowest per job
Cost each	lowest	moderate	lowest in bulk
Start-up cost	minimal	low	enormous

A microcontroller is suitable when its built-in processor and devices fit the task; most small controllers, including Machdyne's modules, are microcontrollers. A custom chip is suitable when millions will be made and the design will not change. An FPGA is suitable when the hardware itself must be designed, studied or changed: for parallel processing, unusual interfaces, recreations of old machines, and computers whose owners can inspect and improve the processor.

An FPGA also protects a design against obsolescence. A processor design written in a hardware description language is not tied to any one chip. When a particular FPGA goes out of production, the design can be moved to another, with the adjustments described in Moving a Design to Another Board. In this way the computer can outlast the chip it was first built on.

Chapter 16

Describing Hardware

Circuits were once designed by drawing them. Digital circuits of any size are now described in text, in a **hardware description language** (HDL), and tools convert the description into a chip or an FPGA configuration. This chapter introduces Verilog, the HDL used for the designs in this book, and the steps from a description to a working circuit.

16.1 A Different Kind of Language

Verilog resembles C, but it is not a programming language in the usual sense. A program is a list of steps carried out one after another. A Verilog description is a list of pieces of hardware that all exist, and all operate, at the same time.

The line `assign sum = a ^ b;` does not compute a value once. It places an XOR gate whose output is always the XOR of its inputs, for as long as the circuit has power. The order of such lines in the file does not matter, as the order of parts on a circuit diagram does not.

Two languages are in common use: **Verilog**, created in 1984, and **VHDL**, from the same period. Both are standardized and widely supported, and the ideas of one apply directly to the other.

16.2 Verilog Modules

A Verilog design is built from **modules**, each with named **ports** for its inputs and outputs. The full adder from Digital Logic is written as follows:

```
module full_adder (
    input  wire a, b, cin,
    output wire sum, cout
);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));
endmodule
```

A **wire** is a connection, carrying whatever drives it. Each `assign` statement is a piece of combinational logic driving a wire, written with the same operators as the Boolean algebra in Digital Logic. On an ECP5 FPGA, this module becomes two LUTs.

A module is used inside another by creating an **instance** of it, as a circuit board can carry several copies of one chip. A 32-bit adder could be made of 32 full adder instances, and a computer of thousands of instances of hundreds of modules.

16.3 Numbers and Vectors

A port or wire can carry many bits at once. `wire [7:0] data` declares an 8-bit **vector**, with bits numbered 7 (most significant) down to 0. Single bits and ranges are selected with brackets, so `data[0]` is the lowest bit and `data[7:4]` the top four.

Numbers are written with their width and base: `8'd200` is the 8-bit decimal number 200, `4'b1010` is four bits in binary, and `32'hE000_0000` is 32 bits in hex, with an underscore for readability. Braces join vectors together: `{4'b0000, data[7:4]}` is an 8-bit value whose low four bits are the top of `data`.

16.4 Registers and Clocks

Combinational logic has no memory. Flip-flops and registers are described with an `always` block that runs at each rising clock edge:

```
// Blink an LED from a fast clock by counting clock cycles.
module blink #(
    parameter WIDTH = 24            // 2^24 cycles at 12 MHz: 1.4 s
) (
    input  wire clk,
    output wire led
);
    reg [WIDTH-1:0] count = 0;

    always @(posedge clk)
        count <= count + 1;

    assign led = count[WIDTH-1];    // the top bit changes slowest
endmodule
```

`reg [WIDTH-1:0] count` declares a 24-bit value. Anything assigned inside an `always` block must be declared as a `reg`, whether or not it becomes a hardware register; this one does. `always @(posedge clk)` means "at every rising edge of `clk`", and `count <= count + 1` loads the register with its value plus one. This is a counter built from 24 flip-flops and an adder, as in Digital Logic. Its top bit changes every 2 to the 23rd cycles, which with a 12 MHz clock is about every 0.7 seconds. The LED is therefore on for 0.7 seconds and off for 0.7 seconds, and one complete blink takes the 1.4 seconds given in the comment.

`WIDTH` is a **parameter**, a constant that can be set differently for each instance. A testbench, described under Simulation below, can create a 4-bit version, which blinks after a few cycles instead of millions, and check it quickly in simulation.

16.5 Two Kinds of Assignment

Inside an `always` block, Verilog has two assignment operators, and confusing them is the most common beginner's mistake.

The **non-blocking** assignment `<=` describes a register loading at the clock edge. All non-blocking assignments in a clocked block take effect together, using the values from before the edge, as physical flip-flops do. The following swaps two registers correctly:

```
always @(posedge clk) begin
    a <= b;
    b <= a;
end
```

In hardware, these lines describe two flip-flops with their connections crossed: the input of `a` is wired to the output of `b`, and the input of `b` to the output of `a`. At each edge both load at the same instant, so they exchange their values.

The **blocking** assignment `=` takes effect immediately, so the next line sees the new value. Written with `=`, the second line above would read the new value of `a`, and both registers would end with the old value of `b`. The blocking assignment is used for combinational logic written inside `always @*` blocks, where `@*` means "whenever any input changes".

The general rule is to use `<=` in clocked blocks and `=` in combinational ones, and not to mix them in one block.

16.6 State Machines in Verilog

The traffic light from Digital Logic is written as follows:

```
// The traffic light from Digital Logic, as a state machine.
module traffic (
    input  wire clk,
    input  wire reset,
    input  wire done,               // the timer has run out
    output wire green, yellow, red
);
    localparam GREEN = 2'd0, YELLOW = 2'd1, RED = 2'd2;

    reg [1:0] state;

    always @(posedge clk) begin
        if (reset)
            state <= RED;
        else if (done)
            case (state)
                GREEN:   state <= YELLOW;
                YELLOW:  state <= RED;
                default: state <= GREEN;
            endcase
    end

    assign green  = (state == GREEN);
    assign yellow = (state == YELLOW);
```

```
    assign red    = (state == RED);
endmodule
```

`localparam` gives names to the state numbers. The `always` block is the state register together with its next-state logic. On reset, the state becomes red; otherwise it advances when the timer is done and is unchanged when it is not. The `case` statement selects by the current state. The outputs are combinational, each true only in its own state. This module and the table in Digital Logic describe the same transitions; the module adds a reset.

Every state machine needs a **reset**, a defined state to start from. An FPGA gives its flip-flops known values when it is configured, but a custom chip does not. In addition, a simulator shows unknown values until something sets them, and a running circuit sometimes needs to start again without being reloaded.

16.7 Simulation

Hardware is difficult to debug once it is built, because a value inside a chip cannot be observed directly. Designs are therefore first tested by **simulation**, in which a program computes what the circuit would do, cycle by cycle, and every signal can be inspected.

A simulation is driven by a **testbench**. A testbench is a Verilog module that describes no hardware; it acts as a script that creates an instance of the design, supplies its inputs and checks its outputs. Testbenches may use features that cannot become hardware, such as delays, loops over test values and printed messages. The main part of the full adder's testbench tries all eight input combinations and compares each result with ordinary addition:

```
for (i = 0; i < 8; i = i + 1) begin
    {a, b, cin} = i;
    #1;
    if ({cout, sum} !== a + b + cin)
        errors = errors + 1;
end
```

`#1` waits one time unit for the logic to settle.

A simulator can also record every signal over time, to be viewed as **waveforms**: one horizontal line per signal, drawn high or low, with numbers written along the line for multi-bit values, all against a common time axis, much as a logic analyser displays a real circuit. A fault is found by locating the first moment at which

some signal differs from what was intended and following it back to its cause.

A whole computer can be simulated in the same way. The testbench loads a program's machine code into the simulated memory, from a file of hex numbers, and lets the simulated processor run it. Simulation is many thousands of times slower than the hardware, so only short programs are practical, but every signal is visible at every cycle. Most faults in a new processor or device are found this way before the design is ever loaded into an FPGA.

16.8 From Description to Bitstream

Converting a tested design into an FPGA configuration takes several steps, each done by a tool:

```
Verilog source          what the circuit does
     |
 synthesis              -> netlist: LUTs, flip-flops, RAMs, wires
     |
 place and route        -> each part given a place on the chip,
     |                     each wire a path through the routing
 bitstream generation   -> the configuration bits
     |
 programming            -> loaded into the FPGA or its flash
```

Synthesis reads the HDL, determines the logic it describes, simplifies it, and maps it onto the building blocks the FPGA provides. The result, a **netlist**, lists every LUT and flip-flop and how they connect. Yosys is a widely used open synthesis tool.

Place and route decides which physical block on the chip each part of the netlist will use, then finds paths through the routing for every connection. It also analyses timing and reports the fastest clock the design can use. The open tool nextpnr is widely used for place and route.

Bitstream generation, called packing by some of the open tools, converts the placed and routed design into the configuration format of the particular FPGA.

Programming loads the bitstream into the FPGA, or writes it into the configuration flash so that the FPGA loads it at every start.

The tools also need a **constraints file**, which says which signal

goes to which physical pin, what voltage standard each pin uses, and how fast each clock runs. A board's constraints file is written once and reused by every design for that board.

These steps can take seconds for a small design and many minutes for a large one, because placement and routing are difficult search problems. For a design of any size they need far more memory and speed than a small computer has, so they run on a larger computer. Experimental tools that run on an FPGA computer itself can build simple designs, but not a design as large as the computer's own.

16.9 Reading the Reports

The tools report two numbers that matter on every build.

Utilization states how much of the chip the design uses: the LUTs, block RAMs and other resources used, out of those available. As a chip fills, the placer has less room to work with and timing becomes harder to meet. Above about three quarters full, designs become noticeably harder to build.

Timing gives the maximum frequency of each clock, based on the critical path described in Digital Logic. If it is below the frequency at which the design runs, the circuit may work on the bench and fail when warm, or fail in rare and confusing ways. The remedies are described under Clock Domains.

16.10 Debugging on the Board

A design that works in simulation can still fail on the board: a pin is assigned wrongly in the constraints, an external chip behaves differently from its model, or a timing problem appears only in hardware. Signals inside an FPGA cannot be probed directly, so a design is given ways to show what it is doing:

- **LEDs** driven by internal signals show whether a clock is running, a state has been reached or an error has occurred.
- **Spare pins** can carry internal signals to an oscilloscope or logic analyser.
- **A serial port** in the design can report values as text, which is often the first sign of life from a new design.

- **An internal logic analyser**, a small circuit added to the design, records chosen signals into block RAM when a condition occurs, and sends the recording to another computer to be displayed as waveforms.
- **JTAG**, a standard debug connection present on most larger FPGAs, is used to load bitstreams directly and, with suitable support in the design, to read internal state.

A known good bitstream is kept, changes are made one at a time, and each new part is tested by itself before it is joined to the rest.

16.11 Classic Mistakes

- **Blocking assignments in clocked blocks.** Registers should be assigned with `<=`.
- **Accidental latches.** In a combinational `always @*` block, every output must be assigned in every case. If an `if` has no `else`, the tool must keep the old value and so builds a latch, usually unintentionally.
- **Two drivers.** A signal can be driven from only one place. Assigning the same register in two different `always` blocks fails in synthesis, even if the simulator accepts it.
- **Missing reset.** Registers that must start in a known state need a reset or an initial value.
- **Unsynchronized inputs.** A button, or a single-bit signal from another clock domain, must pass through two flip-flops before it is used, as described in Digital Logic; multi-bit values need the methods in Clock Domains.
- **Width mismatches.** Assigning a 32-bit value to an 8-bit register silently drops the top bits. Tools warn about this, and the warnings should be read.
- **Simulation-only code.** Delays such as `#1`, printed messages and the scripted steps of a testbench cannot become hardware, and synthesis ignores or rejects them. A design that depends on them works only in the simulator.
- **Making clocks with logic.** Dividing a clock with a counter and using the result as another clock causes timing problems. The alternatives are to use a PLL, or to keep one clock and use the counter as an enable.

The blink module above follows the last rule. It does not create a slow clock; it uses the top bit of a counter that runs on the fast

one.

Chapter 17

Systems on a Chip

A **system on a chip** (SOC) places a complete computer, with its processor, memory controller, graphics and peripherals, on a single chip. The processors in phones and tablets are SOCs: each is designed once, made in millions, and cannot be changed once made. In an FPGA, an SOC is a design like any other, often called **gateware** by analogy with software. Every part of it can be read and changed. This chapter describes how an SOC is organized, and follows one small addition from hardware to software.

17.1 Soft Processors

A processor described in an HDL and built in FPGA logic is a **soft processor**, or softcore. Some FPGAs also contain a fixed **hard processor** beside their logic, which is faster but cannot be changed.

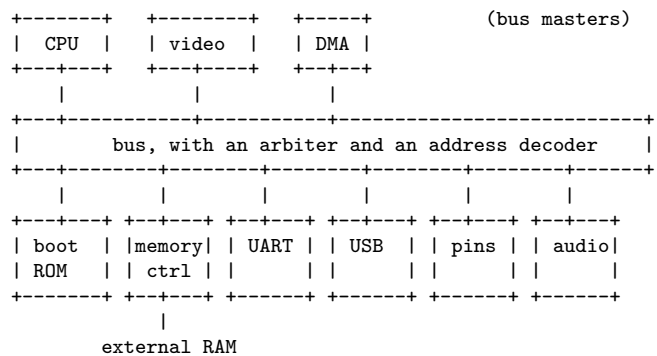
There are many open softcores for RISC-V. Some are small and simple, completing an instruction every few cycles and using one or two thousand LUTs. Others are pipelined and add caches and memory management, at several times the size. The Zeitlos SOC uses PicoRV32, a small, well-tested RISC-V core, and also has an experimental core of its own, Zeitlos32.

A soft processor runs much more slowly than a hard one, typically at tens of megahertz. Its advantages are:

- **It can be read.** Every part of the processor's logic is visible in its source, so it can be audited, studied and trusted to a degree that no commercial chip allows.
- **It can be fixed.** A processor bug is corrected by changing the source and rebuilding the bitstream.
- **It can be extended.** New instructions or features can be added for a particular need.
- **It can be replaced.** A machine built with one instruction set can later be given another, by loading a different design into the same hardware.

17.2 The Shape of an SOC

A typical FPGA computer is organized as follows:



- The **CPU** fetches instructions and moves data over the bus.
- Other **bus masters**, such as the video circuit that reads the framebuffer and the DMA engines described in The Computer, use the bus as well. An **arbiter** decides which master uses the bus when two request it at once.
- An **address decoder** turns each address into a select signal for one device.
- The **boot ROM** is a small program held in block RAM, built into the bitstream itself, so that the processor has a program to run as soon as the FPGA is configured. It is the system's firmware and its bootloader: it loads the operating system from flash.
- The **memory controller** connects the bus to external SRAM, SDRAM or DDR memory.
- The **peripherals** each present a few registers on the bus.

17.3 Adding a Device, End to End

This section traces how a program affects the physical world, using the smallest useful peripheral: one register that the processor can write and read back, whose bits drive eight LEDs. It is the simplest form of the general-purpose pins in the address map of The Computer, at `0xE000_0000`.

A transfer on the bus has a few signals. The master places the **address** on the bus and, for a write, the data on **wdata** with **we** (write enable) set. The address decoder raises **sel** for the one device the address belongs to. For a read, the device answers with its data on **rdata**. Most buses, Wishbone among them, also have an **acknowledge** signal, with which the device says that the transfer is complete, so that a slow device can take several cycles. The device below always answers at once, so its acknowledge is left out.

```
// A peripheral with one register: a byte the CPU can write and
// read back, whose bits drive eight LEDs.
module led_reg (
    input  wire        clk,
    input  wire        reset,
    input  wire        sel,        // the bus selects this device
    input  wire        we,         // and it is a write
    input  wire [31:0] wdata,      // the data the CPU is writing
    output wire [31:0] rdata,      // the data the CPU reads
    output wire [7:0]  leds
);
    reg [7:0] value;

    always @(posedge clk)
        if (reset)
            value <= 8'd0;
        else if (sel && we)
            value <= wdata[7:0];

    assign rdata = {24'd0, value};
    assign leds  = value;
endmodule
```

The register `value` loads the bottom byte of the bus data at a clock edge when this device is selected and the bus is writing. Its contents can be read back at any time and always drive the `leds` output.

The device is selected by the address decoder. In a system with only a few devices, the decoder can look at the top four bits alone:

```
// the address decoder: the top four bits pick the device
wire sel = (addr[31:28] == 4'hE);
```

With this decoder the device answers at every address from `0xE000_0000` to `0xEFFF_FFFF`. A fuller decoder checks more bits, so that several devices can share the range, as the pins and their related devices do in a real system.

The `leds` output is connected to eight pins of the FPGA, and the board's constraints file says which ones. After synthesis, place and route and packing, the new bitstream contains the device.

On the software side, the function `led_on` from the chapter C is a single line, `PINS |= 1u`, where `PINS` names the 32-bit value at address `0xE000_0000`.

A call to `led_on` proceeds as follows:

1. The compiler turns `PINS |= 1u` into an instruction that puts the address `0xE000_0000` in a register, then three more: a load from that address, an OR with 1, and a store back.
2. The load puts the address on the bus. The decoder sees `E` in the top four bits and selects `led_reg`, which places its register on `rdata`. The value comes back into a CPU register.
3. The ALU sets bit 0.
4. The store puts the address and the new value on the bus with write enable set. At the next clock edge, `value` loads the new byte.
5. The `leds` output changes, the pin goes to 3.3 V, current flows through the resistor and LED from Electricity, and the light comes on.

These five steps pass through most of the layers described in this book. Larger peripherals, such as a serial port, a sound mixer or a USB controller, work in the same way, with more registers and more logic behind them.

17.4 Hardware and Software Together

Because its hardware can be changed, an FPGA computer allows a choice that ordinary computers do not: whether a task is done in software or in hardware.

Software is easier to write and change. Hardware can be much faster for some tasks, because it can perform many operations at once and does not spend cycles fetching and decoding instructions. Suitable tasks are small, regular and frequently repeated:

- copying and filling rectangles of pixels, done by a **blitter**
- drawing lines
- mixing several streams of sound
- the multiplications of large numbers on which public-key cryptography depends
- hash functions such as SHA-256

The Zeitlos SOC can include hardware for each of these, and each makes a task practical on a 48 MHz processor that would otherwise be too slow. The usual approach is to write everything in software first, measure where the time is spent, and move only the slowest part into hardware, behind a few registers.

17.5 Moving a Design to Another Board

A design written in an HDL is not tied to one board, and moving it to another is the most common way of adapting an FPGA computer. Most of the design is unchanged: the processor, the bus, the peripherals, the address map and every program that runs on them. The changes concern the physical board:

- **Pins.** The constraints file is rewritten for the new board: which FPGA pin carries each signal, and with what voltage standard. Every pin must suit the voltage of its bank.
- **Clocks.** The new board's crystal may run at a different frequency, so the PLL settings change. A slower or fuller chip may not meet timing at the old clock speed, and the design may need to run slower.
- **Memory.** SRAM, SDRAM, DDR and PSRAM each need a different memory controller, with timings for the particular chip. The memory size changes what software can do.

- **Chip-specific blocks.** PLLs, block RAM, special I/O cells and differential outputs differ between FPGA families. A well-organized design keeps each of them in a small wrapper module, so that only the wrapper changes.
- **Capacity.** A smaller chip may not hold everything. Optional parts, such as accelerators or a second serial port, are left out with a configuration switch.
- **Outputs.** VGA, HDMI and composite video, USB and audio each depend on what the board has.

The usual practice is to keep everything board-specific in one place: a top-level module, a constraints file and a short list of configuration settings for each board. A new board is then brought up in small steps: first a blinking LED, to confirm the clock, the pins and the programming method; then a serial port, for messages; then a memory test; and finally the full system. One SOC design can thus support a whole family of boards, including boards from other makers.

17.6 Linux on an FPGA

Linux can also run on an FPGA computer. It needs a softcore with a memory management unit, as described in Operating Systems, and at least tens of megabytes of memory. LiteX, an open framework for building SOCs, can generate a Linux-capable system for many FPGA boards.

Kakao Linux, a small distribution for Machdyne's larger computers, is built this way. Linux provides thousands of existing programs, at the cost of size, complexity and start-up time. A system designed for the hardware can be small enough to be understood completely. With the configuration flash on a removable module, the same computer can run either.

17.7 Recreating Old Machines

An FPGA can also be configured as a computer from the past. Projects have recreated, closely and sometimes down to individual gates, the processors and custom chips of the Apple II, the Commodore 64, the Amiga, early game consoles and many more, from documentation, from careful measurement and sometimes from microscope photographs of the original chips.

This differs from ordinary emulation, in which a program on a modern processor imitates the old one instruction by instruction. The FPGA becomes hardware that behaves like the original, clock cycle by clock cycle, so software that depended on precise timing also runs correctly. It is one of the most reliable ways to preserve old software, and shows that a computer's design can outlive the chips it was first made from, which is a central aim of timeless computing.

Chapter 18

Small Computers

Not every part of a system needs hardware that can be redesigned. Most computers in the world have no screen and no keyboard. They are built into keyboards, chargers, washing machines, cars and storage devices, and each runs one program for its whole life. These are microcontrollers, and they are well suited to many timeless applications, especially automation. This chapter describes them, and explains how small programmable modules can be combined with a larger computer to build systems that last.

18.1 Microcontrollers

A **microcontroller** is a complete computer on one inexpensive chip, with a processor, flash memory for its program, a small amount of RAM and a set of built-in peripherals. Typical peripherals are:

- general-purpose pins
- timers, for measuring time and producing PWM signals
- analog-to-digital converters
- UART, SPI and I2C controllers

A small RISC-V microcontroller might have 16 KB of flash and 2 KB of RAM. This is enough for reading sensors, switching outputs and making decisions many thousands of times a second.

Unlike an FPGA computer, a microcontroller is fixed. Its proces-

sor and peripherals are set by the manufacturer; only the program can be changed. In return, a microcontroller is cheap, small, easy to use and economical with power. Many can sleep on a few microamps and wake on a timer or a pin change, so they can run for years from a small battery.

The program in a microcontroller is called its **firmware**. It is usually written in C, compiled on another computer, and written into the chip's flash through a programming interface: either a debug connection of one or two wires, such as SWD or the older and wider JTAG, or a small resident program called a **bootloader** that accepts new firmware over a serial port or USB.

18.2 An Interpreter on the Chip

Alternatively, an interpreter can be written into a microcontroller's firmware once, and programs sent to it afterwards as text. Machdyne's modules work this way, with Machdyne BASIC as their firmware.

This approach has several advantages for timeless computing:

- **No toolchain is needed** to change a program. Any computer, or any device that can send text over a serial line or a bus, can program the module.
- **Programs are readable.** The program inside the module is plain text that can be listed, read and changed in place years later, by someone who has never seen it before.
- **Programs are portable.** A BASIC program does not depend on the chip that runs it, so it can be moved to a different module when the old chip is no longer made.

The disadvantage is speed. An interpreted program runs many times more slowly than compiled firmware. For a controller that checks a temperature once a second, this does not matter.

18.3 Modules

A **module**, in this sense, is a microcontroller on a small circuit board, with enough around it to be useful on its own and a standard set of contacts for connecting it to a larger system.

Machdyne's modules are designed to last. Each combines a RISC-V microcontroller with long-lived non-volatile memory, either F-RAM or EEPROM, which holds its programs and data as files. The first of them, the LS10, has a microcontroller with 16 KB of flash and 2 KB of RAM, and 8 KB of F-RAM. The modules fit a 12-pin footprint that can be soldered to a board, pushed into a socket or plugged into a six-pin Pmod connector. The first six pins follow a standard called **Sechs**.

18.4 The Sechs Interface

Sechs, German for "six", defines six contacts:

Pin	Name	Use
1	A	shared I2C bus: clock
2	B	shared I2C bus: data
3	C	the module's own I/O
4	D	the module's own I/O
5	GND	ground
6	3V3	3.3 V power

Pins A and B form an I2C bus, as described in Talking to the World, which is shared by every module connected to it. Pins C and D belong to each module, for its sensor, its output, a local I2C device or a serial console.

Sechs describes three roles:

- a **module**, which runs programs
- a **carrier**, the board a module plugs into, which supplies power and determines the use of pins C and D
- a **master**, usually a host computer, which controls modules over the shared bus

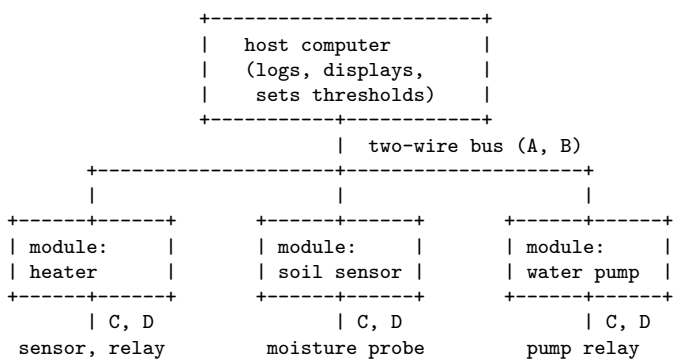
Over the bus, a host can find every module by scanning addresses and ask each one what it is. It can stop and start a module's

program, read and write its sixteen program registers, send it a new program, and open a console to type commands into it directly. A new module answers at a default address, and the host can assign each module its own address so that several can share a bus.

The safety rules listed in Talking to the World (inactive pins at power-on, a boot window in which the host can always stop the module, and programs replaced only when complete) make this robust. Sechs is still a draft standard, and the details of its registers may change, but its pinout and these rules are its settled foundation.

18.5 Building Systems from Modules

Modules allow a larger system to be built by giving each task its own small computer.



In this greenhouse, each module runs its own `BOOT` program, like the heater controller in BASIC. Each module continues to work if the host is switched off or fails. The host reads each module's registers to log and display its state, and writes registers to change settings. A failed module can be replaced with a spare and reprogrammed in minutes, while the rest of the system continues to run.

This design follows the principles in Timeless Computing. Each part is simple enough to be understood completely, and each can be replaced. A failure affects one part rather than the whole system. The programs are stored as text in long-lived memory, so whoever maintains the system in future can read and change

them.

18.6 Portability: Machines and Languages

There are two general ways to make programs outlive the hardware they were written for. The first is a common **instruction set**, or virtual machine: every device runs the same machine code, either directly or through a small interpreter. The second is a common **language**: every device accepts the same source programs and runs them in whatever way suits its chip.

Machdyne's modules have used both. The Zwölf platform, from which Sechs developed, defined a small 8-bit processor with a stack and a few instructions, which every module was to implement regardless of the chip it was built from. Sechs keeps the six-pin interface but does not define a common instruction set. Portability comes instead from Machdyne BASIC, which can be interpreted on some chips and could be compiled on others.

A common instruction set makes compiled programs portable, and many systems have used one, from early educational computers to modern bytecode languages. A common language is simpler to build, and leaves each device free to run programs in its own way. In either case, what matters is a stable, documented layer between the programs and the hardware, so that either can change without the other.

18.7 Module Lifetime

Two measures describe the life of a small device. **Data retention** is how long its memory keeps what was written to it. For F-RAM, as described in Memory and Storage, it is rated at a century or more, depending on temperature. The **functional life** of the device as a whole depends on its other parts (the microcontroller, the solder joints, the connectors and the capacitors) and on its operating conditions, and it is generally shorter.

Modules address this with the approach used throughout the book. The important information is kept in the most durable form, the parts are cheap and replaceable, and the design and documentation are open, so that a replacement can always be made.

The next part combines this material in one complete machine.

Part VI

The Machine

Chapter 19

The Machine

The previous chapters described computers in general. This chapter describes a particular computer: a Machdyne FPGA computer running Zeitlos. It does not assume that the reader has one. It explains how the ideas of this book appear in a real machine, how it can be adapted, and how it can be returned to a working state when an experiment goes wrong.

Zeitlos changes quickly, and the system's own documentation describes it in exact detail. This chapter gives an overview that makes that documentation easier to read.

19.1 What It Is

With a keyboard, mouse, monitor and USB power supply connected, the machine starts to a graphical desktop. The screen is 640 by 480 pixels in a single colour, in the manner of early workstations: white, amber or green on black, or black on white like paper. A dock along the bottom starts programs, and windows can be moved, resized and stacked.

The programs supplied with it cover the timeless applications listed in Timeless Computing: a text editor, a file browser, a document reader, a spreadsheet, a calculator, a calendar and clock, a terminal, an image viewer, music and sound players, a browser for simple web pages and for Gopher and Gemini, and a chat client,

along with games and educational programs. This book is among the documents it carries.

The core of the system, consisting of the kernel, the drivers, the window manager, the terminal, the editor, the file browser and the Scheme language, is stored in the computer's flash memory, so the machine works with no storage card at all. A microSD card adds further programs and space for files.

19.2 The Hardware

Machdyne makes several FPGA computers, and Zeitlos also runs on some boards from other manufacturers, so details vary. The common parts are:

- an **FPGA**, usually a Lattice ECP5, which becomes the whole computer
- **external RAM**, from 1 MB of SRAM to tens of megabytes of SDRAM, or larger DDR3 memory on some boards
- **configuration flash**, often on a removable MMOD, holding the FPGA's design and the core of the operating system
- **USB host ports** for a keyboard, a mouse, storage sticks and network adapters
- **video output**: DVI through an HDMI connector, VGA, or composite for a television
- a **microSD** slot
- **Pmod** sockets for expansion
- power from a USB supply, a few watts

19.3 The System on the Chip

Each part of the Zeitlos SOC was described in an earlier chapter:

In this book	In the Zeitlos SOC
The Computer	RISC-V CPU (RV32IM), 48 MHz, PicoRV32 core
The Computer	Wishbone bus, address map, caches
Operating Systems	address translation per program, memory protection
Talking to the World	framebuffer, blitter and line drawing

In this book	In the Zeitlos SOC
Talking to the World	8-channel audio mixer, USB host, serial ports
Talking to the World	pins on the Pmod sockets, I2C and SPI
Communication	Ethernet
Security	random number generator
Systems on a Chip	multiplier and hash accelerators for cryptography

One address space, with a window per program. There is no memory management unit. Zeitlos uses the simple translation by offset described in Protecting Memory: each program is loaded somewhere in main memory, but always sees itself in the window at `0x8000_0000`, and on each context switch the kernel sets the offset for the next program. Programs therefore need no adjustment for where they are loaded. Every byte of memory also keeps its single physical address. When a program sends data in a message, the kernel translates the program's window address into the physical one, so the receiver can read the data where it lies, without a copy.

Protection for containment, not secrecy. A memory protection unit, of the kind described in the same section, allows a program to write only its own memory and the devices it is permitted to use, and to run instructions only from its own memory; it may read nearly any memory. A faulty program is stopped with a report, and everything else continues. Zeitlos is a single-user system, and this protection exists to contain errors, not to defend against a hostile program the owner chose to run.

There is also a **game mode**, in which a 320 by 240 region of the screen is doubled in size to fill the display and, on boards built with colour, can use up to sixteen colours. It is used by games and by the BASIC computer described below.

19.4 Starting Up

The start-up sequence closely follows Operating Systems:

1. The FPGA loads the Zeitlos design from its configuration flash and becomes the computer. Some boards first run a small bootloader that waits a few seconds for an update over USB.
2. A boot program, built into the design in block RAM, copies the kernel from flash into main memory and starts it.
3. The kernel starts the filesystem driver, then the USB driver, reads the configuration file, and starts the window manager, the network driver and any other services the configuration specifies.
4. The desktop appears.

A program is looked for first in the `/apps` folder of the SD card, then in flash; only the kernel and the filesystem driver always come from flash. As a result, a newer version of a program can be tried from the card, as described under Getting Back to Safety.

19.5 The Operating System

Zeitlos is a **microkernel**, as described in Operating Systems. The kernel schedules programs, manages memory, passes messages, routes interrupts and enforces protection. The filesystem, USB and network are driver programs that run like any other. If one crashes, the kernel restarts it a moment later, and most programs using it recover, although open files may need to be opened again.

Scheduling is preemptive, driven by a timer several hundred times a second. When a program sends a message and waits for the answer, the processor switches directly to the program that will answer it. As a result, the desktop responds without noticeable delay on a 48 MHz processor: work passes directly to where it is needed, and programs waiting for an event use no time.

Programs communicate through messages and through **ports**: named services that other programs can connect to. A terminal window can connect in the same way to the Scheme interpreter, to a shell modelled on Unix (following POSIX, the standard that describes Unix-like systems), to the BASIC computer, to a serial line, or to another machine across the network.

Files are arranged in one tree. The main folders are:

Path	Contents
/	the SD card, with the flash underneath
/apps	programs
/data	each program's own files, one folder each
/docs	documentation, including this book
/sys	the system configuration
/ram	a RAM disk, fast and lost at power-off
/usb	a USB stick, when one is plugged in
/home	the owner's files

Settings are kept in a plain text configuration file, which can be read and edited like any other. A few settings that belong to the machine rather than to a card, such as the password, are kept in a small, power-safe area of flash.

19.6 The Languages

All three languages from the programming chapters are on the machine.

Scheme is always available, from flash, through a terminal window. Besides standard Scheme, it has procedures for files, windows and drawing, starting and stopping programs, simple network transfers, and the pins and buses on the expansion sockets, each with built-in help. It is a quick way to automate a task or to explore the system.

BASIC runs as a separate small computer, in the tradition of the 1980s. It has a 320 by 240 screen, shown in a window or full screen, graphics statements and its own files in one folder, but no access to the rest of the system, which makes it a safe environment for learning. Programs for Machdyne's modules can be written and tried here, or on a virtual module that imitates the limits of a real one exactly, before being sent to a real module.

C programs are compiled on another computer with the standard open tools, as Zeitlos itself is. An experimental compiler, `zcc`, runs on the machine itself and can build simple programs, but not the kernel or the system's own programs. On machines with

enough memory, the Unix-style shell provides familiar commands, pipes and the vi editor.

19.7 Messages Between Machines

Zeitlos includes a bulletin board system, of the kind described in Communication, and a protocol for exchanging messages between machines, **zfed**. It follows the store-and-forward tradition of the bulletin boards rather than the always-connected servers of the web:

- **No central server.** Each machine, or **node**, chooses the other nodes it exchanges messages with, and any arrangement of connections works.
- **Absence is normal.** When two nodes connect, each takes from the other the messages it does not yet have. A machine that is switched off for a week, or reachable only now and then over a slow link, loses nothing.
- **Signed, unchangeable messages.** Every message is signed by the node that made it and named by its hash, as described in Security. Any node can pass a message on, but none can alter it, and a message that arrives by two routes is stored once.
- **Explicit membership.** A network is defined by a signed list of its members, and messages from outside it are refused at once.
- **Modern cryptography.** Connections between nodes are encrypted with a hybrid of classical and post-quantum methods.

Forum posts, private mail between nodes and other small signed items travel in this way, between Zeitlos machines and larger computers alike. The protocol is designed so that a 48 MHz FPGA computer can take a full part in it.

19.8 Adapting the Machine

The machine can be adapted at several levels, each deeper than the one before.

1. Settings. Screen mode and colour, keyboard layout, time zone, the services that start, and many other settings are lines in the configuration file. Most can be changed with the settings program.

2. Scripts and schedules. A few lines of Scheme can rename files, process text, fetch data from the network or open a small window with buttons. A built-in scheduler runs programs at given times or intervals, or at start-up.

3. Programs. A program written in C, compiled on another computer and placed in `/apps` becomes an application like the others, with its own window if it needs one.

4. Hardware. The Pmod sockets give direct access to pins from Scheme and C, with I2C and SPI built on top. Sensors, displays and relays connect here, as do Sechs modules: a command in the Unix-style shell can find modules on the bus, read their registers and send them BASIC programs. A virtual workbench can simulate parts such as pin expanders, LEDs and buttons on screen, so that a program's logic can be tested before anything is wired.

5. The gateware. The SOC itself is open Verilog, the machine's gateware. With the open FPGA tools on another computer, it can be modified, extended with a peripheral as described in Systems on a Chip, and rebuilt. An experimental set of FPGA tools, `zfpga`, runs on the machine and can build simple designs, such as a blinking LED, but not the machine's own gateware.

6. The system. The source of every program and of the kernel is available. Any program can be rebuilt and tried by placing the new version in the card's `/apps` folder while the original remains in flash. The kernel, the filesystem driver and the gateware are changed by rewriting the flash, which is best done on a spare MMOD.

19.9 An Example Adaptation

This example adapts the machine to log the temperature of a room every hour. It uses two of the levels above: the hardware (level 4), and scripts and schedules (level 2).

1. **The device.** A temperature sensor is connected to an I2C pin pair on a Pmod socket, following the steps in Connecting a Device. The data sheet gives its address, and a scan of the bus confirms that it answers.
2. **A trial.** In the Scheme terminal, the I2C procedures (described by `help`) read one byte from the sensor. The result is checked against a thermometer.
3. **A script.** A short Scheme file reads the sensor and appends a line with the date, the time and the temperature to a text file in `/home`. Run by hand, it adds one line to the file.
4. **A schedule.** Run every hour, by the system's scheduler or by a loop in the script itself, it builds up a record of the temperature in plain text that any program can read.
5. **A record.** The script, the schedule setting and a note of the sensor and its wiring are kept with the owner's other files, and copied with them.

A deeper change follows the same pattern. A new peripheral in the gateware is first tested in simulation, then loaded directly into the FPGA's configuration memory, which keeps it only until the power is turned off. Only when the new design works is it written to a spare MMOD. At each step the previous working state remains available.

19.10 Getting Back to Safety

The machine is designed so that a failed experiment can always be undone:

- **The core system is in flash**, and the machine starts without a card. A card with a faulty program on it can be removed.
- **Card copies override flash copies** for programs. Deleting the copy in `/apps` restores the original.
- **A rescue console** appears if the desktop cannot start, and can be called up by holding Escape during start-up. From it, programs can be started by hand and problems investigated.

- **Crashes are contained.** A faulty program is stopped and reported; a faulty driver is restarted, up to a few times a minute.
- **A design loaded directly is not kept.** A design loaded from another computer into the FPGA's configuration memory, without writing the flash, lasts only until the power is turned off; the FPGA then loads its working design from flash again. The region of flash holding the bootloader is protected from any action Zeitlos can take.
- **A second MMOD** is the safest place to try a new design. It can be fitted for an experiment and the original refitted afterwards. The machine can also copy its own configuration module to a spare as a backup.
- **Release images** restore the whole machine to its original state, and are written to the flash with the open tools from another computer.

With a spare MMOD holding a known good system, and a copy of the release image, almost any change to the machine can be undone in a few minutes.

19.11 Where to Go Next

The system's own documentation is in the `/docs` folder and can be read in the document reader. It begins with an index and a welcome page that covers the desktop and every keyboard shortcut, and has a page for most programs, each language and each part of the hardware. In the Scheme interpreter, `help` followed by a name explains any procedure.

Zeitlos is under active development. Some features described in its documentation are marked experimental, and some boards are better supported than others. Its documentation states which, and is always more current than a book.

Chapter 20

Computing That Lasts

Most people assume that electricity, spare parts, software updates and the Internet will always be available. Usually they are, but they are not guaranteed, and in some parts of the world they are already scarce. Natural disasters, conflicts, economic difficulties, changes in law and the decisions of distant companies can each remove something a computer depends on.

Survival computing, introduced in Timeless Computing, considers how computing could continue under those conditions. It is best treated as a design exercise rather than a prediction. The properties that make a computer more likely to survive difficult times also make it cheaper, simpler and easier to own in ordinary times. This chapter collects the practical lessons of the whole book.

20.1 Power Supply

Planning a supply starts from how much power the equipment uses, and therefore how much energy it needs. Energy is power multiplied by time, measured in **watt-hours** (Wh). A computer drawing 3 W for eight hours a day needs 24 Wh a day.

This is a small amount. A common USB power bank, rated at 10,000 to 20,000 mAh at its cells' 3.7 V, stores about 37 to 74 Wh, of which perhaps two thirds can be delivered at 5 V after conversion losses: enough for one to two days of such use. A

small solar panel can replace the energy used. A 20 W panel might deliver about 50 to 100 Wh on a sunny day, depending on season, latitude and weather, and much less on an overcast one. A practical system has a generous margin, and enough battery capacity for several cloudy days.

Several practices make the energy last longer:

- **The display often uses the most power.** A small monitor, or a television on composite video, may draw more than the computer.
- **Unused equipment can be switched off.** A module that sleeps between readings uses almost no energy.
- **A slower clock uses less power.** CMOS circuits use power mainly when they switch, as described in Electricity.
- **Conversion wastes energy.** Each step from one voltage to another loses some, so equipment that runs directly from 5 V USB power wastes less.

20.2 The Environment

Electronic components age at a rate determined by their conditions.

Heat is the most important cause of ageing. It accelerates nearly every kind of wear: memory loses data sooner, as described in Memory and Storage, capacitors dry out, and batteries degrade. A rough rule for many components is that every additional ten degrees Celsius halves their life, so equipment and stored media last longest when kept cool.

Moisture corrodes contacts and boards. Equipment is best stored dry, and equipment brought in from the cold is allowed to warm up before it is switched on, so that condensation can evaporate.

Dust blocks ventilation and can conduct when damp. **Static electricity** destroys chips, as described in Electricity. An enclosure protects against all of these. A case made with a 3D printer can also be replaced or modified freely.

20.3 Spares and Parts

Replacements are hardest to find when they are needed, so they are best obtained in advance:

- Keep **spares of parts that wear out or are easily lost**: storage cards, configuration modules, cables, power supplies and small programmable modules.
- Prefer **standard connectors and voltages**, such as USB power, Pmod sockets and common batteries, which can be replaced with whatever is available.
- Prefer **socketed and modular parts**. A memory module that plugs in can be replaced by hand; one soldered to the board cannot.
- Keep **basic tools**: a multimeter, a soldering iron, small screwdrivers and a magnifier. With these and the schematics, most faults in a small computer can be found, and many repaired.
- Keep **the schematics, data sheets and documentation** for every board in use, in a form that can be read without the board working. The schematic is the most important document for a repair, as described in Reading a Schematic.

20.4 Data

Data is kept intact over decades, on the media described in Memory and Storage, by a few practices:

- Keep **at least three copies** of anything important, on at least two kinds of medium, with one copy in another place.
- Use **plain formats**: text for writing, and simple, widely documented formats for everything else, which remain readable long after any particular program has gone.
- Record a **hash** of each important file, check copies against it periodically, and replace any that have failed.
- **Refresh** copies onto new media before the old media reach the end of their expected life. Flash cards and USB sticks are for moving data, not for keeping it for decades.
- Keep **small, vital data**, such as keys, passwords and essential notes, on media built for longevity, such as F-RAM, and on paper.
- Keep **a list** of what is stored where, so that every copy can

be found.

A routine makes these practices reliable. Work in progress is copied daily or after each session, to a second medium; everything is copied weekly or monthly to a medium that is then disconnected and stored apart. A copy that is always connected, or a folder synchronized automatically with another machine, is not a backup in this sense: a file deleted, damaged or encrypted by faulty or hostile software in one place is changed in the other as well. A backup is tested by restoring files from it, at least occasionally, because an unreadable copy is found out only when it is needed.

20.5 Software and the Means to Rebuild It

Software does not wear out, but it can become unusable. The programs that build it stop running on new computers, the websites that hosted it disappear, and the knowledge of how it was put together is lost.

The protection against this is to keep three things together, offline, and in more than one place:

- **the source code** of every program and design in use
- **the tools** needed to build it, ideally as source code themselves, together with the versions known to work
- **written instructions** for building and restoring everything, tested by following them

The most independent arrangement is a machine that can **rebuild itself**: one that can edit and compile its own software, and design and build its own hardware configuration, with no other computer involved. Each step towards this removes a dependency on something that might later be unavailable. Self-rebuilding is useful, but it is not a requirement of timeless computing, and few systems achieve it. Zeitlos does not: its experimental compiler and FPGA tools build simple programs and designs on the machine, but not the system itself. For such a system, the tools that build it, and a computer that runs them, are part of what must be kept.

Recoverability is equally important: a known good state that can be restored in minutes. That requires a spare copy of the configuration flash and of the system image, and a restore that

has been practised before it is needed.

20.6 Keeping Information

A computer without information has limited use. Collections of reference works, manuals, maps and books can be kept offline and read on small machines, especially in plain text. Such a collection should contain what would be needed if it could not be looked up elsewhere: technical documentation for the equipment in use, references for the owner's work, and the knowledge the owner would want to pass on. It should be updated whenever possible. Machdyne's Ark project prepares collections of this kind for offline computers, in sizes from a single compressed text file of about 2 MB to several gigabytes. Its smallest editions are condensed summaries of knowledge generated with AI, and the larger ones add selected texts. The 8 MB edition is sized to fit on F-RAM.

20.7 Licences

Openness, one of the principles in Timeless Computing, is given legal force by a **licence**: the terms under which the owner of a work's copyright allows others to use, copy, change and share it. Without a licence, a design or program that can be read may still not be legally copied or changed.

Open licences are of two broad kinds:

- **Permissive** licences, such as the MIT, BSD, Apache and Lone Dynamics Open licences, allow nearly any use, including in closed products, provided the original notice is kept. The Lone Dynamics Open License, used for most of Machdyne's designs and software, is similar to the one-clause BSD licence.
- **Copyleft** licences, such as the GNU General Public License (GPL), allow the same, on the condition that changed versions which are distributed are released under the same licence, with their source. The CERN Open Hardware Licence applies the same choice to hardware designs, in permissive and copyleft forms.

A work can also be **dedicated to the public domain**, with no conditions at all, as this book is under CC0, the Creative Com-

mons public domain dedication. Patents and trademarks are separate from copyright: a patent can restrict building an invention whatever its licence, and a trademark protects a product's name rather than its design. For the owner of a timeless computer, the practical test is that the licences of its designs and software must allow it to be repaired, changed and rebuilt by anyone, without permission.

20.8 People

The most durable store of knowledge is people. A machine that only one person understands depends on that person. Knowledge should therefore be taught, written down and shared within a community of people who use and maintain the same kinds of systems. Open designs make this possible: anyone can study them, and no permission is needed to repair, copy or improve them.

This book is intended to contribute to that work, and anyone may copy, print, translate and correct it.

20.9 A Checklist

- Know how much power the equipment needs, and have a way to supply it without the grid.
- Keep equipment cool, dry, clean and in an enclosure.
- Keep spares of the parts that wear out, and the tools and schematics to repair the rest.
- Keep at least three copies of important data, in plain formats, and check them.
- Keep the source, the tools and the instructions needed to rebuild the system, and test them.
- Keep a known good system image and a spare copy of the configuration flash.
- Keep an offline library of the information the work depends on.
- Learn how the machines work, and teach others.

20.10 Where to Begin

Timeless computing is not reached in a single step. Each step towards it is useful on its own, and none requires special equipment.

A reader new to the subject can begin with what is already at hand: write a short program in BASIC or Scheme, keep important writing in plain text files and make a verified copy of them, or study how one familiar device works inside. A reader with more experience can go further: build a small controller that does one job for years, read the source code of a system in daily use, add a peripheral to an FPGA design, or move a design to a different board.

The hardware, software and documentation described in this book are open, and so is the book itself. Each can be studied, corrected and improved by anyone who uses it, and each improvement that is shared helps it last. Corrections and suggestions for future editions are welcome through the Machdyne community and the book's source repository, listed in Sources.

A computer lasts when the people who depend on it understand how it works. This book was written to help make that understanding possible.

Part VII

Appendices

Appendix A

Glossary

Terms are explained in more detail in the chapter named in brackets.

- **ADC**: analog-to-digital converter; measures a voltage as a number. (Talking to the World)
- **address**: the number that identifies a location in memory or a device register. (The Computer)
- **address map**: the list of which device answers at which addresses. (The Computer)
- **algorithm**: a method for solving a problem in a finite number of exact steps. (Thinking in Programs)
- **ALU**: arithmetic logic unit; the part of a processor that calculates. (Digital Logic)
- **arbiter**: the circuit that decides which bus master uses the bus when several request it. (Systems on a Chip)
- **ASCII**: the 1963 standard that gives numbers to English letters, digits and symbols. (Information)
- **ASIC**: application-specific integrated circuit; a chip made for one design. (Inside an FPGA)
- **assembly language**: machine instructions written as readable text, one per line. (The Computer)
- **atomic**: (of an operation) performed as one step that cannot be interrupted or divided. (Operating Systems)
- **authenticated encryption**: encryption that also detects any change to the message. (Security)
- **bare metal**: running a program directly on hardware, with

no operating system. (Operating Systems)

- **baud rate**: the agreed speed of a serial line, in bits per second. (Talking to the World)
- **bipolar transistor**: a transistor controlled by a small current into its base. (Electricity)
- **bit**: a single binary digit, 0 or 1. (Information)
- **bitstream**: the file that configures an FPGA. (Inside an FPGA)
- **blitter**: a circuit that copies and fills blocks of pixels without the processor. (The Computer)
- **block RAM**: dedicated memory blocks inside an FPGA. (Inside an FPGA)
- **bootloader**: a program whose task is to load another program, such as the operating system. (Operating Systems)
- **brown-out**: a fall in supply voltage below what a circuit needs, which makes it misbehave. (Electricity)
- **bus**: the shared wires and rules that connect a processor to memory and devices. (The Computer)
- **bus master**: a device that can start transfers on a bus, such as a CPU or a DMA engine. (The Computer)
- **byte**: eight bits. (Information)
- **cache**: a small, fast memory holding copies of recently used data. (The Computer)
- **capacitance**: the amount of charge a capacitor stores per volt, measured in farads. (Electricity)
- **carry chain**: a fast dedicated path for carries between FPGA logic blocks. (Inside an FPGA)
- **cast**: in C, a type in parentheses that converts a value to that type. (C)
- **certificate**: a public key signed by an organization that others trust. (Security)
- **checksum**: a value computed from data, used to detect damage. (Memory and Storage)
- **clock**: a steady signal whose edges time a synchronous circuit. (Digital Logic)
- **clock domain**: the part of a design driven by one clock. (Inside an FPGA)
- **closure**: a function together with the variables it captured where it was made. (Scheme)
- **CMOS**: the circuit style, using pairs of complementary transistors, of nearly all digital chips. (Electricity)
- **combinational logic**: logic whose outputs depend only on

its present inputs. (Digital Logic)

- **compiler**: a program that translates source code into machine code ahead of time. (Thinking in Programs)
- **constraints file**: the file that assigns a design's signals to FPGA pins and states its clock speeds. (Describing Hardware)
- **context switch**: saving one process's state and restoring another's. (Operating Systems)
- **copyleft**: a licence condition that changed versions must be shared under the same licence. (Computing That Lasts)
- **CPU**: central processing unit; the processor. (The Computer)
- **CRC**: cyclic redundancy check; a strong kind of checksum. (Memory and Storage)
- **critical path**: the slowest path through a circuit, which limits its clock speed. (Digital Logic)
- **critical section**: code that must not be interrupted while it uses shared data. (Operating Systems)
- **cryptography**: the use of mathematics to protect information. (Security)
- **current**: the flow of electric charge, measured in amperes. (Electricity)
- **DAC**: digital-to-analog converter; produces a voltage from a number. (Talking to the World)
- **data sheet**: the manufacturer's description of a component and its limits. (Electricity)
- **deadlock**: a state in which two or more activities each wait for the other for ever. (Operating Systems)
- **debouncing**: ignoring the rapid flicker of a switch as it opens or closes. (Talking to the World)
- **decoder**: a circuit that turns a binary number into one active output. (Digital Logic)
- **DHCP**: the protocol that assigns IP addresses to machines automatically. (Communication)
- **diff**: a list of the lines that differ between two versions of a text. (Thinking in Programs)
- **digital signature**: a value made with a private key that proves who wrote a message. (Security)
- **DMA**: direct memory access; a device reading or writing memory without the processor. (The Computer)
- **DNS**: the domain name system, which translates names into IP addresses. (Communication)

- **DRAM**: dynamic RAM; dense memory that must be refreshed constantly. (Memory and Storage)
- **driver**: software that operates one kind of device. (Operating Systems)
- **DSP block**: a dedicated multiplier inside an FPGA. (Inside an FPGA)
- **EEPROM**: electrically erasable memory, rewritten a byte at a time. (Memory and Storage)
- **F-RAM**: ferroelectric RAM; non-volatile, byte-writable, extremely durable memory. (Memory and Storage)
- **fault**: an access refused by memory protection, reported to the kernel. (Operating Systems)
- **filesystem**: the rules that turn storage blocks into named files and folders. (Memory and Storage)
- **firmware**: software built into a piece of hardware, such as a microcontroller's program or a PC's start-up code. (Operating Systems)
- **fixed point**: fractions stored as whole numbers with an agreed scale. (Information)
- **flash**: non-volatile memory erased in blocks; it wears with use. (Memory and Storage)
- **flip-flop**: a one-bit memory that changes only at a clock edge. (Digital Logic)
- **floating point**: numbers stored as a fraction and an exponent, like scientific notation. (Information)
- **FPGA**: field-programmable gate array; a chip whose logic is set by a configuration file. (Inside an FPGA)
- **framebuffer**: the memory holding the image shown on the screen. (Talking to the World)
- **function**: a named, reusable piece of a program. (Thinking in Programs)
- **garbage collector**: the part of a language system that reclaims unused memory automatically. (Scheme)
- **gate**: a circuit computing a simple logical function such as AND or OR. (Digital Logic)
- **gateware**: the design loaded into an FPGA, as software is loaded into a processor. (Systems on a Chip)
- **glyph**: the drawn shape of a character in a font. (Information)
- **GPIO**: general-purpose input/output; pins controlled directly by software. (Talking to the World)
- **ground**: the reference point of a circuit, defined as 0 volts.

(Electricity)

- **hard processor**: a fixed processor built into an FPGA beside its logic. (Systems on a Chip)
- **hash**: a fingerprint of data; a cryptographic hash cannot practically be forged. (Memory and Storage)
- **HDL**: hardware description language, such as Verilog or VHDL. (Describing Hardware)
- **heap**: memory requested and released by a program while it runs. (C)
- **hexadecimal**: base 16, written with the digits 0 to 9 and A to F. (Information)
- **hold time**: how long a flip-flop's input must stay steady after the clock edge. (Digital Logic)
- **host**: the computer in charge of a bus or of attached devices. (Talking to the World, Small Computers)
- **hybrid scheme**: classical and post-quantum methods used together. (Security)
- **I2C**: a two-wire bus connecting many devices by address. (Talking to the World)
- **information-theoretic security**: security that holds whatever computing power an attacker has. (Security)
- **interpreter**: a program that carries out another program directly, without translating it into machine code. (Thinking in Programs)
- **interrupt**: a signal that makes the processor stop and handle an event. (The Computer)
- **IP**: the Internet Protocol, which addresses and routes packets between networks. (Communication)
- **ISA**: instruction set architecture; the instructions a processor understands. (The Computer)
- **journal**: a log of intended changes that lets a filesystem recover after a power cut. (Memory and Storage)
- **JTAG**: a standard debug and programming connection on chips and boards. (Describing Hardware)
- **kernel**: the core of an operating system. (Operating Systems)
- **latch**: a memory element made from gates with feedback. (Digital Logic)
- **least privilege**: giving each program only the access it needs. (Security)
- **LED**: light-emitting diode. (Electricity)
- **licence**: the terms on which a work may be used, copied,

changed and shared. (Computing That Lasts)
- **linear regulator**: a regulator that turns excess voltage into heat. (Electricity)
- **little-endian**: storing the least significant byte of a word at the lowest address. (The Computer)
- **lock**: a variable that lets only one activity at a time enter a critical section. (Operating Systems)
- **logic analyser**: an instrument that records many digital signals over time. (Electricity)
- **LUT**: look-up table; the small memory that implements logic in an FPGA. (Inside an FPGA)
- **memory-mapped I/O**: reaching devices by reading and writing addresses. (The Computer)
- **metastability**: a flip-flop caught between 0 and 1 by an input changing at the clock edge. (Digital Logic)
- **microcontroller**: a complete small computer on one chip. (Small Computers)
- **microkernel**: a kernel that runs drivers and services as separate processes. (Operating Systems)
- **MMOD**: a small socketed SPI memory module used in Machdyne computers. (Timeless Computing, Talking to the World)
- **MMU**: memory management unit; translates the addresses each program uses. (Operating Systems)
- **module**: in hardware, a small plug-in board; in Verilog, a unit of design. (Small Computers, Describing Hardware)
- **MPU**: memory protection unit; limits which memory a program may use. (Operating Systems)
- **multiplexer**: a circuit that chooses one of several inputs. (Digital Logic)
- **net**: in a schematic, a set of pins that are connected together. (Electricity)
- **netlist**: a list of logic elements and the connections between them. (Describing Hardware)
- **non-volatile**: keeping data without power. (Memory and Storage)
- **nonce**: a number used once with a key, so that no two messages are encrypted alike. (Security)
- **one-time pad**: encryption by XOR with a random key as long as the message, used once. (Security)
- **open-drain**: an output that can only pull a wire low. (Electricity)

- **oscilloscope**: an instrument that draws a voltage as it changes over time. (Electricity)
- **overflow**: a result too large for the bits that must hold it. (Information)
- **packet**: a block of data with a header, sent across a network. (Communication)
- **page**: a fixed-size block of memory, typically 4 KiB, translated as a unit by an MMU. (Operating Systems)
- **PC**: program counter; the register holding the address of the next instruction. (The Computer)
- **permissive licence**: a licence that allows nearly any use, provided the notice is kept. (Computing That Lasts)
- **pipeline**: a processor arrangement that overlaps the steps of successive instructions. (The Computer)
- **PLL**: phase-locked loop; makes new clock frequencies from an existing clock. (Inside an FPGA)
- **Pmod**: a common six- or twelve-pin expansion connector. (Talking to the World)
- **pointer**: a variable holding a memory address. (C)
- **port**: in networking, a number identifying a program on a machine; in Zeitlos, a named service. (Communication, The Machine)
- **port (Verilog)**: a named input or output of a module. (Describing Hardware)
- **POSIX**: the standard that describes Unix-like operating systems. (The Machine)
- **post-quantum**: (of a public-key method) believed secure against quantum computers. (Security)
- **private key**: the secret half of a public-key pair. (Security)
- **privilege level**: the authority of running code; only privileged code may change protection. (Operating Systems)
- **process**: a running program. (Operating Systems)
- **protocol**: an agreement on the meaning and order of messages. (Talking to the World)
- **public key**: the half of a key pair that can be given to anyone. (Security)
- **pull-up resistor**: a resistor that holds a wire high when nothing drives it. (Electricity)
- **PWM**: pulse-width modulation; varying the on-time of a fast pulse to control power. (Talking to the World)
- **race condition**: an error whose effect depends on the timing of two activities. (Operating Systems)

- **RAM**: random-access memory, in which any address can be reached as quickly as any other; usually volatile working memory. (Memory and Storage)
- **recursion**: a function that calls itself. (Thinking in Programs)
- **reference designator**: a part's label, such as R4 or U2, on the schematic and the board. (Electricity)
- **register**: a group of flip-flops holding a word; also a device's addressable control location. (Digital Logic, The Computer)
- **regression**: the return of a bug that had been fixed. (Thinking in Programs)
- **REPL**: read-eval-print loop; an interactive language prompt. (Operating Systems)
- **RISC-V**: an open instruction set architecture. (The Computer)
- **salt**: a random value stored with a password hash, so that guesses cannot be shared. (Security)
- **scheduler**: the part of an operating system that decides which process runs. (Operating Systems)
- **schematic**: a diagram of a circuit's components and their connections. (Electricity)
- **SDRAM**: synchronous DRAM, which transfers data in step with a clock. (Memory and Storage)
- **Sechs**: a six-contact standard for small programmable modules. (Small Computers)
- **sequential logic**: logic with memory, whose outputs depend on past inputs. (Digital Logic)
- **setup time**: how long a flip-flop's input must be steady before the clock edge. (Digital Logic)
- **shift register**: a chain of flip-flops that moves bits along at each clock edge. (Digital Logic)
- **SOC**: system on a chip; a complete computer on one chip. (Systems on a Chip)
- **soft processor**: a processor described in an HDL and built in FPGA logic; a softcore. (Systems on a Chip)
- **softcore**: a processor built from FPGA logic. (Systems on a Chip)
- **SPI**: a fast four-wire serial bus. (Talking to the World)
- **SRAM**: static RAM; fast memory that needs no refresh. (Memory and Storage)
- **stack**: memory used last-in, first-out, holding each function call's data. (The Computer)

- **state machine**: a register holding a state, and logic that chooses the next state. (Digital Logic)
- **survival computing**: computing designed to continue without cheap power, new parts or the Internet. (Timeless Computing)
- **switching regulator**: a regulator that switches rapidly and smooths the result, wasting little power. (Electricity)
- **symmetric encryption**: encryption in which one shared key encrypts and decrypts. (Security)
- **synthesis**: turning an HDL description into a netlist. (Describing Hardware)
- **system call**: a request from a program to the kernel. (Operating Systems)
- **tail call**: a call that is the last action of a function, which needs no new stack space. (Scheme)
- **TCP**: the protocol that provides reliable, ordered byte streams over IP. (Communication)
- **testbench**: a simulation-only module that tests a hardware design. (Describing Hardware)
- **threat model**: a statement of what is to be protected, from whom, and at what cost. (Security)
- **time constant**: R x C, the time a capacitor takes to charge to about 63 percent. (Electricity)
- **TLS**: the protocol that secures web and other connections. (Communication)
- **transistor**: an electrically controlled switch. (Electricity)
- **TRNG**: true random number generator; measures physical noise. (Security)
- **truth table**: a table listing a logic function's output for every input combination. (Digital Logic)
- **two's complement**: the standard way of representing negative binary numbers. (Information)
- **UART**: a device that sends and receives bytes on a serial line. (Talking to the World)
- **UDP**: a transport protocol that sends single messages with no guarantee of delivery. (Communication)
- **UEFI**: the firmware of current PCs, which replaced the BIOS. (Operating Systems)
- **USB**: universal serial bus. (Talking to the World)
- **UTF-8**: the standard encoding of Unicode text. (Information)
- **version control**: a system that records every version of a

project and its changes. (Thinking in Programs)

- **virtual address**: an address as a program uses it, before translation. (Operating Systems)
- **volatile**: losing data without power; in C, a value that may change outside the program's control. (Memory and Storage, C)
- **voltage**: the electrical pressure between two points, measured in volts. (Electricity)
- **watt-hour**: a unit of energy: one watt for one hour. (Computing That Lasts)
- **wear levelling**: spreading writes across a memory so that no part wears out early. (Memory and Storage)
- **word**: the natural unit of data for a processor, 32 bits in this book. (Information)
- **Zeitlos**: an open system on a chip and operating system for FPGA computers. (The Machine)
- **zfed**: the Zeitlos protocol for signed messages between independent machines. (The Machine)

Appendix B

A Short History

Computing builds on more than two thousand years of discovery. This appendix gives a brief account of how the ideas in this book came about, followed by a timeline of the main events.

B.1 From Electricity to Electronics

For most of history, electricity was a curiosity: a spark from rubbed amber, a twitch in a frog's leg. It became useful in the nineteenth century, once Alessandro Volta's battery provided a steady current and Georg Ohm, Michael Faraday and James Clerk Maxwell explained how current, magnetism and voltage are related. The first great application was communication. From the 1840s the electric telegraph carried messages across countries, and from the 1860s across oceans, coded as short and long pulses, and the telephone followed in the 1870s. Both are early examples of information carried as electrical signals, the subject of Information.

The vacuum tube, invented in the first decade of the twentieth century, made electronics possible. A small voltage on one part of a tube controls a larger current through it, so a tube can amplify a signal or act as a switch with no moving parts. Tubes made radio and television practical, and in the 1940s they made the first electronic computers possible.

B.2 Machines That Calculate

The idea of a general-purpose computing machine is older than electronics. In the 1830s Charles Babbage designed the Analytical Engine, a mechanical computer driven by instructions on punched cards, and Ada Lovelace described how it could be programmed. It was never built. In 1854 George Boole published an algebra of true and false, which in 1937 Claude Shannon showed could describe circuits built from switches: the basis of Digital Logic. In 1936 Alan Turing described an abstract machine that could carry out any computation that can be described precisely, and so defined what a computer is in principle.

The Second World War turned these ideas into machines. Konrad Zuse built programmable computers from telephone relays in Germany. In Britain, Colossus used vacuum tubes to help break enemy codes. In the United States, ENIAC, completed in 1945 with about 18,000 tubes, was programmed by rewiring panels of cables, which could take days.

B.3 The Stored Program

The decisive step came in the mid-1940s with the idea of keeping the program in the computer's memory, alongside its data. It was set out in a 1945 report on the EDVAC computer, written by John von Neumann from work with J. Presper Eckert and John Mauchly, and first run in practice by the Manchester Baby in 1948. A stored-program computer can be given a new task by loading new numbers into its memory, and can even create programs itself. Nearly every computer since has been built this way, and The Computer describes how.

Programming languages followed. Early programs were written in machine code or assembly language. In the late 1950s, languages such as FORTRAN and Lisp let programs be written in a form closer to mathematics, and translated into machine code by the computer itself. BASIC, created in 1964, was designed so that students in any subject could use a computer.

B.4 Smaller and Cheaper

The transistor, demonstrated in 1947, did the work of a vacuum tube while being smaller, more reliable and far more economical with power. From 1958 the integrated circuit placed many transistors on one piece of silicon, and the number that fitted on a chip doubled roughly every two years for half a century, a trend known as Moore's law. In 1971 a complete processor fitted on a single chip, the microprocessor.

Inexpensive microprocessors made the personal computer possible. Kits appeared in the mid-1970s, and by the early 1980s millions of homes and schools had small computers that started directly into BASIC. Their owners could see a program's effect the moment it was typed, and many learned to program by entering listings printed in magazines. These machines were slow by later standards, but they responded instantly, because there was little software between the keyboard and the screen. Graphical interfaces, with windows and a pointer, had been developed in research laboratories in the 1970s and reached ordinary users in the 1980s.

At the same time, Unix, developed at Bell Labs from 1969, and the C language in which it was rewritten from 1973, showed that an operating system could be small, written in a high-level language, and moved from one kind of computer to another. Many of the ideas in Operating Systems come from it.

B.5 Networks

Computers were first connected to each other in the late 1960s, when the ARPANET linked research institutions in the United States. Its packet-switching design, and from 1983 the TCP/IP protocols, became the basis of the Internet described in Communication. Outside the research world, people connected home computers over the telephone network to bulletin board systems from 1978 onwards. The World Wide Web, created at CERN between 1989 and 1991, made the Internet accessible to the general public.

B.6 Growth and Complexity

Each new generation of hardware was faster and had more memory, and software grew to use it. Operating systems, applications and web pages became many times larger and more capable, and also harder to understand, to secure and to keep running on older hardware. In 1995 the computer scientist Niklaus Wirth drew attention to the observation that software was becoming slower more rapidly than hardware was becoming faster. The problems described at the start of Timeless Computing are the long-term result.

B.7 Open Designs

Two developments made the alternative described in this book possible. The first was free and open-source software. From the 1980s, projects such as GNU and, from 1991, Linux showed that a complete system could be developed in public and shared with anyone.

The second was open hardware. FPGAs were introduced in 1985, but for thirty years they could be configured only with their manufacturers' closed tools. In 2010 work began at the University of California, Berkeley on RISC-V, an instruction set that anyone may implement without a licence. In 2015 Project IceStorm documented the configuration format of a family of FPGAs and released the first complete open-source toolchain for it, and open tools for other families followed. For the first time, every layer of a working computer, from its processor to its applications, could be designed, studied and changed with open tools. The chapters Inside an FPGA to The Machine describe what that makes possible.

B.8 Timeline

B.8.1 Discovering Electricity

- **About 600 BC.** Thales of Miletus is said to have noticed that rubbed amber attracts light objects. The Greek word for amber, *elektron*, gives electricity its name.
- **1600.** William Gilbert publishes a study of magnetism and static electricity, and coins the Latin word *electricus*.

- **1780s.** Luigi Galvani observes that electricity makes a frog's leg twitch.
- **1800.** Alessandro Volta builds the first battery, the voltaic pile, giving experimenters a steady current for the first time.
- **1820.** Hans Christian Oersted discovers that an electric current deflects a compass needle, linking electricity and magnetism.
- **1827.** Georg Ohm publishes the law relating voltage, current and resistance.
- **1831.** Michael Faraday discovers electromagnetic induction, the principle of the generator and the transformer.
- **1830s and 1840s.** The electric telegraph, including Samuel Morse's system and code, sends messages across countries at the speed of electricity.
- **1860s.** James Clerk Maxwell's equations unify electricity, magnetism and light.
- **1876.** Alexander Graham Bell patents the telephone.
- **1880s.** Electric power networks are built. Alternating current, developed by Nikola Tesla and others, wins out for long-distance transmission.
- **1897.** J. J. Thomson identifies the electron.

B.8.2 Electronics

- **1904.** John Ambrose Fleming invents the vacuum tube diode.
- **1906.** Lee de Forest adds a control grid, making the triode, which can amplify and switch signals.
- **1947.** John Bardeen and Walter Brattain, in William Shockley's group at Bell Labs, build the first transistor.
- **1958 and 1959.** Jack Kilby at Texas Instruments and Robert Noyce at Fairchild Semiconductor independently invent the integrated circuit.
- **1963.** CMOS logic is invented at Fairchild.
- **1965.** Gordon Moore observes that the number of transistors on a chip is doubling at a steady rate, a trend that held for decades.
- **1985.** Xilinx sells the first commercial FPGA.

B.8.3 Computers

- **1837.** Charles Babbage designs the Analytical Engine, a mechanical general-purpose computer that was never completed.
- **1843.** Ada Lovelace publishes notes on the Analytical Engine, including a method for computing Bernoulli numbers on it, often called the first computer program.
- **1854.** George Boole publishes the algebra of logic.
- **1936.** Alan Turing describes a theoretical machine that can carry out any computation, laying the foundation of computer science.
- **1937.** Claude Shannon shows that Boole's algebra can describe and design switching circuits.
- **1941.** Konrad Zuse completes the Z3, an electromechanical programmable computer.
- **1944.** Colossus, the first programmable electronic digital computer, designed by Tommy Flowers, breaks codes at Bletchley Park.
- **1945.** ENIAC, built by John Mauchly and J. Presper Eckert, is completed. It is reprogrammed by rewiring.
- **1948.** The Manchester Baby runs the first program stored in electronic memory.
- **1958.** John McCarthy creates Lisp.
- **1964.** John Kemeny and Thomas Kurtz create BASIC at Dartmouth College.
- **1969.** The ARPANET, ancestor of the Internet, links its first computers. Work on Unix begins at Bell Labs.
- **1971.** Intel's 4004 puts a whole processor on one chip.
- **1972.** Dennis Ritchie creates C.
- **1975.** Guy Steele and Gerald Sussman create Scheme. The Altair 8800 kit starts the personal computer era.
- **1977 to 1985.** Home and personal computers spread: the Apple II, the TRS-80, the BBC Micro, the Commodore 64 and the ZX Spectrum, all of which started up into BASIC, alongside the IBM PC (1981), the Macintosh (1984) and the Amiga (1985).
- **1978.** The first public bulletin board system goes online.
- **1983.** The ARPANET switches to TCP/IP.
- **1984.** Verilog is created.
- **1989 to 1991.** Tim Berners-Lee creates the World Wide Web.

- **1991.** Linus Torvalds releases Linux.

B.8.4 Open Hardware

- **2010.** Work on RISC-V begins at the University of California, Berkeley. Its specification is published openly and anyone may implement it.
- **2015.** Project IceStorm documents the configuration format of the Lattice iCE40 and releases the first complete open-source FPGA toolchain. Open tools for other FPGA families follow.

Appendix C

Tables

C.1 ASCII Characters

The ASCII characters from 32 to 127, with their codes in decimal and hexadecimal. Codes 32 to 126 are printable; 127, DEL, is a control character. Capital and lowercase letters differ by exactly 32 (hex 20), a single bit, and the digit characters 0 to 9 are 48 to 57, so subtracting 48 from a digit's code gives its value.

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
32	20	space	56	38	8	80	50	P	104	68	h
33	21	!	57	39	9	81	51	Q	105	69	i
34	22	"	58	3A	:	82	52	R	106	6A	j
35	23	#	59	3B	;	83	53	S	107	6B	k
36	24	$	60	3C	<	84	54	T	108	6C	l
37	25	%	61	3D	=	85	55	U	109	6D	m
38	26	&	62	3E	>	86	56	V	110	6E	n
39	27	'	63	3F	?	87	57	W	111	6F	o
40	28	(	64	40	@	88	58	X	112	70	p
41	29	)	65	41	A	89	59	Y	113	71	q
42	2A	*	66	42	B	90	5A	Z	114	72	r
43	2B	+	67	43	C	91	5B	[	115	73	s
44	2C	,	68	44	D	92	5C	\	116	74	t
45	2D	-	69	45	E	93	5D	]	117	75	u
46	2E	.	70	46	F	94	5E	^	118	76	v
47	2F	/	71	47	G	95	5F	_	119	77	w
48	30	0	72	48	H	96	60	`	120	78	x
49	31	1	73	49	I	97	61	a	121	79	y
50	32	2	74	4A	J	98	62	b	122	7A	z
51	33	3	75	4B	K	99	63	c	123	7B	{
52	34	4	76	4C	L	100	64	d	124	7C	|
53	35	5	77	4D	M	101	65	e	125	7D	}
54	36	6	78	4E	N	102	66	f	126	7E	~
55	37	7	79	4F	O	103	67	g	127	7F	DEL

C.2 Control Characters

The codes below 32 control a terminal or printer instead of printing. These are the ones still in common use:

Dec	Hex	Name	Keyboard	Use
0	00	NUL		ends a string in C
3	03	ETX	Ctrl+C	interrupt a running program
4	04	EOT	Ctrl+D	end of input
7	07	BEL	Ctrl+G	sound the bell
8	08	BS	Ctrl+H	backspace
9	09	HT	Ctrl+I, Tab	horizontal tab
10	0A	LF	Ctrl+J	line feed: new line
13	0D	CR	Ctrl+M, Enter	carriage return
27	1B	ESC	Esc	starts terminal escape sequences

C.3 Powers of Two

n	2 to the n	Common name
4	16	values in a nibble
8	256	values in a byte
10	1,024	1 KiB
12	4,096	a common flash sector
16	65,536	64 KiB; 16-bit values
20	1,048,576	1 MiB
24	16,777,216	16 MiB; 24-bit colours
30	1,073,741,824	1 GiB
32	4,294,967,296	4 GiB; 32-bit values

C.4 Unit Prefixes

Prefix	Symbol	Factor	Example
giga	G	1,000,000,000	1 GHz processor clock
mega	M	1,000,000	48 MHz
kilo	k	1,000	10k resistor
milli	m	0.001	5 mA through an LED
micro	u	0.000001	10 uA in sleep
nano	n	0.000000001	21 ns per clock cycle at 48 MHz

The proper symbol for micro is the Greek letter mu; a plain u is the usual substitute where it cannot be typed.

C.5 Electrical Formulas

```
V = I x R              Ohm's law: volts = amps x ohms
P = V x I              power: watts = volts x amps
E = P x t              energy: watt-hours = watts x hours
R = (Vsupply - Vled) / I     series resistor for an LED
f = 1 / T              frequency = 1 / period
```

C.6 Resistor Colour Code

Small resistors are marked with coloured bands. On a four-band resistor, the first two bands are digits, the third is the number of zeros to add, and the fourth, set apart from the others, is the tolerance.

Colour	Digit	Multiplier	Tolerance
black	0	1	
brown	1	10	1%
red	2	100	2%
orange	3	1k	
yellow	4	10k	
green	5	100k	0.5%
blue	6	1M	0.25%
violet	7	10M	0.1%
grey	8		
white	9		
gold		0.1	5%
silver		0.01	10%

Red, violet, brown and gold is 2, 7 and one zero: 270 ohms, within 5%. A five-band resistor has three digits before the multiplier.

C.7 Surface-Mount Markings

Surface-mount resistors are too small for colour bands and are marked with a short code:

Marking	Rule	Value
472	two digits, then the number of zeros	4.7k
103		10k
100		10 ohms
4701	three digits, then the number of zeros (1%)	4.7k
4R7	R marks the decimal point	4.7 ohms
R10		0.1 ohm
01C	EIA-96: an E96 value number and a letter	10k
0, 000	a zero-ohm link, used as a jumper	0

In the EIA-96 code, used on the smallest 1% resistors, the two digits number the 96 values of the E96 series, from 01 for 100 to

96 for 976, and the letter is a multiplier: Z 0.001, Y or R 0.01, X or S 0.1, A 1, B or H 10, C 100, D 1,000, E 10,000 and F 100,000. A table of the E96 series is needed to decode the number; 01C is 100 x 100.

Capacitors that are marked use three digits in picofarads, the third again the number of zeros: 104 is 100,000 pF, or 100 nF, and 105 is 1 uF. Small ceramic capacitors are usually not marked at all, and are measured or identified from the board's documentation. On a diode, the band marks the cathode; on a tantalum capacitor, the band marks the positive end. On a chip, a dot or notch marks pin 1, and the short code printed on top is explained in the marking section of its data sheet.

C.8 Standard Resistor Values

Resistors are made in series of values spaced so that each is a fixed ratio above the last. The E12 series, for 10% tolerance, has twelve values in each decade:

```
10   12   15   18   22   27   33   39   47   56   68   82
```

The same values repeat in every decade: 2.7 ohms, 27 ohms, 270 ohms, 2.7k, 27k and so on. The E24 series adds a value between each pair, for 5% tolerance.

C.9 Logic Level Thresholds

A digital input reads a voltage as 0 or 1 only when it is clearly low or clearly high; in between, the result is not defined. Typical limits for the common logic families:

Family	Supply	Low input at most	High input at least
5 V CMOS	5.0 V	1.5 V	3.5 V
5 V TTL	5.0 V	0.8 V	2.0 V
3.3 V LVCMOS	3.3 V	0.8 V	2.0 V
2.5 V LVCMOS	2.5 V	0.7 V	1.7 V
1.8 V LVCMOS	1.8 V	0.63 V	1.17 V

A 3.3 V output drives a 5 V TTL input correctly, but not always a 5 V CMOS input. A 5 V output connected to a 3.3 V input can

damage it unless the input is specified as 5 V tolerant. The data sheet of each part gives its exact limits.

Appendix D

Three Languages Side by Side

This appendix shows four small programs, each written in BASIC, Scheme and C. Each version computes the same result, and the differences show how each language expresses repetition, choice and functions. All of the programs fit within the 16-bit integers of Machdyne BASIC.

D.1 Greatest Common Divisor

Euclid's algorithm from Thinking in Programs finds the greatest common divisor of 1071 and 462, which is 21.

BASIC, with a loop built from `IF` and `GOTO`:

```
10 A = 1071: B = 462
20 IF B = 0 THEN GOTO 60
30 T = A MOD B
40 A = B: B = T
50 GOTO 20
60 PRINT A
```

Scheme, as a recursive function. The recursive call is a tail call, so it runs as a loop:

```
(define (gcd-of a b)
  (if (= b 0)
      a
```

```
      (gcd-of b (remainder a b))))

(gcd-of 1071 462)             ; => 21
```

C, with a `while` loop:

```
int gcd(int a, int b)
{
    while (b != 0) {
        int t = a % b;
        a = b;
        b = t;
    }
    return a;
}
```

D.2 Factorial

The factorial of 7, written 7!, is 1 x 2 x 3 x 4 x 5 x 6 x 7 = 5040. It is the largest factorial that fits in a 16-bit signed integer: 8! is 40320.

BASIC, multiplying in a `FOR` loop:

```
10 F = 1
20 FOR I = 1 TO 7
30 F = F * I
40 NEXT I
50 PRINT F
```

Scheme, following the definition directly: the factorial of 0 is 1, and the factorial of n is n times the factorial of n - 1. The multiplication happens after the recursive call returns, so this is not a tail call, and each call uses stack space.

```
(define (factorial n)
  (if (= n 0)
      1
      (* n (factorial (- n 1)))))

(factorial 7)                 ; => 5040
```

C, with a `for` loop:

```
int factorial(int n)
{
    int f = 1;
    for (int i = 1; i <= n; i++)
        f *= i;
    return f;
}
```

D.3 A Conversion Table

A table of temperatures from 0 to 100 degrees Celsius in steps of 20, with their equivalents in Fahrenheit: F = C x 9 / 5 + 32. Each version prints six rows of two numbers, from 0 and 32 to 100 and 212; BASIC places the second number in the next print zone.

BASIC, where `STEP` sets the increment and the comma moves to the next print zone:

```
10 FOR C = 0 TO 100 STEP 20
20 PRINT C, C * 9 / 5 + 32
30 NEXT C
```

Scheme, with a procedure that prints one row and then calls itself for the next:

```
(define (table c)
  (if (<= c 100)
      (begin
        (display c)
        (display " ")
        (display (+ (/ (* c 9) 5) 32))
        (display "\n")
        (table (+ c 20)))))

(table 0)
```

C, with `printf`, where each `%d` is replaced by the next number:

```
    for (int c = 0; c <= 100; c += 20)
        printf("%d %d\n", c, c * 9 / 5 + 32);
```

In BASIC and C, the multiplication is done before the division, so the integer division loses nothing. Written as `C / 5 * 9`, the result would be rounded down for any value of C that is not a multiple of 5.

D.4 The Collatz Sequence

The sequence starts from a number. An even number is halved; an odd one is multiplied by 3 and increased by 1. This repeats until the number is 1. Every starting number ever tried reaches 1 eventually, although no one has proved that all of them do. Starting from 27, it takes 111 steps, and the number rises as high as 9232 on the way.

BASIC, with a decision inside the loop:

```
10 N = 27: S = 0
20 IF N = 1 THEN GOTO 60
30 IF N MOD 2 = 0 THEN N = N / 2 ELSE N = 3 * N + 1
40 S = S + 1
50 GOTO 20
60 PRINT S
```

Scheme, with `cond` choosing among three cases, and the step count carried as an argument:

```
(define (collatz n steps)
  (cond ((= n 1) steps)
        ((even? n) (collatz (/ n 2) (+ steps 1)))
        (else (collatz (+ (* 3 n) 1) (+ steps 1)))))

(collatz 27 0)                ; => 111
```

C, with `if` and `else` inside a `while` loop:

```
int collatz(int n)
{
    int steps = 0;
    while (n != 1) {
        if (n % 2 == 0)
            n = n / 2;
        else
            n = 3 * n + 1;
        steps++;
    }
    return steps;
}
```

D.5 Common Constructs

Construct	BASIC	Scheme	C
comment	REM note	; note	/* note */
assign	A = 5	(set! a 5)	a = 5;
arithmetic	A * (B + 1)	(* a (+ b 1))	a * (b + 1)
remainder	A MOD B	(remainder a b)	a % b
equality	A = B	(= a b)	a == b
choice	IF A > 0 THEN …	(if (> a 0) x y)	if (a > 0) …
loop	FOR I = 1 TO 10	(let loop ((i 1)) …)	for (i = 1; …)
function	GOSUB 100	(define (f x) …)	int f(int x)
output	PRINT A	(display a)	printf(“%d”, a);
truth	-1, 0	#t, #f	non-zero, 0

Appendix E

Language Reference

This appendix summarizes the three programming languages and the hardware description language used in this book. Each summary covers the language as it is defined by its standard or reference document, which changes rarely or not at all, and leaves out the libraries and interfaces of particular systems, which change often.

E.1 Machdyne BASIC 1

Machdyne BASIC 1 is frozen: later versions may add to it but must not change the meaning of any BASIC 1 program. Every value is a 16-bit signed integer, from -32768 to 32767, and arithmetic wraps around at those limits. The variables are the 26 letters `A` to `Z`.

E.1.1 BASIC Statements

Statement	Effect
LET v = e, or v = e	assign the value of e to variable v
PRINT items	print; a semicolon joins, a comma tabs
INPUT v, INPUT “text”; v	read a number from the console
IF e THEN … ELSE …	run the rest of the line if e is not 0
GOTO n	continue at line n
GOSUB n, RETURN	call a subroutine at line n, and return
FOR v = a TO b STEP s	start a loop; the body runs at least once

Statement	Effect
NEXT v	end of the loop body
END	stop the program
REM text	a comment, to the end of the line
SLEEP e	wait e seconds
WAIT e	wait e milliseconds
PINS m, m, m, m	declare the use of pins 1 to 4
OUT p, e	set output pin p low (0) or high (not 0)
LED e	turn the LED off (0) or on (not 0)
I2C a, v	write byte v to I2C device a
I2C a, r, v	write byte v to register r of device a
REG n, e	set program register n (0 to 15)
OPEN “f” FOR m AS #1	open file f for INPUT, OUTPUT or APPEND
PRINT #1, items	write to the open file
INPUT #1, v	read the next number from the open file
CLOSE #1	close the open file

A colon separates several statements on one line.

E.1.2 BASIC Limits

Every implementation provides at least these:

Limit	At least
program memory	1,024 bytes, tokenized
elements per line	64
active FOR loops	6
active GOSUB calls	8
variables	26, A to Z
program registers	16, each 0 to 255
open files	1

E.1.3 BASIC Functions

Function	Value
IN(p)	digital pin p, 0 or 1
ADC(p)	analog input p, 0 to 1023
I2CR(a)	a byte from I2C device a; -1 if it does not answer
I2CR(a, r)	a byte from register r of I2C device a
REG(n)	program register n, 0 to 255
EOF(1)	-1 at the end of the open input file, otherwise 0

E.1.4 BASIC Operators

From highest to lowest precedence. Operators of equal precedence are evaluated from left to right.

Precedence	Operators	Meaning
1	-	negation
2	* / MOD	multiply, divide, remainder
3	+ -	add, subtract
4	= == <> < > <= >=	comparisons: -1 true, 0 false
5	NOT	bitwise not
6	AND	bitwise and
7	OR	bitwise or

Division truncates towards zero, so `-7 / 2` is -3. `MOD` takes the sign of its left operand.

E.1.5 BASIC Pin Modes

Mode	Meaning	Pins
-	not used	any
IN	digital input	any
PP	push-pull output	any
OD	open-drain output	any
AIN	analog input	3, 4
I2C	I2C bus: 3 is the clock, 4 the data	3 and 4
UART	reserved for a later version	3 and 4
NET	left to the system, for the bus it is on	1 and 2

`RUN` starts every program with the declaration `PINS NET, NET, -, -`.

E.1.6 BASIC Commands

Command	Effect
RUN	run the program from its lowest line
LIST	show the program
NEW	erase the program
SAVE name	save the program as a text file
LOAD name	replace the program with a saved one
TYPE name	show the contents of a file
DIR	list the files
DEL name	delete a file
FORMAT YES	erase all files
HELP	list the commands and keywords

File names have up to eight letters, digits, `_` or `-`, and an optional extension of up to three; a name without an extension is given `.BAS`. A system that starts a program by itself runs `BOOT.BAS`. Ctrl+C stops a running program.

E.1.7 BASIC Error Messages

Message	Cause
SYNTAX ERROR	not a valid line or command
TOO LONG	too many elements in a line
TOO BIG	a number above 32767
NO LINE	a jump to a missing line
DIV BY 0	division or MOD by zero
NO MEMORY	the program does not fit
NO FOR, NO GOSUB	NEXT or RETURN without its start
TOO DEEP	too many loops or calls active
PIN NOT DECLARED	a use that PINS did not declare
BAD PINS, ON A BUS	a pin declaration or use not allowed
OUT OF RANGE	a pin or register number out of range
I2C ERROR	an I2C device did not answer
BAD NAME, NOT FOUND	an invalid or missing file name
DISK FULL, DIR FULL	no room for the file
BAD FILE #	a file number other than 1
FILE OPEN, NOT OPEN	a file is open, or none is

Message	Cause
END OF FILE	reading past the end of the file
NOT FORMATTED	the storage is not formatted
DAMAGED, I/O ERROR	the storage cannot be read
NOT SUPPORTED	the hardware is not present
BREAK	stopped with Ctrl+C

E.2 Scheme Reference

The Scheme in this book follows the Revised(4) Report on the Algorithmic Language Scheme (R4RS). Machdyne Scheme implements a documented subset of it. It leaves out hygienic macros, continuations that can be resumed after they have returned, separate exact and inexact numbers (all numbers are floating point), and the procedures for ports and files, which each system provides in its own way. The tables list the standard forms and procedures that it implements.

E.2.1 Scheme Special Forms

```
(quote x)  'x                  x itself, not evaluated
(lambda (args) body)           a new procedure
(define name value)            give a name to a value
(define (name args) body)      define a procedure
(set! name value)              change the value of a variable
(if test then else)            choose one of two expressions
(cond (test e ...) ...         the first clause whose test is
      (else e ...))              true
(case key ((v ...) e ...) ...) the clause that lists key
(and e ...)  (or e ...)        stop at the first false, or the
                                 first true, value
(begin e ...)                  in order; the value of the last
(let ((name value) ...) body)  local names, bound together
(let* ...)                     bound one after another
(letrec ...)                   bound for mutual recursion
(let loop ((var init) ...)     named let: a loop that calls
  body)                          loop to repeat
(do ((var init step) ...)      an iteration that ends when
    (test result) body)          test is true
(delay e)                      a promise, run by force
`x  ,x  ,@x                    quasiquote, unquote, splice
```

E.2.2 Scheme Standard Procedures

Kind	Procedures
equality	eq? eqv? equal? not
types	boolean? symbol? number? char? string? vector? pair? null? procedure? list?
pairs	cons car cdr set-car! set-cdr! caar cadr … cddddr
lists	list length append reverse list-tail list-ref memq memv member assq assv assoc map for-each
numbers	+ - * / = < > <= >= zero? positive? negative? odd? even? max min abs quotient remainder modulo gcd lcm floor ceiling truncate round sqrt expt number->string string->number
characters	char=? char<? char>? char<=? char>=? char-alphabetic? char-numeric? char-whitespace? char-upper-case? char-lower-case? char->integer integer->char char-upcase char-downcase
strings	make-string string string-length string-ref string-set! string=? string<? string>? string<=? string>=? substring string-append string-copy string-fill! string->symbol symbol->string
vectors	make-vector vector vector-length vector-ref vector-set! vector->list list->vector vector-fill!
control	apply force call-with-current-continuation (call/cc)
in and out	display write read

Machdyne Scheme also provides `dynamic-wind` and `eval`, which were added to the standard in its next revision, R5RS.

Only `#f` is false. Procedures whose names end in `?` return true or false, and those whose names end in `!` change data in place.

E.3 C Reference

The C in this book follows the 1999 standard, C99.

E.3.1 C Keywords

C99 has 37 reserved words:

```
auto        break       case        char        const       continue
default     do          double      else        enum        extern
float       for         goto        if          inline      int
long        register    restrict    return      short       signed
sizeof      static      struct      switch      typedef     union
unsigned    void        volatile    while       _Bool       _Complex
_Imaginary
```

E.3.2 C Operators

From highest to lowest precedence. Operators on the same line have equal precedence; the last column gives the order in which a chain of them is grouped.

```
()  []  ->  .  x++  x--          call, index, member     left
++x --x  +  -  !  ~  *  &        prefix, unary           right
  sizeof  (type)                 size, cast              right
*  /  %                         multiply, divide        left
+  -                            add, subtract           left
<<  >>                          shift                   left
<  <=  >  >=                    compare                 left
==  !=                          equal, not equal        left
&                               bitwise and             left
^                               bitwise exclusive or    left
|                               bitwise or              left
&&                              logical and             left
||                              logical or              left
?:                              conditional             right
=  +=  -=  *=  /=  %=           assignment              right
  <<=  >>=  &=  ^=  |=
,                               sequence                left
```

Two rules prevent many mistakes: `&`, `^` and `|` bind more loosely than `==`, so `(flags & MASK) == 0` needs its parentheses; and `=` assigns while `==` compares.

E.3.3 C Standard Headers

Header	Provides
stdio.h	input and output: printf, scanf, fopen, fgets
stdlib.h	malloc, free, exit, atoi, strtol, qsort, abs
string.h	strlen, strcmp, strcpy, memcpy, memset
ctype.h	isdigit, isalpha, isspace, toupper
math.h	sqrt, sin, cos, pow, floor

Header	Provides
stdint.h	exact-width integers: uint8_t, int16_t, uint32_t
stdbool.h	bool, true, false
stddef.h	size_t, NULL, offsetof
limits.h	INT_MAX, CHAR_BIT and other limits
assert.h	assert
time.h	time, clock

E.3.4 printf Conversions

Conversion	Prints	Example	Output
%d	a signed integer	(“%d”, -42)	-42
%u	an unsigned integer	(“%u”, 42u)	42
%x, %X	hexadecimal	(“%x”, 255)	ff
%o	octal	(“%o”, 8)	10
%c	a character	(“%c”, 65)	A
%s	a string	(“%s”, “hi”)	hi
%f	a floating-point number	(“%.2f”, 3.14159)	3.14
%p	a pointer	(“%p”, ptr)	address
%%	a percent sign	(“%%”)	%

A number between `%` and the letter sets a minimum width, `0` pads with zeros, and `.n` sets the precision: `%04X` prints 255 as `00FF`.

E.3.5 C Escape Sequences

```
\n    line feed              \\    backslash
\r    carriage return        \'    single quote
\t    tab                    \"    double quote
\0    the zero byte          \x41  the byte 0x41, "A"
```

E.4 Verilog Reference

The Verilog in this book follows the 2005 standard, IEEE 1364-2005, and uses only the parts that can be synthesized into hardware, together with those needed for testbenches.

E.4.1 Verilog Constructs

Construct	Meaning
module name (ports); … endmodule	a unit of design
input, output, inout	port directions
wire, reg	a connection; a value set in always
[7:0]	an 8-bit vector, bit 7 the highest
parameter, localparam	a constant, settable or local
assign w = expr;	combinational logic
always @(posedge clk)	logic clocked on the rising edge
always @*	combinational logic in a block
begin … end	group statements
if … else, case … endcase	choices inside an always block
<=	non-blocking: for clocked logic
=	blocking: for combinational logic
initial	testbench code, run once at start
#n	in a testbench, wait n time units
$display, $finish	in a testbench, print and stop

E.4.2 Verilog Numbers and Operators

```
8'd200  4'b1010  16'hFF   width, base (d, b, h, o), value
{a, b}                    concatenation
{4{a}}                    repetition: a four times
x[3]  x[7:4]              a bit, a range of bits
&  |  ^  ~                bitwise and, or, exclusive or, not
&&  ||  !                 logical and, or, not
&x  |x  ^x                reduction: and, or, xor of all bits
==  !=  <  >  <=  >=      comparisons
+  -  *  <<  >>           arithmetic and shifts
c ? a : b                 choice: a multiplexer
```

Appendix F

RISC-V Reference

This appendix lists the RV32I base instruction set and the M extension, which together form RV32IM, the instruction set of the processors described in this book. The RISC-V specifications are frozen: an instruction, once ratified, keeps its meaning.

F.1 Registers

RV32I has 32 registers of 32 bits, `x0` to `x31`, and the program counter, `pc`. Register `x0` always reads as zero, and writes to it are discarded. The other registers are alike to the hardware; their roles come from the **calling convention**, the agreement that lets separately compiled functions call each other. Assemblers accept both the plain names and the convention's names.

Register	Name	Role	Kept by callee
x0	zero	always zero	-
x1	ra	return address	no
x2	sp	stack pointer	yes
x3	gp	global pointer	-
x4	tp	thread pointer	-
x5-x7	t0-t2	temporaries	no
x8	s0, fp	saved register, frame pointer	yes
x9	s1	saved register	yes
x10-x11	a0-a1	arguments, return values	no
x12-x17	a2-a7	arguments	no

Register	Name	Role	Kept by callee
x18-x27	s2-s11	saved registers	yes
x28-x31	t3-t6	temporaries	no

A function receives its first eight arguments in `a0` to `a7` and returns its result in `a0`. It may change the temporaries and argument registers freely, but must restore the saved registers and `sp` before it returns. The stack grows downward, towards lower addresses, and `sp` is kept a multiple of 16.

F.2 Instruction Formats

Every instruction is 32 bits. The six formats place the registers in the same positions wherever they occur, and the immediate is scattered in the B and J formats so that its bits, too, stay in fixed places:

```
     31        25 24    20 19    15 14  12 11       7 6       0
R    funct7       rs2      rs1      funct3 rd          opcode
I    imm[11:0]             rs1      funct3 rd          opcode
S    imm[11:5]    rs2      rs1      funct3 imm[4:0]    opcode
B    imm[12,10:5] rs2      rs1      funct3 imm[4:1,11] opcode
U    imm[31:12]                             rd          opcode
J    imm[20,10:1,11,19:12]                  rd          opcode
```

Immediates are sign-extended: the highest bit of the immediate is copied into all the bits above it, so a 12-bit immediate holds -2048 to 2047. Branch and jump offsets count in bytes and are always even.

F.3 RV32I

In the descriptions, `rd` is the destination register, `rs1` and `rs2` the source registers, and `imm` the immediate. "Unsigned" means the values are compared as numbers from 0 to 4,294,967,295 instead of signed numbers.

F.3.1 Arithmetic and Logic

Instruction	Format	Effect
add rd, rs1, rs2	R	rd = rs1 + rs2
sub rd, rs1, rs2	R	rd = rs1 - rs2
and rd, rs1, rs2	R	rd = rs1 AND rs2, bit by bit
or rd, rs1, rs2	R	rd = rs1 OR rs2
xor rd, rs1, rs2	R	rd = rs1 XOR rs2
sll rd, rs1, rs2	R	shift left by the low 5 bits of rs2
srl rd, rs1, rs2	R	shift right, filling with zeros
sra rd, rs1, rs2	R	shift right, copying the sign bit
slt rd, rs1, rs2	R	rd = 1 if rs1 < rs2, else 0
sltu rd, rs1, rs2	R	as slt, unsigned
addi rd, rs1, imm	I	rd = rs1 + imm
andi, ori, xori	I	and, or, xor with imm
slli, srli, srai	I	shift by a constant, 0 to 31
slti, sltiu	I	set if less than imm
lui rd, imm	U	rd = imm shifted left by 12
auipc rd, imm	U	rd = pc + (imm shifted left by 12)

There is no subtract-immediate instruction: `addi` with a negative immediate does the same. Arithmetic wraps around on overflow and sets no flags; a program that needs to detect overflow compares the result.

F.3.2 Loads and Stores

Instruction	Format	Effect
lw rd, imm(rs1)	I	load the 32-bit word at rs1 + imm
lh, lhu rd, imm(rs1)	I	load 16 bits, sign- or zero-extended
lb, lbu rd, imm(rs1)	I	load 8 bits, sign- or zero-extended
sw rs2, imm(rs1)	S	store the word in rs2 at rs1 + imm
sh rs2, imm(rs1)	S	store the low 16 bits of rs2
sb rs2, imm(rs1)	S	store the low 8 bits of rs2

Memory is addressed in bytes and is little-endian: the lowest byte of a word is stored at the lowest address. A word should be at an address that is a multiple of 4, and a halfword at a multiple of 2; a processor may refuse other accesses or perform them slowly.

F.3.3 Branches and Jumps

Instruction	Format	Effect
beq rs1, rs2, label	B	branch if rs1 = rs2
bne rs1, rs2, label	B	branch if rs1 is not rs2
blt, bge	B	branch if less, or greater or equal
bltu, bgeu	B	as blt and bge, unsigned
jal rd, label	J	rd = pc + 4, then jump to label
jalr rd, imm(rs1)	I	rd = pc + 4, then jump to rs1 + imm

A branch reaches 4 KiB in either direction, `jal` 1 MiB. `jalr` with an address built by `lui` or `auipc` reaches anywhere.

F.3.4 System

Instruction	Effect
fence	order memory accesses as seen by other devices
ecall	call the operating system or other environment
ebreak	stop for a debugger

The instructions that read and write the control and status registers, which hold the processor's interrupt and trap settings, form a separate extension, Zicsr. Their use depends on the processor.

F.4 The M Extension

Instruction	Effect
mul rd, rs1, rs2	the low 32 bits of rs1 x rs2
mulh rd, rs1, rs2	the high 32 bits, both signed
mulhu rd, rs1, rs2	the high 32 bits, both unsigned
mulhsu rd, rs1, rs2	the high 32 bits, rs1 signed and rs2 unsigned
div rd, rs1, rs2	rs1 / rs2, signed, rounded towards zero
divu rd, rs1, rs2	rs1 / rs2, unsigned
rem rd, rs1, rs2	the remainder of div, with the sign of rs1
remu rd, rs1, rs2	the remainder of divu

Division by zero does not stop the program: `div` and `divu` give -1 (all bits set), and `rem` and `remu` give rs1.

F.5 Pseudo-instructions

Assemblers accept these common forms and translate them into one or two real instructions.

Pseudo-instruction	Meaning	Real instructions
nop	do nothing	addi x0, x0, 0
li rd, value	load any 32-bit constant	lui and addi, or addi
la rd, label	load an address	auipc and addi
mv rd, rs	copy a register	addi rd, rs, 0
not rd, rs	invert the bits	xori rd, rs, -1
neg rd, rs	negate	sub rd, x0, rs
seqz rd, rs	rd = 1 if rs is 0	sltiu rd, rs, 1
beqz, bnez rs, label	branch if zero, not zero	beq, bne with x0
bgt, ble rs, rt, label	the other comparisons	blt, bge swapped
j label	jump	jal x0, label
call label	call a function	auipc and jalr ra
ret	return from a function	jalr x0, 0(ra)

Appendix G

Connector Pinouts

This appendix lists the pins of connectors in common use with small computers. Pin 1 is usually marked on a board by a square pad, a dot, an arrow or a small number. Signal names ending in N are active when low; a plus or minus sign marks one half of a differential pair. Before a connection is made from these tables, the voltages are checked as described in Connecting a Device.

G.1 Pmod

A Pmod connector has one row of six pins, or two rows of six. Pins 5 and 6 are ground and the supply, usually 3.3 V; on a 12-pin connector, so are pins 11 and 12. Pins 7 to 10 repeat the arrangement of pins 1 to 4.

```
 6-pin:   1   2   3   4  GND VCC
12-pin:   1   2   3   4  GND VCC
          7   8   9  10  GND VCC
```

The Pmod specification defines standard uses of pins 1 to 4, so that a module and a host can agree without further documentation:

Type	Pin 1	Pin 2	Pin 3	Pin 4
1, GPIO	GPIO	GPIO	GPIO	GPIO
2, SPI	CS	MOSI	MISO	SCK
3, UART	CTS	TXD	RXD	RTS
5, motors	DIR1	EN1	DIR2	EN2
6, I2C	INT	RESET	SCL	SDA
7, I2S	LRCLK	DAC data	ADC data	BCLK

Many modules do not follow the standard types exactly, so the module's own documentation decides.

G.2 Sechs

The six contacts of a Sechs module, described in Small Computers, match one row of a 6-pin Pmod connector:

Pin	Name	Use
1	A	shared I2C bus clock (the module is a slave)
2	B	shared I2C bus data
3	C	the module's own I/O; as a UART, the module receives on C
4	D	the module's own I/O; as a UART, the module transmits on D
5	GND	ground
6	3V3	3.3 V, supplied by the carrier

The carrier provides the pull-up resistors for A and B, of 2.2k to 10k; modules never do. Anything the carrier drives onto C or D passes through a series resistor of about 1k.

G.3 Zwölf

Zwölf modules, which comply with Sechs when they run Sechs firmware such as Machdyne BASIC, have twelve contacts in two rows: the six of Sechs, four more for the module's own uses, a reset and a write protect. Viewed from the top:

```
 1   2   3   4   5   6
 7   8   9  10  11  12
```

Pin	Name	Use
1	A	shared I2C bus clock (slave)
2	B	shared I2C bus data
3	C	local I2C clock (master), or I/O
4	D	local I2C data, or I/O
5	GND	ground
6	3V3	3.3 V
7	E	module-specific
8	F	module-specific
9	G	module-specific
10	H	module-specific, often an interrupt or the programming pin
11	RESETN	reset; left open or pulled low
12	WPN	write protect; left open or pulled low, never driven high

G.4 Werkzeug

Werkzeug, Machdyne's USB multi-tool, is built around an RP2040 microcontroller. Its 20-pin I/O header, numbered as in its documentation:

```
 1  2  3  4  5  6  7  8  9 10
11 12 13 14 15 16 17 18 19 20
```

Pin	Signal	Pin	Signal
1	GPIO0	11	GPIO8
2	GPIO1	12	GPIO9
3	GPIO2	13	GPIO10
4	GPIO3	14	GPIO11
5	GPIO4	15	GPIO26, ADC0
6	GPIO5	16	GPIO27, ADC1
7	GPIO6	17	GPIO28, ADC2
8	GPIO7	18	GPIO29, ADC3
9	GND	19	GND
10	3V3	20	3V3

Its 12-pin Pmod connector carries GPIO19, 17, 15 and 13 on pins 1 to 4, and GPIO18, 16, 14 and 12 on pins 7 to 10. Its two LEDs, green on GPIO20 and red on GPIO21, light when their pins are low.

G.5 Raspberry Pi GPIO

The 40-pin header of the Raspberry Pi, used on most of its models since 2014. Pin 1 has a square pad, and the odd-numbered pins form the row nearer the centre of the board. Signals are at 3.3 V, and the pins are not 5 V tolerant.

Pin	Signal	Pin	Signal
1	3.3 V	2	5 V
3	GPIO2, I2C SDA	4	5 V
5	GPIO3, I2C SCL	6	GND
7	GPIO4	8	GPIO14, UART TX
9	GND	10	GPIO15, UART RX
11	GPIO17	12	GPIO18, PWM0
13	GPIO27	14	GND
15	GPIO22	16	GPIO23
17	3.3 V	18	GPIO24
19	GPIO10, SPI MOSI	20	GND
21	GPIO9, SPI MISO	22	GPIO25
23	GPIO11, SPI SCK	24	GPIO8, SPI CE0
25	GND	26	GPIO7, SPI CE1
27	ID_SD	28	ID_SC
29	GPIO5	30	GND
31	GPIO6	32	GPIO12, PWM0
33	GPIO13, PWM1	34	GND
35	GPIO19	36	GPIO16
37	GPIO26	38	GPIO20
39	GND	40	GPIO21

Pins 27 and 28 are reserved for the memory chip that identifies an add-on board.

G.6 Serial Connectors

A **TTL serial** cable or adapter, of the kind used for a serial console, usually has a six-pin connector in this order, with signals at 3.3 V or 5 V:

Pin	Signal	Wire colour, where used
1	GND	black
2	CTS	brown
3	VCC	red
4	TXD	orange
5	RXD	yellow
6	RTS	green

TXD is the adapter's output and connects to the computer's receive pin; RXD connects to its transmit pin.

An **RS-232** serial port on a 9-pin D connector, as on older computers, swings to plus and minus 12 V and must not be connected directly to a 3.3 V pin:

Pin	Signal	Pin	Signal
1	DCD, carrier detect	6	DSR, set ready
2	RXD, receive	7	RTS, request to send
3	TXD, transmit	8	CTS, clear to send
4	DTR, terminal ready	9	RI, ring indicator
5	GND		

These are the pins of a computer's port. A cable between two computers, a **null modem** cable, crosses pins 2 and 3.

G.7 USB Connectors

Pin	USB-A, USB-B	Micro-B	Wire colour
1	VBUS, 5 V	VBUS	red
2	D-	D-	white
3	D+	D+	green
4	GND	ID	
5		GND	black

D+ and D- are a differential pair carrying data at USB 2.0 speeds. The ID pin of a micro-B plug is grounded in an adapter cable that makes the device act as a host.

A **USB-C** connector has 24 pins, in two rows that mirror each other, so that the plug fits either way up:

Pins	Signal	Use
A1, A12, B1, B12	GND	ground
A4, A9, B4, B9	VBUS	power, 5 V unless more is agreed
A6, B6	D+	USB 2.0 data
A7, B7	D-	USB 2.0 data
A5, B5	CC1, CC2	configuration: orientation, power
A8, B8	SBU1, 2	sideband use, for other modes
A2, A3, B2, B3	TX pairs	high-speed data, USB 3 and others
A10, A11, B10, B11	RX pairs	high-speed data, USB 3 and others

A device that is powered from USB-C must connect each of its CC pins to ground through a 5.1k resistor. Without them, a USB-C power supply does not turn on its 5 V output, and a simple board that works from a USB-A port receives no power from a USB-C charger.

G.8 Ethernet Cables

An Ethernet cable has eight wires in four twisted pairs, ending in RJ45 plugs. Two standards give the order of the colours; a cable with the same standard at both ends is a straight cable, which is the usual kind.

Pin	T568B	T568A	10 and 100 Mbit/s
1	white/orange	white/green	transmit +
2	orange	green	transmit -
3	white/green	white/orange	receive +
4	blue	blue	not used
5	white/blue	white/blue	not used
6	green	orange	receive -
7	white/brown	white/brown	not used
8	brown	brown	not used

The transmit and receive labels are those of a computer. Gigabit Ethernet uses all four pairs in both directions. Most current equipment detects a crossed cable and adapts to it.

G.9 VGA Connector

A VGA connector is a 15-pin D connector in three rows. The colour signals are analog voltages from 0 to 0.7 V into 75 ohms; the sync signals are digital.

Pin	Signal	Pin	Signal
1	red	9	5 V, for DDC
2	green	10	GND, sync return
3	blue	11	reserved
4	reserved	12	DDC data (SDA)
5	GND	13	horizontal sync
6	red return	14	vertical sync
7	green return	15	DDC clock (SCL)
8	blue return		

DDC is an I2C bus on which the monitor reports its supported modes, from a small memory chip.

G.10 HDMI Connector

A standard HDMI connector, type A, has 19 pins. DVI carries the same digital video signals on a larger connector.

Pin	Signal	Pin	Signal
1	TMDS data 2+	11	TMDS clock shield
2	TMDS data 2 shield	12	TMDS clock-
3	TMDS data 2-	13	CEC
4	TMDS data 1+	14	reserved
5	TMDS data 1 shield	15	DDC clock (SCL)
6	TMDS data 1-	16	DDC data (SDA)
7	TMDS data 0+	17	DDC and CEC ground
8	TMDS data 0 shield	18	5 V
9	TMDS data 0-	19	hot plug detect
10	TMDS clock+		

The four TMDS pairs carry the picture as fast serial streams, one pair for each colour and one for the clock. DDC works as on VGA. CEC lets devices control each other, and hot plug detect tells the source that a display is connected.

G.11 SD Cards

A microSD card has eight contacts. It can be used in its own four-bit mode or as an SPI device, which needs fewer signals, as described in Talking to the World:

Pin	SD mode	SPI mode
1	DAT2	not used
2	CD, DAT3	CS
3	CMD	MOSI, data in
4	VDD, 3.3 V	VDD, 3.3 V
5	CLK	SCK
6	VSS, ground	VSS, ground
7	DAT0	MISO, data out
8	DAT1	not used

Appendix H

Sources

This book does not assume access to a network. For readers who have access, this appendix lists where to find the projects it describes, their source code and documentation, and further reading. Web addresses change over time; if one no longer works, search for the project's name.

H.1 Machdyne

What	Where
Machdyne	https://machdyne.com
Zeitlos	https://zeitlos.org
Community bulletin board	telnet bbs.machdyne.com
Source code of all projects	https://github.com/machdyne

The source of each project mentioned in this book is in a repository of the same name under:

```
https://github.com/machdyne
```

Repository	Contents
tc	this book, with the tests for its examples
zeitlos	the Zeitlos SOC and operating system
basic	Machdyne BASIC, its language reference and module firmware

Repository	Contents
ms	Machdyne Scheme, before its integration into Zeitlos
sechs	the Sechs module interface standard
zwolf	the Zwölf modules, which comply with Sechs
mmod	the MMOD memory module standard
kakao	Kakao Linux for Machdyne computers
ark	an offline information collection
blaustahl	the Blaustahl F-RAM storage device
kaltstahl	the Kaltstahl F-RAM storage device
ferrit	the FERRIT F-RAM archival system
einszeit	the Einszeit one-time-pad security device
werkzeug	the Werkzeug multi-tool
zucker	Zucker, the SOC that preceded Zeitlos

H.2 Open Hardware Tools

Tool	Source
Yosys, synthesis	https://github.com/YosysHQ/yosys
nextpnr, place and route	https://github.com/YosysHQ/nextpnr
Project IceStorm, iCE40	https://github.com/YosysHQ/icestorm
Project Trellis, ECP5	https://github.com/YosysHQ/prjtrellis
Project X-Ray, Xilinx 7	https://github.com/f4pga/prjxray
Project Peppercorn, GateMate	https://github.com/YosysHQ/prjpeppercorn
Icarus Verilog, simulation	https://github.com/steveicarus/iverilog
PicoRV32, a RISC-V softcore	https://github.com/YosysHQ/picorv32
LiteX, SOC builder	https://github.com/enjoy-digital/litex

H.3 Standards

- The RISC-V instruction set specifications are published by RISC-V International at https://riscv.org.
- The Revised(4) Report on the Algorithmic Language Scheme (R4RS), 1991, defines the Scheme dialect used in this book. Later reports, such as R5RS and R7RS, are its successors.
- The C language is standardized by ISO; the examples in this book follow the 1999 standard, C99.

- Verilog is standardized as IEEE 1364; the examples follow Verilog-2005.

H.4 Further Reading

These books cover the subjects of this book in more depth. All are widely available in libraries.

- Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*. How computers work, built up from switches, for any reader.
- Sarah L. Harris and David Harris, *Digital Design and Computer Architecture, RISC-V Edition*. Digital logic, Verilog and processor design, ending with a working RISC-V processor.
- Paul Horowitz and Winfield Hill, *The Art of Electronics*. The standard practical reference on electronics.
- Harold Abelson and Gerald Jay Sussman, with Julie Sussman, *Structure and Interpretation of Computer Programs*. Programming and the nature of programming languages, taught in Scheme.
- Brian Kernighan and Dennis Ritchie, *The C Programming Language*. A concise introduction to C, by Kernighan and C's creator, Ritchie.
- Andrew Tanenbaum and Albert Woodhull, *Operating Systems: Design and Implementation*. Operating systems explained through a complete small microkernel system.
- John Kemeny and Thomas Kurtz, *Back to BASIC*. The creators of BASIC on why they made it and what it was for.